

- - 1. 原始字面量
 - 2. 指针空值类型——nullptr
 - 3. constexpr ——常量表达式
 - 3.1 const
 - 3.2 constexpr
 - 3.3 常量表达式函数
 - 3.3.1 修饰函数
 - 3.3.2 修饰模板函数
 - 3.3.3 修饰构造函数
 - 4. 自动类型推导
 - 4.1 auto
 - 4.1.1 推导规则
 - 4.1.2 auto 的限制
 - 4.1.3 auto 的应用
 - 4.2 decltype
 - 4.2.1 推导规则
 - 4.2.2 decltype的应用
 - 4.3 返回类型后置
 - 5. final和override
 - 5.1 final
 - 5.1.1 修饰函数
 - 5.1.2 修饰类
 - 5.2 override
 - 6. 模板优化
 - 6.1 模板的右尖括号
 - 6.2 默认模板参数
 - 7. using
 - 7.1 定义别名
 - 7.2 模板的别名
 - 8. 委托构造和继承构造函数
 - 8.1 委托构造
 - 8.2 继承构造函数
 - 9. 列表初始化
 - 9.1 统一的初始化
 - 9.2 列表初始化细节
 - 9.2.1 聚合体
 - 9.2.2 非聚合体
 - 9.3 std::initializer_list
 - 9.3.1 作为普通函数参数
 - 9.3.2 作为构造函数参数
 - 10. 基于范围的for循环
 - 10.1 for循环新语法
 - 10.2 使用细节
 - 10.2.1 关系型容器(键, 值)
 - 10.2.2 元素只读
 - 10.2.3 访问次数
 - 11. 可调对象包装器、绑定器
 - 11.1 可调对象
 - 11.2 可调对象包装器(适用于业务逻辑复杂的场景)
 - 11.2.1 基本用法
 - 11.2.2 作为回调函数使用

- 11.3 绑定器
- 12. Lambda表达式(匿名函数)
 - 12.1 基本用法
 - 12.2 捕获列表
 - 12.3 返回值
 - 12.4 函数本质
- 13. 右值引用
 - 13.1 右值
 - 13.2 右值引用 (引用实际效果还是取别名, 本质是指针常量。作用: 延长右值的销毁时间)
 - 13.3 性能优化
 - 13.4 && 的特性
- 14. 转移和完美转移
 - 14.1 move (返回值为右值——将亡值)
 - 14.2 forward
- 15. 智能指针
 - 15.1 shared_ptr (共享智能指针)
 - 15.1.1 初始化
 - 15.1.1.1 通过构造函数初始化
 - 15.1.1.2 通过拷贝或者移动构造函数初始化
 - 15.1.1.3 通过std::make_shared初始化
 - 15.1.1.4 通过 reset方法初始化(重置智能指针)
 - 15.1.2 获取原始指针
 - 15.1.3 指定删除器
 - 15.2 unique_ptr (独占智能指针)
 - 15.2.1 初始化
 - 15.2.2 删除器
 - 15.3 weak_ptr (弱引用智能指针)
 - 15.3.1 初始化
 - 15.3.2 其他常用方法
 - 15.3.2.1 use_count()
 - 15.3.2.2 expired()
 - 15.3.2.3 lock()
 - 15.3.2.4 reset()
 - 15.3.3 返回管理this的shared_ptr
 - 15.3.4 解决循环引用问题
- 16. chrono库
 - 16.1 时间间隔duration
 - 16.1.1 常用类成员
 - 16.1.2 类的使用
 - 16.2 时间点 time point
 - 16.3 时钟类clocks
 - 16.3.1 system_clock
 - 16.3.2 steady_clock
 - 16.3.3 high_resolution_clock
 - 16.4 转换函数
 - 16.4.1 duration_cast
 - 16.4.2 time_point_cast
- 17. 数字类型与字符串之间的转换
 - 17.1 数值转换为字符串
 - 17.2 字符串转换为数值
- 18. POD类型
 - 18.1 “平凡”类型
 - 18.2 “标准布局”类型

- 18.3 对POD类型的判断
 - 18.3.1 对“平凡类型”的判断
 - 18.3.2 对“标准布局”类型的判断
- 19. 默认函数控制 =default 与 =delete
 - 19.1 类与默认函数
 - 19.2 =default
 - 19.2.1 在类内部指定函数为默认函数
 - 19.2.2 在类外部指定函数为默认函数
 - 19.3 =delete
 - 19.3.1 禁止使用默认生成的函数
 - 19.3.2 禁止使用自定义函数
- 20. 静态断言 static_assert
 - 20.1 断言
 - 20.2 静态断言
 - 20.3 附录
- 21. noexcept
 - 21.1 异常
 - 21.1.1 基本语法
 - 21.1.2 异常接口声明
 - 21.2 noexcept
 - 21.2.1 noexcept修饰符（常用于移动构造、析构等这些不应该抛出异常的情况下）
 - 21.2.2 noexcept 操作符（模板元编程、条件性地声明函数为 noexcept。）
- 22. 类成员的快速初始化
 - 22.1 C++98 标准的类成员初始化
 - 22.2 C++11 标准的类成员初始化
 - 22.2.1 初始化类的非静态成员
 - 22.2.2 类内部赋值和初始化列表
- 23. long long 整形
 - 23.1 long long 类型
 - 23.2 扩展的整形
- 24. 扩展的 friend 语法
 - 24.1 语法改进
 - 24.2 为类模板声明友元
- 25. 强类型枚举
 - 25.1 枚举
 - 25.1.1 枚举的使用
 - 25.1.2 枚举的缺陷
 - 25.2 强类型枚举
 - 25.2.1 优势
 - 25.2.2 对原有枚举的扩展
- 26. 非受限联合体
 - 26.1 什么是非受限联合体
 - 26.2 非受限联合体的使用
 - 26.2.1 静态类型的成员
 - 26.2.2 非POD类型成员
 - 26.2.2.1 placement new
 - 26.2.2.2 自定义非受限联合体构造函数
 - 26.2.2.3 匿名的非受限联合体

1. 原始字面量

- 字面量：固定的值，数字、字符、字符串等固定值。如：H\t，存在歧义，一种是 H、\、t；另一种是 H、\t(制表符)
- 使用原始字面量消除歧义
 - 定义方式：

- R “xxx(原始字符串)xxx” ()两边的 xxx 是描述字符串（使用描述字符串时，前后字符串必须相同），转义时会自动忽略掉，所以可省略 R “()”
- 使用场景用例：

```
1 // 设置样式表
2 QString styleSheet = R"(
3     QLabel {
4         color: #333333; /* 设置标签字体颜色 */
5     }
6     QLineEdit {
7         background-color: #ffffff; /* 设置输入框背景色 */
8         border: 1px solid #cccccc; /* 设置输入框边框 */
9         border-radius: 5px; /* 设置输入框圆角 */
10    }
11    QPushButton {
12        background-color: #007bff; /* 深蓝色按钮 */
13        color: white; /* 设置按钮文字颜色 */
14        border: none; /* 移除按钮边框 */
15        border-radius: 5px; /* 设置按钮圆角 */
16        padding: 10px 20px; /* 设置按钮内边距 */
17    }
18    QPushButton:hover {
19        background-color: #00695c; /* 设置鼠标悬停时的背景色 */
20    }
21    QPushButton:pressed {
22        background-color: #3e8e41; /* 设置鼠标按下时的背景色 */
23    }
24    )";
25 this->setStyleSheet(styleSheet);
26
27 //例子
28 #include <iostream>
29 #include <string>
30
31 using namespace std;
32
33 void open_file_w()
34 {
35     string aa;
36     #if 1
37         aa = R"(D:\E\A\B)";
38     #else
39         aa = "D:\\E\\A\\B"; //如果想要正确表达，需要使用转义字符\\,否则会报错
40     }
```

```

41     #endif
42
43
44     cout << aa << endl;
45
46 }
47
48 int main() {
49
50     open_file_w();
51     return 0;
52 }
53
54 //输出: D:\E\A\B

```

Fence 1

2. 指针空值类型——nullptr

- 在 C++ 程序开发中，为了提高程序的健壮性，一般会在定义指针的同时完成初始化操作，或者在指针的指向尚未明确的情况下，**都会给指针初始化为 NULL，避免产生野指针（没有明确指向的指针，操作也这种指针极可能导致程序发生异常）**。C++98/03 标准中，将一个指针初始化为空指针的方式有 2 种：

```

1 char *ptr = 0;
2 char *ptr = NULL;           //本质上NULL宏定义成0

```

Fence 2

在底层源码中 NULL 这个宏是这样定义的：

```

1 #ifndef NULL
2     #ifdef __cplusplus
3         #define NULL 0
4     #else
5         #define NULL ((void *)0)
6     #endif
7 #endif

```

Fence 3

也就是说如果源码是 C++ 程序 **NULL 就是 0**，如果是 C 程序 NULL 表示 (void *)0。那么为什么要这样做呢？** 是由于 C++ 中，void * 类型无法隐式转换为其他类型的指针，此时使用 0 代替 ((void *)0)，*用于解决空指针的问题*。*这个 0 (0x0000 0000) 表示的就是虚拟地址空间中的 0 地址，这块地址是只读的**。

- C++ 中将 NULL 定义为字面常量 0，并不能保证在所有场景下都能很好的工作，比如，函数重载时，NULL 和 0 无法区分：

```

1 #include <iostream>
2 using namespace std;
3
4 void func(char *p)
5 {
6     cout << "void func(char *p)" << endl;
7 }

```

```

8
9 void func(int p)
10 {
11     cout << "void func(int p)" << endl;
12 }
13
14 int main()
15 {
16     func(NULL);    // 想要调用重载函数 void func(char *p)
17     func(250);    // 想要调用重载函数 void func(int p)
18
19     return 0;
20 }
21
22
23 //输出结果为
24 //void func(int p)
25 //void func(int p)

```

Fence 4

通过打印的结果可以看到，虽然调用 `func(NULL)`，最终链接到的还是 `void func(int p)` 和预期是不一样的，其实这个原因前边已经说的很明白了，在 C++ 中 **NULL 和 0 是等价的**。

- 出于兼容性的考虑，C++11 标准并没有对 `NULL` 的宏定义做任何修改，而是另其炉灶，引入了一个新的关键字 `nullptr`。**`nullptr` 专用于初始化空类型指针**，不同类型的指针变量都可以使用 `nullptr` 来初始化

```

1 #include <iostream>
2 using namespace std;
3
4 void func(char *p)
5 {
6     cout << "void func(char *p)" << endl;
7 }
8
9 void func(int p)
10 {
11     cout << "void func(int p)" << endl;
12 }
13
14 int main()
15 {
16     int*   ptr1 = nullptr;    // 隐式转换成 int* 指针类型
17     char*   ptr2 = nullptr;    // 隐式转换成 char*
18     double* ptr3 = nullptr;    // 隐式转换成 double*
19
20     func(nullptr);
21     func(250);
22     return 0;
23 }
24
25 //输出结果
26 //void func(char *p)
27 //void func(int p)

```

3. constexpr —— 常量表达式

Fence 5

3.1 const

- 在 C++11 之前只有 const 关键字，从功能上来说这个关键字有双重语义：**变量只读**，**修饰常量**，举一个简单的例子：

```
1 void func(const int num)
2 {
3     const int count = 24;
4     int array[num];           // error, num是一个只读变量，不是常量
5     int array1[count];       // ok, count是一个常量
6
7     int a1 = 520;
8     int a2 = 250;
9     const int& b = a1;
10    b = a2;                   // error
11    a1 = 1314;
12    cout << "b: " << b << endl; // 输出结果为1314
13 }
14
```

Fence 6

- 函数 void func(const int num) 的参数 num 表示这个变量是 **只读的**，但不是常量，因此使用 int array[num]; 这种方式定义一个数组，编译器是会报错的，提示 num 不可用作为常量来使用。
- const int count = 24; 中的 count 却是一个 **常量**，因此可以使用这个常量来定义一个静态数组。
- 另外，**变量只读并不等价于常量**，二者是两个概念不能混为一谈，分析一下这句测试代码 const int& b = a1;:
 - b 是一个常量的引用，所以 b 引用的变量是不能被修改的，也就是说 b = a2; 这句代码语法是错误的。
 - 在 const 对于变量 a1 是没有任何约束的，a1 的值变了 b 的值也就变了
 - 引用 b 是只读的，但是并不能保证它的值是不可改变的，也就是说它不是常量。

3.2 constexpr

- 在 C++11 中添加了一个新的关键字 constexpr，这个关键字是用来修饰常量表达式的。所谓 **常量表达式**，指的就是由多个 (≥1) 常量 (值不会改变) 组成并且在编译过程中就得到计算结果的表达式。
- 在介绍 gcc/g++ 工作流程的时候说过，C++ 程序从编写完毕到执行分为四个阶段：**预处理、编译、汇编和链接** 4 个阶段，得到可执行程序之后就可以运行了。需要额外强调的是，**常量表达式和非常量表达式的计算时机不同，非常量表达式只能在程序运行阶段计算出结果，但是常量表达式的计算往往发生在程序的编译阶段**，这可以极大提高程序的执行效率，因为表达式只需要在编译阶段计算一次，节省了每次程序运行时都需要计算一次的时间。
- 那么问题来了，编译器如何识别表达式是不是常量表达式呢？在 C++11 中添加了 constexpr 关键字之后就可以在程序中使用它来修饰常量表达式，用来提高程序的执行效率。在使用中建议将 const 和 constexpr 的功能区分开，即 **凡是表达“只读”语义的场景都使用 const，表达“常量”语义的场景都使用 constexpr**。
- 在定义常量时，const 和 constexpr 是等价的，都可以在程序的编译阶段计算出结果，例如：

```

1  const int m = f(); // 不是常量表达式，m的值只有在运行时才会获取。
2  const int i=520;   // 是一个常量表达式
3  const int j=i+1;   // 是一个常量表达式
4
5  constexpr int i=520; // 是一个常量表达式
6  constexpr int j=i+1; // 是一个常量表达式

```

- 对于 C++ 内置类型的数据，可以直接用 constexpr 修饰，**但如果是自定义的数据类型（用 struct 或者 class 实现），直接用 constexpr 修饰是不行的。**

```

1  // 此处的constexpr修饰是无效的
2  constexpr struct Test
3  {
4      int id;
5      int num;
6  };
7
8
9  struct Test
10 {
11     int id;
12     int num;
13 };
14
15 int main()
16 {
17     constexpr Test t{ 1, 2 }; //实例化对象时使用
18     constexpr int id = t.id;
19     constexpr int num = t.num;
20     // error, 不能修改常量
21     t.num += 100; //t.num += 100;的操作是错误的，对象t是一个常量，因此它的成员也是常量，常量是不能被修改的。
22     cout << "id: " << id << ", num: " << num << endl;
23
24     return 0;
25 }
26

```

Fence 8

3.3 常量表达式函数

- 为了提高 C++ 程序的执行效率，我们可以将程序中值不需要发生变化的变量定义为常量，也可以使用 constexpr 修饰函数的返回值，这种函数被称作常量表达式函数，这些函数主要包括以下几种：**普通函数/类成员函数、类的构造函数、模板函数。**

3.3.1 修饰函数

- 温馨提示：由于现在编译器版本都比较高，默认使用的 C++ 标准也比较高（大于 C++11），相关源代码请基于 C++11 标准进行测试。
- constexpr 并不能修改任意函数的返回值，使这些函数成为常量表达式函数，必须要满足以下几个条件：
 - 函数必须要有返回值，并且 return 返回的表达式必须是常量表达式。

```
1 // error, 不是常量表达式函数 (没有返回值)
2 constexpr void func1()
3 {
4     int a = 100;
5     cout << "a: " << a << endl;
6 }
7
8 // error, 不是常量表达式函数 (返回值不是常量表达式)
9 constexpr int func2()
10 {
11     int a = 100;
12     return a;
13 }
```

Fence 9

- 函数 func1() 没有返回值，不满足常量表达式函数要求
- 函数 func2() 返回值不是常量表达式，不满足常量表达式函数要求

```
1 #include <iostream>
2 using namespace std;
3
4 constexpr int func()
5 {
6     auto a = 9;
7     return a;
8 }
9
10 int main()
11 {
12     func();
13 }
```

问题 输出 调试控制台 终端

```
• dabin@Kevin:~/code$ g++ test.cpp -std=gnu++11 指定使用C++11标准
test.cpp: In function 'constexpr int func()':
test.cpp:8:1: error: body of 'constexpr' function 'constexpr int func()' not a return-statement
8 | }
  | ^
• dabin@Kevin:~/code$ g++ test.cpp 使用默认标准编译
• dabin@Kevin:~/code$ g++ -dM -E -x c++ /dev/null | grep -F __cplusplus
#define __cplusplus 201703L ← C++17 标准
• dabin@Kevin:~/code$
```

Figure 1

- 由此可见在更新的 C++ 标准里边放宽了对 constexpr 的语法限制。
- 函数在使用之前，必须有对应的定义语句。

```
1 #include <iostream>
```

```

2   using namespace std;
3
4   constexpr int func1();
5   int main()
6   {
7       constexpr int num = func1();    // error
8       return 0;
9   }
10
11  constexpr int func1()
12  {
13      constexpr int a = 100;
14      return a;
15  }
16
17
18  // 在测试程序constexpr int num = func1();中，还没有定义func1()就直接调用了，应该将
    func1()函数的定义放到main()函数的上边。

```

Fence 10

- 整个函数的函数体中，不能出现非常量表达式之外的语句（using 指令、typedef 语句以及 static_assert 断言、return 语句除外）。

```

1   // error
2   constexpr int func1()
3   {
4       constexpr int a = 100;
5       constexpr int b = 10;
6       for (int i = 0; i < b; ++i)
7       {
8           cout << "i: " << i << endl;
9       }
10      return a + b;
11  }
12
13  // ok
14  constexpr int func2()
15  {
16      using mytype = int;    //using == typedef
17      constexpr mytype a = 100;
18      constexpr mytype b = 10;
19      constexpr mytype c = a * b;
20      return c - (a + b);
21  }
22
23
24  //因为func1()是一个常量表达式函数，在函数体内部是不允许出现非常量表达式以外的操作，因此函数
    体内部的for循环是一个非法操作。

```

Fence 11

- 以上三条规则不仅对应普通函数适用，对应类的成员函数也是适用的：

```

1  class Test
2  {
3  public:
4      constexpr int func()
5      {
6          constexpr int var = 100;
7          return 5 * var;
8      }
9  };
10
11 int main()
12 {
13     Test t;
14     constexpr int num = t.func();
15     cout << "num: " << num << endl;
16
17     return 0;
18 }

```

Fence 12

3.3.2 修饰模板函数

- C++11 语法中，constexpr 可以修饰函数模板，但由于模板中类型的不确定性，因此函数模板实例化后的模板函数是否符合常量表达式函数的要求也是不确定的。**如果 constexpr 修饰的模板函数实例化结果不满足常量表达式函数的要求，则 constexpr 会被自动忽略，即该函数就等同于一个普通函数。**

```

1  #include <iostream>
2  using namespace std;
3
4  struct Person {
5      const char* name;
6      int age;
7  };
8
9  // 定义函数模板 （定义的时函数模板--重点时模板，使用时就是模板函数--重点时函数）
10 template<typename T>
11 constexpr T display(T t) {
12     return t;
13 }
14
15 int main()
16 {
17     struct Person p { "luffy", 19 };
18     //普通函数
19     struct Person ret = display(p);
20     cout << "luffy's name: " << ret.name << ", age: " << ret.age << endl;
21
22     //常量表达式函数
23     constexpr int ret1 = display(250);
24     cout << ret1 << endl;
25
26     constexpr struct Person p1 { "luffy", 19 };
27     constexpr struct Person p2 = display(p1);

```

```

28     cout << "luffy's name: " << p2.name << ", age: " << p2.age << endl;
29     return 0;
30 }
31

```

Fence 13

- 在上面示例程序中定义了一个函数模板 `display()`，但由于其返回值类型未定，因此在实例化之前无法判断其是否符合常量表达式函数的要求：
 - `struct Person ret = display(p);` 由于参数 `p` 是变量，所以实例化后的函数不是常量表达式函数，此时 `constexpr` 是无效的
 - `constexpr int ret1 = display(250);` 参数是常量，符合常量表达式函数的要求，此时 `constexpr` 是有效的
 - `constexpr struct Person p2 = display(p1);` 参数是常量，符合常量表达式函数的要求，此时 `constexpr` 是有效的

3.3.3 修饰构造函数

- 如果想用直接得到一个常量对象，也可以使用 `constexpr` 修饰一个构造函数，这样就可以得到一个常量构造函数了。常量构造函数有一个要求：**构造函数的函数体必须为空，并且必须采用初始化列表的方式为各个成员赋值。**

```

1  #include <iostream>
2  using namespace std;
3
4  struct Person {
5      constexpr Person(const char* p, int age) :name(p), age(age)
6      {
7      }
8      const char* name;
9      int age;
10 };
11
12 int main()
13 {
14     constexpr struct Person p1("luffy", 19);
15     cout << "luffy's name: " << p1.name << ", age: " << p1.age << endl;
16     return 0;
17 }

```

Fence 14

4. 自动类型推导

- 在 C++11 中增加了很多新的特性，比如可以使用 **auto（使用时，必须初始化）** 自动推导变量的类型，还能够结合 **decltype（基于已经存在的类型，进行推导）** 来表示函数的返回值。使用新的特性可以让我们写出更加简洁，更加现代的代码。

4.1 auto

- 在 C++11 之前 auto 和 static 是对应的，表示变量是自动存储的，但是非 static 的局部变量默认都是自动存储的，因此这个关键字变得非常鸡肋，在 C++11 中他们赋予了新的含义，使用这个关键字能够像别的语言一样自动推导出变量的实际类型。

4.1.1 推导规则

- C++11 中 **auto 并不代表一种实际的数据类型，只是一个类型声明的“占位符”**，auto 并不是万能的在任意场景下都能够推导出变量的实际类型，**使用 auto 声明的变量必须要进行初始化，以让编译器推导出它的实际类型**，在编译时将 auto 占位符替换为真正的类型。使用语法如下：

```
1  auto 变量名 = 变量值;
2
3
4  //例子
5  auto x = 3.14;      // x 是浮点型 double
6  auto y = 520;       // y 是整形 int
7  auto z = 'a';       // z 是字符型 char
8  auto nb;            // error, 变量必须要初始化
9  auto double nb1;    // 语法错误, 不能修改数据类型
```

Fence 15

- 不仅如此，auto 还可以和指针、引用结合起来使用也可以带上 const、volatile 限定符，在不同的场景下有对应的推导规则，规则内容如下：
 - 当变量不是指针或者引用类型时，推导的结果中不会保留 const、volatile（告诉编译器该关键字下的变量，经常变化）关键字
 - 当变量是指针或者引用类型时，推导的结果中会保留 const、volatile 关键字

```
1  // 变量d的数据类型为 int, 因此auto关键字被推导为 int类型
2  int tmp = 110;
3  auto *a = &tmp;      // 变量a的数据类型为 int*, 因此auto关键字被推导为 int类型
4  auto b = &tmp;       // 变量b的数据类型为 int*, 因此auto关键字被推导为 int*类型
5  auto &c = tmp;       // 变量c的数据类型为 int&, 因此auto关键字被推导为 int类型
6  auto d = tmp;       // 变量d的数据类型为 int, 因此auto关键字被推导为 int类型
7
8
9  // 带const限定的变量, 使用auto进行类型推导的例子
10 int tmp = 250;
11 const auto a1 = tmp; //变量a1的数据类型为 const int, 因此auto关键字被推导为 int类型
12
13 auto a2 = a1;        //变量a2的数据类型为 int, 但是a2没有声明为指针或引用因此
                       //const属性被去掉, auto被推导为 int
14 const auto &a3 = tmp; //变量a3的数据类型为 const int&, a3被声明为引用因此 const属
                       //性被保留, auto关键字被推导为 int类型
15
16 auto &a4 = a3;        //变量a4的数据类型为 const int&, a4被声明为引用因此 const属
                       //性被保留, auto关键字被推导为 const int类型
```

Fence 16

4.1.2 auto 的限制

- auto 关键字并不是万能的，在以下这些场景中是不能完成类型推导的：
 - 不能作为函数参数使用。因为只有函数调用的时候才会给函数参数传递实参，auto 要求必须要给修饰的变量赋值，因此二者矛盾。

```
1 | int func(auto a, auto b)    // error
2 | {
3 |     cout << "a: " << a << ", b: " << b << endl;
4 | }
```

Fence 17

- 不能用于类的非静态成员变量的初始化

```
1 | class Test
2 | {
3 |     auto v1 = 0;                // error
4 |     static auto v2 = 0;        // error, 类的静态非常量成员不允许在类内部直接初始
    化, 类外初始化
5 |     static const auto v3 = 10;  // ok
6 | }
```

Fence 18

- 不能使用 auto 关键字定义数组

```
1 | int func()
2 | {
3 |     int array[] = {1,2,3,4,5}; // 定义数组
4 |     auto t1 = array;           // ok, t1被推导为 int* 类型
5 |     auto t2[] = array;         // error, auto无法定义数组
6 |     auto t3[] = {1,2,3,4,5};;  // error, auto无法定义数组
7 | }
```

Fence 19

- 无法使用 auto 推导出模板参数

```
1 | template <typename T>
2 | struct Test{}
3 |
4 | int func()
5 | {
6 |     Test<double> t;
7 |     Test<auto> t1 = t;          // error, 无法推导出模板类型
8 |     return 0;
9 | }
```

Fence 20

4.1.3 auto 的应用

- 比较常用的场景：
 - 用于 STL 的容器遍历。

```
1 // c++11之前
2 #include <map>
3 int main()
4 {
5     map<int, string> person;
6     map<int, string>::iterator it = person.begin();
7     for (; it != person.end(); ++it)
8     {
9         // do something
10    }
11    return 0;
12 }
13
14
15 // C++11
16 #include <map>
17 int main()
18 {
19     map<int, string> person;
20     // 代码简化
21     for (auto it = person.begin(); it != person.end(); ++it)
22     {
23         // do something
24     }
25     return 0;
26 }
```

Fence 21

- 用于泛型编程，在使用模板的时候，很多情况下我们不知道变量应该定义为什么类型，比如下面的代码：

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class T1
6 {
7 public:
8     static int get()
9     {
10         return 10;
11     }
12 };
13
14 class T2
15 {
16 public:
```

```

17     static string get()
18     {
19         return "hello, world";
20     }
21 };
22
23 template <class A>
24 void func(void)
25 {
26     auto val = A::get();
27     cout << "val: " << val << endl;
28 }
29
30 int main()
31 {
32     func<T1>();
33     func<T2>();
34     return 0;
35 }
36

```

Fence 22

在这个例子中定义了泛型函数 func，在函数中调用了类 A 的静态方法 get()，这个函数的返回值是不能确定的，如果不使用 auto，就需要再定义一个模板参数，并且在外部调用时手动指定 get 的返回值类型，具体代码如下：

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class T1
6  {
7  public:
8      static int get()
9      {
10         return 0;
11     }
12 };
13
14 class T2
15 {
16 public:
17     static string get()
18     {
19         return "hello, world";
20     }
21 };
22
23 template <class A, typename B>          // 添加了模板参数 B
24 void func(void)
25 {
26     B val = A::get();
27     cout << "val: " << val << endl;
28 }
29

```



```

28     }
29
30     int main()
31     {
32         func<T1, int>();           // 手动指定返回值类型 -> int
33         func<T2, string>();       // 手动指定返回值类型 -> string
34         return 0;
35     }
36

```

Fence 23

4.2 decltype

- 在某些情况下，不需要或者不能定义变量，但是希望得到某种类型，这时候就可以使用 C++11 提供的 `decltype` 关键字了，它的作用是在编译器编译的时候推导出一个表达式的类型，语法格式如下：

```

1  decltype (表达式) 推导变量

```

Fence 24

- `decltype` 是“declare type”的缩写，意思是“声明类型”。`decltype` 的推导是在编译期完成的，它只是用于表达式类型的推导，并不会计算表达式的值。来看一组简单的例子：

```

1  int a = 10;
2  decltype(a) b = 99;           // b -> int
3  decltype(a+3.14) c = 52.13;   // c -> double
4  decltype(a+b*c) d = 520.1314; // d -> double

```

Fence 25

- 可以看到 `decltype` 推导的表达式可简单可复杂，在这一点上 `auto` 是做不到的，`auto` 只能推导已初始化的变量类型。

4.2.1 推导规则

- 通过上面的例子我们初步感受了一下 `decltype` 的用法，但不要认为 `decltype` 就这么简单，在它简单的背后隐藏着很多的细节，下面分三个场景依次讨论一下：
 - 表达式为普通变量或者普通表达式或者类表达式，在这种情况下，使用 `decltype` 推导出的类型和表达式的类型是一致的。

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Test
6  {
7  public:
8      string text;
9      static const int value = 110;
10 };
11
12 int main()
13 {
14     int x = 99;

```

```

15     const int &y = x;
16     decltype(x) a = x;           // 变量a被推导为 int类型
17     decltype(y) b = x;           // 变量b被推导为 const int &类型
18     decltype(Test::value) c = 0; // 变量c被推导为 const int类型
19
20     Test t;
21     decltype(t.text) d = "hello, world"; // 变量d被推导为 string类型
22
23     return 0;
24 }

```

Fence 26

- 表达式是函数调用，使用 decltype 推导出的类型和函数返回值一致。

```

1     class Test{...};
2     //函数声明
3     int func_int();           // 返回值为 int
4     int& func_int_r();        // 返回值为 int&（左值引用）
5     int&& func_int_rr();      // 返回值为 int&&（右值引用）
6
7     const int func_cint();    // 返回值为 const int
8     const int& func_cint_r(); // 返回值为 const int&
9     const int&& func_cint_rr(); // 返回值为 const int&&
10
11     const Test func_ctest(); // 返回值为 const Test
12
13     //decltype类型推导
14     int n = 100;
15     decltype(func_int()) a = 0; // 变量a被推导为 int类型
16     decltype(func_int_r()) b = n; // 变量b被推导为 int&类型
17     decltype(func_int_rr()) c = 0; // 变量c被推导为 int&&类型
18     decltype(func_cint()) d = 0; // 变量d被推导为 int类型
19     decltype(func_cint_r()) e = n; // 变量e被推导为 const int &类型
20     decltype(func_cint_rr()) f = 0; // 变量f被推导为 const int &&类型
21     decltype(func_ctest()) g = Test(); // 变量g被推导为 const Test类型
22
23     /*
24     左右值引用
25     绑定的对象类型不同：
26     左值引用（lvalue reference）绑定到左值。
27     右值引用（rvalue reference）绑定到右值。
28     修改原对象：
29     左值引用可以用于修改原对象的值。
30     右值引用主要用于移动语义，避免不必要的复制。
31     移动语义：
32     右值引用允许你从临时对象中“窃取”资源，从而提高性能。
33     通过 std::move 将左值转换为右值，可以使用右值引用。
34     */

```

Fence 27

- 函数 func_cint() 返回的是一个纯右值（在表达式执行结束后不再存在的数据，也就是临时性的数据，也就是字面量、常量），对于纯右值而言，只有类类型可以携带const、volatile限定符，除此之外需要忽略掉这两个限定符，因此推导出的变量d的类型为 int 而不是 const int。

- 表达式是一个左值（可以取地址的变量,且有表达式的左值），或者被括号()包围，使用 decltype推导出的是**表达式类型的引用**（如果有const、volatile限定符不能忽略）。

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  class Test
6  {
7  public:
8      int num;
9  };
10
11 int main() {
12     const Test obj;
13     //带有括号的表达式
14     decltype(obj.num) a = 0;           // obj.num 为类的成员访问表达式，符合场景 1，因
    此 a 的类型为 int
15     decltype((obj.num)) b = a;       // obj.num 带有括号，符合场景 3，因此 b 的类型
    为 const int&。
16     //加法表达式
17     int n = 0, m = 0;
18     decltype(n + m) c = 0;           // n+m 得到一个右值，符合场景 1，因此 c 的类型
    为 int
19     decltype(n = n + m) d = n;       // n = n+m 得到一个左值 n，符合场景 3，因此 d
    的类型为 int&
20     return 0;
21 }

```

Fence 28

4.2.2 decltype的应用

- 关于decltype的应用多出现在泛型编程中。比如我们编写一个类模板，在里边添加遍历容器的函数，操作如下：

```

1  #include <list>
2  using namespace std;
3
4  template <class T>
5  class Container
6  {
7  public:
8      void func(T& c)
9      {
10         for (m_it = c.begin(); m_it != c.end(); ++m_it)
11         {
12             cout << *m_it << " ";
13         }
14         cout << endl;
15     }
16 private:
17     ??? m_it; // 这里不能确定迭代器类型
18 };

```

```

19
20 int main()
21 {
22     const list<int> lst;
23     Container<const list<int>> obj;
24     obj.func(lst);
25     return 0;
26 }
27
28
29 // 在程序的第 17 行出了问题，关于迭代器变量一共有两种类型：只读（T:: const_iterator）和读写
    （T:: iterator），有了 decltype 就可以完美的解决这个问题了，当 T 是一个 非 const 容器得到一个
    T:: iterator，当 T 是一个 const 容器时就会得到一个 T:: const_iterator。
30
31 #include <list>
32 #include <iostream>
33 using namespace std;
34
35 template <class T>
36 class Container
37 {
38 public:
39     void func(T& c)
40     {
41         for (m_it = c.begin(); m_it != c.end(); ++m_it)
42         {
43             cout << *m_it << " ";
44         }
45         cout << endl;
46     }
47 private:
48     decltype(T().begin()) m_it; // 这里不能确定迭代器类型 T() 匿名对象的创建
49 };
50
51 int main()
52 {
53     const list<int> lst{ 1,2,3,4,5,6,7,8,9 };
54     Container<const list<int>> obj;
55     obj.func(lst);
56     return 0;
57 }
58
59
60 // decltype(T().begin())这种写法在 vs2017/vs2019 下测试可用完美运行。

```

Fence 29

4.3 返回类型后置

- 在泛型编程中，可能需要通过参数的运算来得到返回值的类型，比如下面这个场景：

```

1 #include <iostream>
2 using namespace std;
3 // R-> 返回值类型, T-> 参数 1 类型, U-> 参数 2 类型

```

```

4   template <typename R, typename T, typename U>
5   R add(T t, U u)
6   {
7       return t + u;
8   }
9
10  int main()
11  {
12      int x = 520;
13      double y = 13.14;
14      // auto z = add <decltype(x + y), int, double>(x, y);
15      auto z = add <decltype(x + y)>(x, y);    // 简化之后的写法
16      cout << "z: " << z << endl;
17      return 0;
18  }

```

Fence 30

- 关于返回值，从上面的代码可以推断出和表达式 `t+u` 的结果类型是一样的，因此可以通过通过 `decltype` 进行推导，关于模板函数的参数 `t` 和 `u` 可以通过实参自动推导出来，因此在程序中也就也可以不写。虽然通过上述方式问题被解决了，但是解决方案有点过于理想化，因为对于调用者来说，是不知道函数内部执行了什么样的处理动作的。
- 因此如果要想解决这个问题就得直接在 `add` 函数身上做文章，先来看第一种写法：

```

1   template <typename T, typename U>
2   decltype(t+u) add(T t, U u)
3   {
4       return t + u;
5   }

```

Fence 31

- 当我们在编译器中将这几行代码改出来后就直接报错了，因此 `decltype` 中的 `t` 和 `u` 都是函数参数，直接这样写相当于变量还没有定义就直接用上了，这时候变量还不存在，有点心急了。
- 在 C++11 中增加了返回类型后置语法，说明白一点就是将 `decltype` 和 `auto` 结合起来完成返回类型的推导。其语法格式如下：

```

1   // 符号 -> 后边跟随的是函数返回值的类型
2   auto func(参数 1, 参数 2, ...) -> decltype(参数表达式)

```

Fence 32

通过对上述返回类型后置语法代码的分析，得到结论：**auto 会追踪 decltype() 推导出的类型**，因此上边的 `add()` 函数可以做如下的修改：

```

1   #include <iostream>
2   using namespace std;
3
4   template <typename T, typename U>
5   // 返回类型后置语法
6   auto add(T t, U u) -> decltype(t+u)    // decltype() 的括号中不一定和返回值一样，
    也可以是能推导出返回值的类型的值
7   {
8       return t + u;

```

```

9   }
10
11  int main()
12  {
13      int x = 520;
14      double y = 13.14;
15      // auto z = add <int, double>(x, y);
16      auto z = add(x, y);      // 简化之后的写法
17      cout << "z: " << z << endl;
18      return 0;
19  }

```

Fence 33

为了进一步说明这个语法，我们再看一个例子：

```

1   #include <iostream>
2   using namespace std;
3
4   int& test(int &i)
5   {
6       return i;
7   }
8
9   double test(double &d)
10  {
11      d = d + 100;
12      return d;
13  }
14
15  template <typename T>
16  // 返回类型后置语法
17  auto myFunc(T& t) -> decltype(test(t))
18  {
19      return test(t);
20  }
21
22  int main()
23  {
24      int x = 520;
25      double y = 13.14;
26      // auto z = myFunc <int>(x);
27      auto z = myFunc(x);      // 简化之后的写法
28      cout << "z: " << z << endl;
29
30      // auto z = myFunc <double>(y);
31      auto z1 = myFunc(y);      // 简化之后的写法
32      cout << "z1: " << z1 << endl;
33      return 0;
34  }
35
36  // 在这个例子中，通过 decltype 结合返回值后置语法很容易推导出来 test(t)函数可能出现的返回值类型，并将其作用到了函数 myFunc()上。
37  // 输出结果

```

```
38 // z: 520
39 // z1: 113.14
```

Fence 34

5. final和override

5.1 final

- C++中增加了final关键字来限制某个类不能被继承，或者某个虚函数不能被重写，和Java的final关键字的功能是类似的。如果使用final修饰函数，只能修饰虚函数，并且要把final关键字放到类或者函数的后面。

5.1.1 修饰函数

- 如果使用final修饰函数，只能修饰虚函数，这样就能阻止子类重写父类的这个函数了：

```
1  class Base                                // 基类: Base
2  {
3  public:
4      virtual void test()
5      {
6          cout << "Base class...";
7      }
8  };
9
10 class Child : public Base                  // 子类: Child
11 {
12 public:
13     void test() final
14     {
15         cout << "Child class...";
16     }
17 };
18
19 class GrandChild : public Child            // 孙子类: GrandChild
20 {
21 public:
22     // 语法错误，不允许重写
23     void test()
24     {
25         cout << "GrandChild class...";
26     }
27 };
28
29
30 //test()是基类中的一个虚函数，在子类中重写了这个方法，但是不希望孙子类中继续重写这个方法了，因此在子类中将 test()方法标记为 final，孙子类中对这个方法就只有使用的份了。
```

Fence 35

5.1.2 修饰类

- 使用final关键字修饰过的类是不允许被继承的，也就是说这个类不能有派生类。

```
1  class Base
2  {
3  public:
4      virtual void test()
5      {
6          cout << "Base class...";
7      }
8  };
9
10 class Child final: public Base
11 {
12 public:
13     void test()
14     {
15         cout << "Child class...";
16     }
17 };
18
19 // error, 语法错误
20 class GrandChild : public Child
21 {
22 public:
23 };
24
25
26 // Child 类是被 final 修饰过的，因此 Child 类不允许有派生类 GrandChild 类的继承是非法的，
    Child 是个断子绝孙的类。
```

Fence 36

5.2 override

- override关键字**确保在派生类中声明的重写函数与基类的虚函数有相同的签名，同时也明确表明将会重写基类的虚函数**，这样就可以保证重写的虚函数的正确性，也提高了代码的可读性，和final一样这个关键字要写到方法的后面。使用方法如下：

```
1  class Base
2  {
3  public:
4      virtual void test()
5      {
6          cout << "Base class...";
7      }
8  };
9
10 class Child : public Base
11 {
12 public:
13     void test() override
```



```

14     {
15         cout << "Child class...";
16     }
17 };
18
19 class GrandChild : public Child
20 {
21 public:
22     void test() override
23     {
24         cout << "Child class...";
25     }
26 };
27
28 // 上述代码中第 13 行和第 22 行就是显示指定了要重写父类的 test()方法，使用了 override 关键字
    之后，假设在重写过程中因为误操作，写错了函数名或者函数参数或者返回值编译器都会提示语法错误，提高了
    程序的正确性，降低了出错的概率。
29
30 // 多态条件：
31 // 1.有继承关系
32 // 2.子类重写父类虚函数
33 // 3.父类指针或者引用指向子类函数

```

Fence 37

6. 模板优化

6.1 模板的右尖括号

- 在泛型编程中，模板实例化有一个非常繁琐的地方，那就是**连续的两个右尖括号 (>>) 会被编译器解析成右移操作符**，而不是模板参数表的结束。我们先来看一段关于容器遍历的代码，在创建的类模板Base中提供了遍历容器的操作函数traversal():

```

1 // test.cpp
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5
6 template <typename T>
7 class Base
8 {
9 public:
10     void traversal(T& t)
11     {
12         auto it = t.begin();
13         for (; it != t.end(); ++it)
14         {
15             cout << *it << " ";
16         }
17         cout << endl;
18     }
19 };
20
21

```

```

22     int main()
23     {
24         vector<int> v{ 1,2,3,4,5,6,7,8,9 };
25         Base<vector<int>>> b;
26         b.traversal(v);
27
28         return 0;
29     }
30

```

Fence 38

如果使用C++98/03标准(在>>中间添加空格解决)来编译上边的这段代码，就会得到如下的错误提示：

```

1  test.cpp:25:20: error: '>>' should be '> >' within a nested template argument list
2      Base<vector<int>>> b;

```

Fence 39

- 根据错误提示中描述模板的两个右尖括之间需要添加空格，这样写起来就非常的麻烦，**C++11改进了编译器的解析规则，尽可能地将多个右尖括号(>)解析成模板参数结束符**，方便我们编写模板相关的代码。
- 上面的这段代码，在支持C++11的编译器中编译是没有任何问题的，如果使用g++直接编译需要加参数-std=c++11：

```

1  $ g++ test.cpp -std = c++11 -o app

```

Fence 40

6.2 默认模板参数

- 在C++98/03标准中，类模板可以有默认模板参数：

```

1  #include <iostream>
2  using namespace std;
3
4  template<typename T=int, T t=520>
5  class Test
6  {
7  public:
8      void print()
9      {
10         cout << "current value: " << t << endl;
11     }
12 };
13
14 int main()
15 {
16     Test<> t;
17     t.print();
18
19     Test<int, 1024> t1;
20     t1.print();
21
22     return 0;
23 }

```

- 但是不支持函数的默认模板参数，在C++11中添加了对函数模板默认参数的支持：

```

1  #include <iostream>
2  using namespace std;
3
4  template <typename T=int>    // C++98/03 不支持这种写法，C++11 中支持这种写法
5  void func(T t)
6  {
7      cout << "current value: " << t << endl;
8  }
9
10 int main()
11 {
12     func(100);
13     return 0;
14 }
```

Fence 42

- 通过上面的例子可以得到如下结论：当所有模板参数都有默认参数时，函数模板的调用如同一个普通函数。但对于类模板而言，哪怕所有参数都有默认参数，在使用时也必须模板名后跟随<>来实例化。

另外：函数模板的默认模板参数在使用规则上和其他的默认参数也有一些不同，它没有必须写在参数表最后的限制。这样当默认模板参数和模板参数自动推导结合起来时，书写就显得非常灵活了。我们可以指定函数模板中的一部分模板参数使用默认参数，另一部分使用自动推导，比如下面的例子：

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  template <typename R = int, typename N>
6  R func(N arg)
7  {
8      return arg;
9  }
10
11 int main()
12 {
13     auto ret1 = func(520);
14     cout << "return value-1: " << ret1 << endl;
15
16     auto ret2 = func <double>(52.134);
17     cout << "return value-2: " << ret2 << endl;
18
19     auto ret3 = func <int>(52.134);
20     cout << "return value-3: " << ret3 << endl;
21
22     auto ret4 = func <char, int>(100);
23     cout << "return value-4: " << ret4 << endl;
24
25     return 0;
26 }
```

测试代码输出的结果为:

```
1 | return value-1: 520
2 | return value-2: 52.134
3 | return value-3: 52
4 | return value-4: d
```

- 根据得到的日志输出，分析一下示例代码中调用的模板函数：
 - auto ret = func(520);
 - 函数返回值类型使用了默认的模板参数，函数的参数类型是自动推导出来的为int类型。
 - auto ret1 = func(52.134);
 - 函数的返回值指定为double类型，函数参数是通过实参推导出来的，为double类型
 - auto ret3 = func(52.134);
 - 函数的返回值指定为int类型，函数参数是通过实参推导出来的，为double类型
 - auto ret4 = func<char, int>(100);
 - 函数的参数为指定为int类型，函数返回值指定为char类型，不需要推导
- 当默认模板参数和模板参数自动推导同时使用时（优先级从高到低）：
 - 如果指定了类型，就使用指定类型
 - 如果可以推导出参数类型则使用推导出的类型
 - 如果函数模板无法推导出参数类型，那么编译器会使用默认模板参数
 - 如果无法推导出模板参数类型并且没有设置默认模板参数，编译器就会报错。
- 看一下下面的例子：

```
1 | #include <iostream>
2 | #include <string>
3 | using namespace std;
4 |
5 | // 函数模板定义
6 | template <typename T, typename U = char>
7 | void func(T arg1 = 100, U arg2 = 100)
8 | {
9 |     cout << "arg1: " << arg1 << ", arg2: " << arg2 << endl;
10 | }
11 |
12 | int main()
13 | {
14 |     // 模板函数调用
15 |     func('a');
16 |     func(97, 'a');
17 |     // func();    //编译报错
18 |     return 0;
19 | }
20 |
21 | // 程序输出的结果为：
```

```
22 // arg1: a, arg2: d
23 // arg1: 97, arg2: a
```

Fence 45

分析一下调用的模板函数func():

- func('a'): 参数T被自动推导为char类型, U使用的默认模板参数为char类型
- func(97, 'a');: 参数T被自动推导为int类型, U使用推导出的类型为char
- func();: 参数T没有指定默认模板类型, 并且无法自动推导, 编译器会直接报错
 - **模板参数类型的自动推导是根据模板函数调用时指定的实参进行推断的, 没有实参则无法推导**
 - **模板参数类型的自动推导不会参考函数模板中指定的默认参数。**

7. using

- 在C++中using用于声明命名空间, 使用命名空间也可以防止命名冲突。在程序中声明了命名空间之后, 就可以直接使用命名空间中的定义的类了。在C++11中赋予了using新的功能, 让C++变得更年轻, 更灵活。

7.1 定义别名

- 在C++中可以通过typedef重定义一个类型, 语法格式如下:

```
1 typedef 旧的类型名 新的类型名;
2 // 使用举例
3 typedef unsigned int uint_t;
```

Fence 46

- 被重定义的类型并不是一个新的类型, 仅仅只是原有的类型取了一个新的名字。和以前的声明语句一样, 这里的声明符也可以包含类型修饰, 从而也能由基本数据类型构造出复合类型来。C++11中规定了一种新的方法, 使用别名声明(alias declaration)来定义类型的别名, 即使用using。
- 在使用的时候, 关键字using作为别名声明的开始, 其后紧跟别名和等号, 其作用是把等号左侧的名字规定成等号右侧类型的别名。**类型别名和类型的名字等价, 只要是类型的名字能出现的地方, 就能使用类型别名。使用typedef定义的别名和使用using定义的别名在语义上是等效的。**
- 使用using定义别名的语法格式是这样的:

```
1 using 新的类型 = 旧的类型;
2 // 使用举例
3 using uint_t = int;
```

Fence 47

- 通过using和typedef的语法格式可以看到二者的使用没有太大的区别, 假设我们定义一个函数指针, using的优势就能凸显出来了, 看一下下面的例子:

```

1 // 使用 typedef 定义函数指针
2 typedef int(*func_ptr)(int, double);
3
4 // 使用 using 定义函数指针
5 using func_ptr1 = int(*)(int, double);
6
7 int myfunc (int a, string b)
8 //使用
9 func_ptr f = myfunc;
10 func_ptr1 f1 = myfunc;
11 f(10, "nihao");
12 f1(10, "nih");
13 (*f)(10, "nih"); //相当于直接使用了函数本身

```

Fence 48

- 如果不是特别熟悉函数指针与typedef，第一眼很难看出func_ptr其实是一个别名，其本质是一个函数指针，指向的函数返回类型是int，函数参数有两个分别是int，double类型。
- 使用using定义函数指针别名的写法看起来就非常直观了，把别名的名字强制分离到了左边，而把别名对应的实际类型放在了右边，比较清晰，可读性比较好。

7.2 模板的别名

- 使用typedef重定义类似很方便，但是它有一点限制，比如无法重定义一个模板，比如我们需要一个固定以int类型为key的map，它可以和很多类型的value值进行映射，如果使用typedef这样直接定义就非常麻烦：

```

1 typedef map <int, string> m1;
2 typedef map <int, int> m2;
3 typedef map <int, double> m3;

```

Fence 49

- 在这种情况下我们就不自觉的想到了模板：

```

1 template <typename T>
2 typedef map <int, T> type; // error, 语法错误

```

Fence 50

- 使用typename不支持给模板定义别名，这个简单的需求仅通过typedef很难办到，需要添加一个外敷类：

```

1 #include <iostream>
2 #include <functional>
3 #include <map>
4 using namespace std;
5
6 template <typename T>
7 // 定义外敷类
8 struct MyMap
9 {
10     typedef map <int, T> type;
11 };
12
13 int main(void)

```

```

14 {
15     MyMap <string>:: type m;
16     m.insert(make_pair(1, "luffy"));
17     m.insert(make_pair(2, "ace"));
18
19     MyMap <int>:: type m1;
20     m1.insert(1, 100);
21     m1.insert(2, 200);
22
23     return 0;
24 }
25

```

Fence 51

- 通过上边的例子可以直观的感觉到，需求简单但是实现起来并不容易。在C++11中，新增了一个特性就是可以通过使用using来为一个模板定义别名，对于上面的需求可以写成这样：

```

1 template <typename T>
2 using mymap = map <int, T>;

```

Fence 52

完整的示例代码如下：

```

1 #include <iostream>
2 #include <functional>
3 #include <map>
4 using namespace std;
5
6 template <typename T>
7 using mymap = map <int, T>;
8
9 int main(void)
10 {
11     // map 的 value 指定为 string 类型
12     mymap <string> m;
13     m.insert(make_pair(1, "luffy"));
14     m.insert(make_pair(2, "ace"));
15
16     // map 的 value 指定为 int 类型
17     mymap <int> m1;
18     m1.insert(1, 100);
19     m1.insert(2, 200);
20
21     return 0;
22 }
23

```

Fence 53

- 上面的例子中通过使用using给模板指定别名，就可以基于别名非常方便的给value指定相应的类型，这样使编写的程序变得更加灵活，看起来也更加简洁一些。

最后在强调一点：using语法和typedef一样，并不会创建出新的类型，它们只是给某些类型定义了新的别名。using相较于typedef的优势在于定义函数指针别名时看起来更加直观，并且可以给模板定义别名。

8. 委托构造和继承构造函数

8.1 委托构造

- 委托构造函数允许使用同一个类中的一个构造函数调用其它的构造函数，从而简化相关变量的初始化。下面举例说明：

```
1  #include <iostream>
2  using namespace std;
3
4  class Test
5  {
6  public:
7      Test() {};
```

```
8      Test(int max)
9      {
10         this-> m_max = max > 0 ? max : 100;
11     }
12
13     Test(int max, int min)
14     {
15         this-> m_max = max > 0 ? max : 100;           // 冗余代码
16         this-> m_min = min > 0 && min < max ? min : 1;
17     }
18
19     Test(int max, int min, int mid)
20     {
21         this-> m_max = max > 0 ? max : 100;           // 冗余代码
22         this-> m_min = min > 0 && min < max ? min : 1; // 冗余代码
23         this-> m_middle = mid < max && mid > min ? mid : 50;
24     }
25
26     int m_min;
27     int m_max;
28     int m_middle;
29 };
30
31 int main()
32 {
33     Test t(90, 30, 60);
34     cout << "min: " << t.m_min << ", middle: "
35         << t.m_middle << ", max: " << t.m_max << endl;
36     return 0;
37 }
```

Fence 54

- 在上面的程序中有三个构造函数，但是这三个函数中都有重复的代码，在C++11之前构造函数是不能调用构造函数的，加入了委托构造之后，我们就可以轻松地完成代码的优化了：


```

1  #include <iostream>
2  using namespace std;
3
4  class Test
5  {
6  public:
7      Test() {}
8      Test(int max)
9      {
10         this->m_max = max > 0 ? max : 100;
11     }
12
13     Test(int max, int min): Test(max)
14     {
15         this->m_min = min > 0 && min < max ? min : 1;
16     }
17
18     Test(int max, int min, int mid): Test(max, min)
19     {
20         this->m_middle = mid < max && mid > min ? mid : 50;
21     }
22
23     int m_min;
24     int m_max;
25     int m_middle;
26 };
27
28 int main()
29 {
30     Test t(90, 30, 60);
31     cout << "min: " << t.m_min << ", middle: "
32         << t.m_middle << ", max: " << t.m_max << endl;
33     return 0;
34 }

```

Fence 55

- 在修改之后的代码中可以看到，重复的代码全部没有了，并且在一个构造函数中调用了其他的构造函数用于相关数据的初始化，相当于是一个链式调用。在使用委托构造函数的时候还需要注意一些几个问题：
 - 这种链式的构造函数调用不能形成一个闭环（死循环），否则会在运行期抛异常。
 - 如果要进行多层构造函数的链式调用，建议将构造函数的调用的写在初始列表中而不是函数体内部，否则编译器会提示形参的重复定义。

```

1  Test(int max)
2  {
3      this-> m_max = max > 0 ? max : 100;
4  }
5
6  Test(int max, int min)
7  {
8      Test(max); // error, 此处编译器会报错, 提示形参 max 被重复定义
9      this-> m_min = min > 0 && min < max ? min : 1;
10 }
11

```

Fence 56

- 初始化列表中调用了代理构造函数初始化某个类成员变量之后，就不能在初始化列表中再次初始化这个变量了。

```

1  // 错误, 使用了委托构造函数就不能再次 m_max 初始化了
2  Test(int max, int min) : Test(max), m_max(max)
3  {
4      this-> m_min = min > 0 && min < max ? min : 1;
5  }

```

Fence 57

8.2 继承构造函数

- C++11中提供的继承构造函数可以让派生类直接使用基类的构造函数，而无需自己再写构造函数，尤其是在基类有很多构造函数的情况下，可以极大地简化派生类构造函数的编写。先来看没有继承构造函数之前的处理方式：

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Base
6  {
7  public:
8      Base(int i) : m_i(i) {}
9      Base(int i, double j) : m_i(i), m_j(j) {}
10     Base(int i, double j, string k) : m_i(i), m_j(j), m_k(k) {}
11
12     int m_i;
13     double m_j;
14     string m_k;
15 };
16
17 class Child : public Base
18 {
19 public:
20     Child(int i) : Base(i) {}
21     Child(int i, double j) : Base(i, j) {}
22     Child(int i, double j, string k) : Base(i, j, k) {}

```

```

23     };
24
25     int main()
26     {
27         Child c(520, 13.14, "i love you");
28         cout << "int: " << c.m_i << ", double: "
29             << c.m_j << ", string: " << c.m_k << endl;
30         return 0;
31     }

```

Fence 58

- 通过测试代码可以看出，在子类中初始化从基类继承的类成员，需要在子类中重新定义和基类一致的构造函数，这是非常繁琐的，C++11中通过添加继承构造函数这个新特性完美的解决了这个问题，使得代码更加精简。
- 继承构造函数的使用方法是这样的：通过使用 `using 类名::构造函数名`（其实类名和构造函数名是一样的）来声明使用基类的构造函数，这样子类中就可以不定义相同的构造函数了，直接使用基类的构造函数来构造派生类对象。

```

1     #include <iostream>
2     #include <string>
3     using namespace std;
4
5     class Base
6     {
7     public:
8         Base(int i) : m_i(i) {}
9         Base(int i, double j) : m_i(i), m_j(j) {}
10        Base(int i, double j, string k) : m_i(i), m_j(j), m_k(k) {}
11
12        int m_i;
13        double m_j;
14        string m_k;
15    };
16
17    class Child : public Base
18    {
19    public:
20        using Base::Base;
21    };
22
23    int main()
24    {
25        Child c1(520, 13.14);
26        cout << "int: " << c1.m_i << ", double: " << c1.m_j << endl;
27        Child c2(520, 13.14, "i love you");
28        cout << "int: " << c2.m_i << ", double: "
29            << c2.m_j << ", string: " << c2.m_k << endl;
30        return 0;
31    }
32
33
34

```

- 在修改之后的子类中，没有添加任何构造函数，而是添加了using Base::Base;这样就可以在子类中直接继承父类的所有的构造函数，通过他们去构造子类对象了。
- 另外如果在子类中隐藏了父类中的同名函数，也可以通过using的方式在子类中使用基类中的这些父类函数：

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Base
6  {
7  public:
8      Base(int i) : m_i(i) {}
9      Base(int i, double j) : m_i(i), m_j(j) {}
10     Base(int i, double j, string k) : m_i(i), m_j(j), m_k(k) {}
11
12     void func(int i)
13     {
14         cout << "base class: i = " << i << endl;
15     }
16
17     void func(int i, string str)
18     {
19         cout << "base class: i = " << i << ", str = " << str << endl;
20     }
21
22     int m_i;
23     double m_j;
24     string m_k;
25 };
26
27 class Child : public Base
28 {
29 public:
30     using Base::Base;
31     using Base::func;
32     void func()
33     {
34         cout << "child class: i'am luffy!!!" << endl;
35     }
36 };
37
38 int main()
39 {
40     Child c(250);
41     c.func();
42     c.func(19);
43     c.func(19, "luffy");
44     return 0;
45 }
46
47 // 输出结果
```

```

48 // child class: i'am luffy!!!
49 // base class: i = 19
50 // base class: i = 19, str = luffy

```

Fence 60

- 子类中的func()函数隐藏了基类中的两个func()因此默认情况下通过子类对象只能调用无参的func(), 在上面的子类代码中添加了 `using Base::func;` 之后, 就可以通过子类对象直接调用父类中被隐藏的带参func()函数了。(子类重载了父类方法, 又想使用父类的方法, 可以通过这种方式)

9. 列表初始化

- 关于C++中的变量, 数组, 对象等都有不同的初始化方法, 在这些繁琐的初始化方法中没有任何一种方式适用于所有的情况。为了统一初始化方式, 并且让初始化行为具有确定的效果, 在C++11中提出了列表初始化的概念。

9.1 统一的初始化

- 在C++98/03中, 对应普通数组和可以直接进行内存拷贝 (memcpy()) 的对象是可以使用列表初始化来初始化数据的

```

1 // 数组的初始化
2 int array [] = { 1,3,5,7,9 };
3 double array1 [3] = { 1.2, 1.3, 1.4 };
4
5 // 对象的初始化
6 struct Person
7 {
8     int id;
9     double salary;
10 }zhang{ 1, 3000 };

```

Fence 61

- 在C++11中, 列表初始化变得更加灵活了, 来看一下下面这段初始化类对象的代码:

```

1 #include <iostream>
2 using namespace std;
3
4 class Test
5 {
6 public:
7     Test(int) {}
8 private:
9     Test(const Test &);
10 };
11
12 int main(void)
13 {
14     Test t1(520);
15     Test t2 = 520; // 自动调用 Test (int) 构造函数, 进行隐式转换
16     Test t3 = { 520 };
17     Test t4{ 520 };

```

```

18     int a1 = { 1314 };
19     int a2{ 1314 };
20     int arr1 [] = { 1, 2, 3 };
21     int arr2 []{ 1, 2, 3 };
22     return 0;
23 }
24

```

Fence 62

- 具体地来解读一下上面代码中使用的各种初始化方式：
 - t1：最中规中矩的初始化方式，通过提供的带参构造进行对象的初始化
 - t2：语法错误，因为提供的拷贝构造函数是私有的。如果拷贝构造函数是公共的，520会通过隐式类型转换被Test(int)构造成为一个匿名对象，然后再通过对这个匿名对象进行拷贝构造得到t2（这个错误在VS中不会出现，在Linux中使用g++编译会提示描述的这个错误，截图如下。）

```

4 #include <iostream>
5 using namespace std;
6
7 class Test
8 {
9 public:
10     Test(int)
11     {
12         cout << "construct function..." << endl;
13     }
14 private:
15     Test(const Test&)
16     {
17         cout << "copy construct function..." << endl;
18     }
19 };
20
21 int main()
22 {
23     Test t1(1314);
24     Test t2 = 520;
25 }

```

NORMAL init.cpp cpp 3% 1/32 1 [10]tra...
 "init.cpp" [dos] 32L, 528C written
 robin@OS:~\$ g++ init.cpp -std=c++11
 init.cpp: In function 'int main()':
 init.cpp:15:5: error: 'Test::Test(const Test&)' is private
 Test(const Test&)
 init.cpp:24:15: error: within this context
 Test t2 = 520;
 init.cpp:10:5: note: after user-defined conversion: Test::Test(int)
 Test(int)

Figure 2

- t3和t4：使用了C++11的初始化方式来初始化对象，效果和t1的方式是相同的。
 - 在初始时，{}前面的等号是否书写对初始化行为没有任何影响。
 - t3虽然使用了等号，但是它仍然是列表初始化，因此私有的拷贝构造对它没有任何影响。
- t1、arr1和t2、arr2：这两个是基础数据类型的列表初始化方式，可以看到，和对象的初始化方式是统一的。
- t3、t4、a2、arr2的写法，是C++11中新添加的语法格式，使用这种方式可以直接在变量名后边跟上初始化列表，来进行变量或者对象的初始化。
- 既然使用列表初始化可以对普通类型以及对象进行直接初始化，那么在使用 new 操作符创建新对象的时候可以使用列表初始化进行对象的初始化吗？答案是肯定的，来看下面的例子：

```

1     int * p = new int{520};
2     double b = double{52.134};
3     int * array = new int [3]{1,2,3};

```

- 指针p指向了一个new操作符返回的内存，通过列表初始化将内存数据初始化为520
- 变量b是对匿名对象使用列表初始化之后，再进行拷贝初始化。
- 数组array在堆上动态分配了一块内存，通过列表初始化的方式直接完成了多个元素的初始化。
- 除此之外，列表初始化还可以直接用在函数返回值上：

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Person
6  {
7  public:
8      Person(int id, string name)
9      {
10         cout << "id: " << id << ", name: " << name << endl;
11     }
12 };
13
14 Person func()
15 {
16     return { 9527, "华安" };
17 }
18
19 int main(void)
20 {
21     Person p = func();
22     return 0;
23 }
24
```

Fence 64

- 代码中的return { 9527, "华安" };就相当于return (9527, "华安");，直接返回了一个匿名对象。通过上面的几个例子可以看出在C++11使用列表初始化是非常便利的，它统一了各种对象的初始化方式，而且还让代码的书写更加简单清晰。

9.2 列表初始化细节

9.2.1 聚合体

- 在C++11中，列表初始化的使用范围被大大增强了，但是一些模糊的概念也随之而来，在前面的例子可以得知，列表初始化可以用于自定义类型的初始化，但是对于一个自定义类型，列表初始化可能有两种执行结果：

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  struct T1
6  {
7      int x;
8      int y;
9  }a = { 123, 321 };
```

```

10
11 struct T2
12 {
13     int x;
14     int y;
15     T2(int, int) : x(10), y(20) {}
16 }b = { 123, 321 };
17
18 int main(void)
19 {
20     cout << "a.x: " << a.x << ", a.y: " << a.y << endl;
21     cout << "b.x: " << b.x << ", b.y: " << b.y << endl;
22     return 0;
23 }
24
25 /*
26 结果:
27 a.x: 123, a.y: 321
28 b.x: 10, b.y: 20
29 */

```

Fence 65

- 在上边的程序中都是用列表初始化的方式对对象进行了初始化，但是得到结果却不同，对象b并没有被初始化列表中的数据初始化，这是为什么呢？
 - 对象a是对一个自定义的聚合类型进行初始化，它将以拷贝的形式使用初始化列表中的数据来初始化T1结构体中的成员。
 - 在结构体T2中自定义了一个构造函数，因此实际的初始化是通过这个构造函数完成的。
- 如果使用列表初始化对对象初始化时，还需要判断这个对象对应的类型是不是一个聚合体，如果是初始化列表中的数据就会拷贝到对象中
- 聚合体
 - 普通数组本身可以看做是一个聚合类型

```

1 int x [] = {1,2,3,4,5,6};
2 double y [3][3] = {
3     {1.23, 2.34, 3.45},
4     {4.56, 5.67, 6.78},
5     {7.89, 8.91, 9.99},
6 };
7 char carry [] = {'a', 'b', 'c', 'd', 'e', 'f'};
8 std::string saray [] = {"hello", "world", "nihao", "shijie"};

```

Fence 66

- 满足以下条件的类（class、struct、union）可以被看做是一个聚合类型：
 - 无用户自定义的构造函数。
 - 无私有或保护的静态数据成员。
 - 场景1: 类中有私有成员, 无法使用列表初始化进行初始化


```

1 struct T1
2 {
3     int x;
4     long y;
5 protected:
6     int z;          // 需要构造函数来使用初始化列表
7 }t{ 1, 100, 2};      // error, 类中有私有成员, 无法使用初始化列表初始化

```

- 场景2: 类中有非静态成员可以通过列表初始化进行初始化, 但它不能初始化静态成员变量。

```

1 struct T2
2 {
3     int x;
4     long y;
5 protected:
6     static int z;
7 }t{ 1, 100, 2};      // error 静态成员 z 需要类外初始化

```

Fence 68

结构体中的静态变量 `z` 不能使用列表初始化进行初始化, 它的初始化遵循静态成员的初始化方式。

```

1 struct T2
2 {
3     int x;
4     long y;
5 protected:
6     static int z;
7 }t{ 1, 100};          // ok
8 // 静态成员的初始化
9 int T2::z = 2;

```

Fence 69

- 无基类。
- 无虚函数。
- 类中不能有使用 `{}` 和 `=` 直接初始化的非静态数据成员 (从 `c++14` 开始就支持了)。

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 struct T2
6 {
7     int x;
8     long y;
9 protected:
10     static int z;
11 }t1{ 1, 100 };        // ok
12 // 静态成员的初始化

```

```

13     int T2:: z = 2;
14
15     struct T3
16     {
17         int x;
18         double y = 1.34;
19         int z [3]{1,2,3};
20     };
21
22     int main(void)
23     {
24         T3 t{520, 13.14, {6,7,8}};    // error, c++11 不支持, 从
                                         c++14 开始就支持了
25         return 0;
26     }

```

Fence 70

9.2.2 非聚合体

- 对于聚合类型的类可以直接使用列表初始化进行对象的初始化, 如果不满足聚合条件还想使用列表初始化其实也是可以的, 需要在类的内部自定义一个构造函数, 在构造函数中使用初始化列表对类成员变量进行初始化:

```

1     #include <iostream>
2     #include <string>
3     using namespace std;
4
5     struct T1
6     {
7         int x;
8         double y;
9         // 在构造函数中使用初始化列表初始化类成员
10        T1(int a, double b, int c) : x(a), y(b), z(c){}
11        virtual void print()
12        {
13            cout << "x: " << x << ", y: " << y << ", z: " << z << endl;
14        }
15    private:
16        int z;
17    };
18
19    int main(void)
20    {
21        T1 t{ 520, 13.14, 1314 };    // ok, 基于构造函数使用初始化列表初始化类成员
22        t.print();
23        return 0;
24    }

```

Fence 71

- 可以看到, T1并非一个聚合类型, 因为它有一个Private的非静态成员。但是尽管T2有一个非聚合类型的非静态成员t1, T2依然是一个聚合类型, 可以直接使用列表初始化的方式进行初始化。

- 最后强调一下t2对象的初始化过程，对于非聚合类型的成员t1做初始化的时候，可以直接写一对空的大括号{}，这相当于调用是T1的无参构造函数
- 对于一个聚合类型，使用列表初始化相当于对其中的每个元素分别赋值，而对于非聚合类型，则需要先自定义一个合适的构造函数，此时使用列表初始化将会调用它对应的构造函数。

9.3 std::initializer_list

- 在C++的STL容器中，可以进行任意长度的数据的初始化，使用初始化列表也只能进行固定参数的初始化，如果想要做到和STL一样有任意长度初始化的能力，可以使用std::initializer_list这个轻量级的类模板来实现。
- 先来介绍一下这个类模板的一些特点：
 - 它是一个轻量级的容器类型，内部定义了迭代器iterator等容器必须的概念，遍历时得到的迭代器是只读的。
 - 对于std::initializer_list而言，它可以接收任意长度的初始化列表，但是要求元素必须是同种类型T
 - 在std::initializer_list内部有三个成员接口：size(), begin(), end()。
 - std::initializer_list对象只能被整体初始化或者赋值。

9.3.1 作为普通函数参数

- 如果想要自定义一个函数并且接收任意个数的参数（变参函数），只需要将函数参数指定为std::initializer_list，使用初始化列表{}作为实参进行数据传递即可。

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  void traversal(std:: initializer_list <int> a)
6  {
7      for (auto it = a.begin(); it != a.end(); ++it)
8      {
9          cout << *it << " ";
10     }
11     cout << endl;
12 }
13
14 int main(void)
15 {
16     initializer_list <int> list;
17     cout << "current list size: " << list.size() << endl;
18     traversal(list);
19
20     list = { 1,2,3,4,5,6,7,8,9,0 };
21     cout << "current list size: " << list.size() << endl;
22     traversal(list);
23     cout << endl;
24
25     list = { 1,3,5,7,9 };
26     cout << "current list size: " << list.size() << endl;
27     traversal(list);
28     cout << endl;
29

```

```

30 ///////////////////////////////////////////////////
31 /////////////////////////////////////////////////// 直接通过初始化列表传递数据 ///////////////////////////////////////////////////
32 ///////////////////////////////////////////////////
33 traversal({ 2, 4, 6, 8, 0 });
34 cout << endl;
35
36 traversal({ 11,12,13,14,15,16 });
37 cout << endl;
38
39
40 return 0;
41 }
42
43 /*
44 结果:
45 current list size: 0
46
47 current list size: 10
48 1 2 3 4 5 6 7 8 9 0
49
50 current list size: 5
51 1 3 5 7 9
52
53 2 4 6 8 0
54
55 11 12 13 14 15 16
56 */

```

Fence 72

- **std::initializer_list**拥有一个无参构造函数，因此，它可以直接定义实例，此时将得到一个空的 **std::initializer_list**，因为在遍历这种类型的容器的时候得到的是一个只读的迭代器，因此我们不能修改里面的数据，只能通过值覆盖的方式进行容器内部数据的修改。虽然如此，在效率方面也无需担心，**std::initializer_list**的效率是非常高的，它的内部并不负责保存初始化列表中元素的拷贝，仅仅存储了初始化列表中元素的引用。

9.3.2 作为构造函数参数

- 自定义的类如果在构造对象的时候想要接收任意个数的实参，可以给构造函数指定为std::initializer_list类型，在自定义类的内部还是使用容器来存储接收的多个实参。（Qt框架中使用的比较多）

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  class Test
7  {
8  public:
9      Test(std:: initializer_list <string> list)
10     {
11         for (auto it = list.begin(); it != list.end(); ++it)
12         {
13             cout << *it << " ";

```

```

14         m_names.push_back(*it);
15     }
16     cout << endl;
17 }
18 private:
19     vector<string> m_names;
20 };
21
22 int main(void)
23 {
24     Test t({ "jack", "lucy", "tom" });
25     Test t1({ "hello", "world", "nihao", "shijie" });
26     return 0;
27 }
28
29
30 /*
31 结果
32 jack lucy tom
33 hello world nihao shijie
34 */

```

Fence 73

10. 基于范围的for循环

- 在C++98/03中，不同的容器和数组遍历的方式不尽相同，写法不统一，也不够简洁，而C++11基于范围的for循环可以以简洁、统一的方式来遍历容器和数组，用起来更方便了。

10.1 for循环新语法

- 在介绍新语法之前，先来看一个使用迭代器遍历容器的例子：

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main()
6  {
7      vector<int> t{ 1,2,3,4,5,6 };
8      for (auto it = t.begin(); it != t.end(); ++it)
9      {
10         cout << *it << " ";
11     }
12     cout << endl;
13
14     return 0;
15 }

```

Fence 74

- 我们在遍历的过程中需要给出容器的两端：开头（begin）和结尾（end），因为这种遍历方式不是基于范围来设计的。在基于范围的for循环中，不需要再传递容器的两端，循环会自动以容器为范围展开，并且循环中也屏蔽掉了迭代器的遍历细节，直接抽取容器中的元素进行运算，使用这种方式进行循环遍历会让编码和维护变得更加简便。

- C++98/03中普通的for循环，语法格式：

```
1  for(表达式 1; 表达式 2; 表达式 3)
2  {
3      // 循环体
4  }
```

Fence 75

- C++11基于范围的for循环，语法格式：

```
1  for (declaration : expression)
2  {
3      // 循环体
4  }
```

Fence 76

- 在上面的语法格式中

- **declaration表示遍历声明**，在遍历过程中，当前被遍历到的元素会被存储到声明的变量中。
- **expression是要遍历的对象**，它可以是**表达式、容器、数组、初始化列表**等。

- 使用基于范围的for循环遍历容器，示例代码如下：

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main(void)
6  {
7      vector <int> t{ 1,2,3,4,5,6 };
8      for (auto value : t)
9      {
10         cout << value << " ";
11     }
12     cout << endl;
13
14     return 0;
15 }
```

Fence 77

- 在上面的例子中，**是将容器中遍历的当前元素拷贝到了声明的变量value中**，因此无法对容器中的元素进行写操作，如果需要在遍历过程中修改元素的值，需要使用引用,效率更高。

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main(void)
6  {
7      vector <int> t{ 1,2,3,4,5,6 };
8      cout << "遍历修改之前的容器： ";
9      for (auto &value : t)
```

```

10     {
11         cout << value++ << " ";
12     }
13     cout << endl << "遍历修改之后的容器： ";
14
15     for (auto &value : t)
16     {
17         cout << value << " ";
18     }
19     cout << endl;
20
21     return 0;
22 }
23
24 /*
25     输出结果：
26     遍历修改之前的容器： 1 2 3 4 5 6
27     遍历修改之后的容器： 2 3 4 5 6 7
28     */
29
30 //对容器的遍历过程中，如果只是读数据，不允许修改元素的值，可以使用 const 定义保存元素数据的变量，在定义的时候建议使用 **const auto &**，这样相对于 const auto 效率要更高一些。
31
32 #include <iostream>
33 #include <vector>
34 using namespace std;
35
36 int main(void)
37 {
38     vector<int> t{ 1,2,3,4,5,6 };
39     for (const auto& value : t)
40     {
41         cout << value << " ";
42     }
43
44     return 0;
45 }

```

Fence 78

10.2 使用细节

10.2.1 关系型容器(键，值)

- 使用基于范围的for循环有一些需要注意的细节，先来看一下对关系型容器map的遍历：

```

1     #include <iostream>
2     #include <string>
3     #include <map>
4     using namespace std;
5
6     int main(void)
7     {
8         map<int, string> m{

```

```

9         {1, "lucy"}, {2, "lily"}, {3, "tom"}
10    };
11
12    // 基于范围的 for 循环方式
13    for (auto& it : m)
14    {
15        cout << "id: " << it.first << ", name: " << it.second << endl;
16    }
17
18    // 普通的 for 循环方式
19    for (auto it = m.begin(); it != m.end(); ++it)
20    {
21        cout << "id: " << it-> first << ", name: " << it-> second << endl;
22    }
23
24    return 0;
25 }

```

Fence 79

- 在上面的例子中使用两种方式对map进行了遍历，通过对比有两点需要注意的事项：
 - 使用普通的for循环方式（基于迭代器）遍历关联性容器，auto自动推导出的是一个迭代器类型，需要使用迭代器的方式取出元素中的键值对（和指针的操作方法相同）：
 - it->first
 - it->second
 - 使用基于范围的for循环遍历关联性容器，auto自动推导出的类型是容器中的value_type，相当于一个对组（std::pair）对象，提取键值对的方式如下：
 - it.first
 - it.second

10.2.2 元素只读

- 通过对基于范围的for循环语法的介绍可以得知，在for循环内部声明一个变量的引用就可以修改遍历的表达式中的元素的值，但是这并不适用于所有的情况，**对应set容器来说，内部元素都是只读的，这是由容器的特性决定的，因此在for循环中auto&会被视为const auto &。**

```

1  #include <iostream>
2  #include <set>
3  using namespace std;
4
5  int main(void)
6  {
7      set<int> st{ 1,2,3,4,5,6 };
8      for (auto &item : st)
9      {
10         cout << item++ << endl;    // error, 不能给常量赋值
11     }
12     return 0;
13 }

```

Fence 80

- 除此之外，在遍历关联型容器时也会出现同样的问题，基于范围的for循环中，虽然可以得到一个std::pair引用，但是我们是不能修改里边的first值的，也就是key值。

```
1  #include <iostream>
2  #include <string>
3  #include <map>
4  using namespace std;
5
6  int main(void)
7  {
8      map <int, string> m{
9          {1, "lucy"}, {2, "lily"}, {3, "tom"}
10     };
11
12     for (auto& item : m)
13     {
14         // item.first 是一个常量
15         cout << "id: " << item.first++ << ", name: " << item.second << endl; //
16         error
17     }
18     return 0;
19 }
```

Fence 81

10.2.3 访问次数

- 基于范围的for循环遍历的对象可以是一个表达式或者容器/数组等。假设我们对一个容器进行遍历，在遍历过程中for循环对这个容器的访问频率是一次还是多次呢？我们通过下面的例子验证一下：

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  vector <int> v{ 1,2,3,4,5,6 };
6  vector <int>& getRange()
7  {
8      cout << "get vector range..." << endl;
9      return v;
10 }
11
12 int main(void)
13 {
14     for (auto val : getRange())
15     {
16         cout << val << " ";
17     }
18     cout << endl;
19
20     return 0;
21 }
22
```

```

23  /*
24  输出结果:
25  get vector range...
26  1 2 3 4 5 6
27  */

```

Fence 82

- 从上面的结果中可以看到，不论基于范围的for循环迭代了多少次，函数getRange()只在第一次迭代之前被调用，得到这个容器对象之后就不会再去重新获取这个对象了。
- 对应基于范围的for循环来说，冒号后边的表达式只会被执行一次。在得到遍历对象之后会先确定好迭代的范围，基于这个范围直接进行遍历。如果是普通的for循环，在每次迭代的时候都需要判断是否已经到了结束边界。（只能顺序遍历）

11. 可调用对象包装器、绑定器

11.1 可调用对象

- 在C++中存在“可调用对象”这么一个概念。准确来说，可调用对象有如下几种定义：
 - 是一个函数指针

```

1  int print(int a, double b)
2  {
3      cout << a << b << endl;
4      return 0;
5  }
6  // 定义函数指针
7  int (*func)(int, double) = &print;

```

Fence 83

- 是一个具有operator()成员函数的类对象（仿函数）

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  struct Test
7  {
8      // ()操作符重载
9      void operator()(string msg)
10     {
11         cout << "msg: " << msg << endl;
12     }
13 };
14
15 int main(void)
16 {
17     Test t;

```

```

18     t("我是要成为海贼王的男人!!!");    // 仿函数
19     return 0;
20 }

```

Fence 84

- 是一个可被转换为函数指针的类对象

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  //给函数指针起别名
7  using func_ptr = void(*)(int, string);
8  struct Test
9  {
10     static void print(int a, string b)
11     {
12         cout << "name: " << b << ", age: " << a << endl;
13     }
14
15     // 将类对象转换为函数指针
16     //必须是静态函数（没有 this 指针），普通成员函数（存在 this 指针，只能被对象调用）不
行 非静态成员函数必须与对象绑定
17     operator func_ptr()
18     {
19         return print;
20     }
21 };
22
23 int main(void)
24 {
25     Test t;
26     // 对象转换为函数指针，并调用
27     t(19, "Monkey D. Luffy");
28
29     return 0;
30 }

```

Fence 85

- 是一个类成员函数指针或者类成员指针

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  struct Test
7  {
8     void print(int a, string b)

```

```

9      {
10         cout << "name: " << b << ", age: " << a << endl;
11     }
12     int m_num;
13 };
14
15 int main(void)
16 {
17     // 定义类成员函数指针指向类成员函数
18     //使用 using 定义
19     //using func_ptr = void(Test::*)(int , string);
20     //func_ptr f1 = &Test:: print;          //函数名就是地址，加不加&都行
21     void (Test::*func_ptr)(int, string) = &Test:: print;
22     // 类成员指针指向类成员变量
23     int Test::*obj_ptr = &Test:: m_num;
24
25     Test t;
26     // 通过类成员函数指针调用类成员函数
27     // *f1 解引用 等于使用该函数
28     //(t.*f1)(19, "Monkey D. Luffy");
29     (t.*func_ptr)(19, "Monkey D. Luffy");
30     // 通过类成员指针初始化类成员变量
31     t.*obj_ptr = 1;
32     cout << "number is: " << t.m_num << endl;
33
34     return 0;
35 }

```

Fence 86

- 在上面的例子中满足条件的这些可调用对象对应的类型被统称为可调用类型。C++中的可调用类型虽然具有比较统一的操作形式，但定义方式五花八门，这样在我们试图使用统一的方式保存，或者传递一个可调用对象时会十分繁琐。现在，C++11通过提供std::function 和 std::bind统一了可调用对象的各种操作。

11.2 可调用对象包装器(适用于业务逻辑复杂的场景)

- std::function是可调用对象的包装器。它是一个类模板，可以容纳除了类(非静态)成员（函数）指针之外的所有可调用对象。通过指定它的模板参数，它可以用统一的方式处理函数、函数对象、函数指针，并允许保存和延迟执行它们。

11.2.1 基本用法

- std::function必须要包含一个叫做functional的头文件，可调用对象包装器使用语法如下：

```

1  #include <functional>
2  std::function <返回值类型(参数类型列表)> diy_name = 可调用对象;

```

Fence 87

- 下面的实例代码中演示了可调用对象包装器的基本使用方法：

```

1  #include <iostream>
2  #include <functional>

```

```

3   using namespace std;
4
5   using func_ptr = int (*)(int, int);
6   int add(int a, int b)
7   {
8       cout << a << " + " << b << " = " << a + b << endl;
9       return a + b;
10  }
11
12  class T1
13  {
14  public:
15      static int sub(int a, int b)
16      {
17          cout << a << " - " << b << " = " << a - b << endl;
18          return a - b;
19      }
20
21      operator func_ptr()
22      {
23          return sub;
24      }
25  };
26
27  class T2
28  {
29  public:
30      int operator()(int a, int b)
31      {
32          cout << a << " * " << b << " = " << a * b << endl;
33          return a * b;
34      }
35  };
36
37  int main(void)
38  {
39      // 绑定（包装）一个普通函数
40      function<int(int, int)> f1 = add;
41      // 绑定（包装）一个静态类成员函数
42      function<int(int, int)> f2 = T1::sub;
43      // 绑定（包装）一个仿函数
44      T2 t;
45      function<int(int, int)> f3 = t;
46      // 绑定（包装）一个转换成函数指针的对象
47      T1 tt;
48      function<int(int, int)> f4 = tt;
49
50      // 函数调用
51      f1(9, 3);
52      f2(9, 3);
53      f3(9, 3);
54      f4(9, 3);
55

```

```

56     return 0;
57 }
58
59 //结果:
60 //9 + 3 = 12
61 //9 - 3 = 6
62 //9 * 3 = 27
63 //9 / 3 = 3

```

Fence 88

- 通过测试代码可以得到结论：`std::function`可以将可调用对象进行包装，得到一个统一的格式，包装完成得到的对象相当于一个函数指针，和函数指针的使用方式相同，通过包装器对象就可以完成对包装的函数的调用了。

11.2.2 作为回调函数使用

- 因为回调函数本身就是通过函数指针实现的，使用对象包装器可以取代函数指针的作用，来看一下下面的例子：

```

1  #include <iostream>
2  #include <functional>
3  using namespace std;
4
5  class A
6  {
7  public:
8      // 构造函数参数是一个包装器对象 目的：给一个处理函数，并将函数存储在 callback 中。
9      A(const function<void()>& f) : callback(f)
10     {
11     }
12
13     void notify()
14     {
15         callback(); // 调用通过构造函数得到的函数指针 目的：实际调用该处理函数
16     }
17 private:
18     function<void()> callback;
19 };
20
21 class B
22 {
23 public:
24     //重载 ()，且无参
25     void operator()()
26     {
27         cout << "我是要成为海贼王的男人!!!" << endl;
28     }
29 };
30 int main(void)
31 {
32     B b;
33     A a(b); // 仿函数通过包装器对象进行包装
34     a.notify();
35 }

```

```
36     return 0;
37 }
```

Fence 89

- 通过上面的例子可以看出，使用对象包装器`std::function`可以非常方便的将仿函数转换为一个函数指针，通过进行函数指针的传递，在其他函数的合适的位置就可以调用这个包装好的仿函数了。
- 另外，使用`std::function`作为函数的传入参数，可以将定义方式不相同的可调用对象进行统一的传递，这样大大增加了程序的灵活性。

11.3 绑定器

- `std::bind`用来将可调用对象与其参数一起进行绑定。绑定后的结果可以使用`std::function`进行保存，并延迟调用到任何我们需要的时候。**通俗来讲，它主要有两大作用：
 - 将可调用对象与其参数一起绑定成一个仿函数。
 - 将多元（参数个数为 n ， $n>1$ ）可调用对象转换为一元或者 $(n-1)$ 元可调用对象，即只绑定部分参数。
- 绑定器函数使用语法格式如下：

```
1 // 绑定非类成员函数/变量
2 auto f = std::bind(可调用对象地址, 绑定的参数/占位符);
3 // 绑定类成员函数/变量
4 auto f = std::bind(类函数/成员地址, 类实例对象地址, 绑定的参数/占位符);
```

Fence 90

- 下面来看一个关于绑定器的实际使用的例子：

```
1 #include <iostream>
2 #include <functional>
3 using namespace std;
4
5 void callFunc(int x, const function <void(int)>& f)
6 {
7     if (x % 2 == 0)
8     {
9         f(x);
10    }
11 }
12
13 void output(int x)
14 {
15     cout << x << " ";
16 }
17
18 void output_add(int x)
19 {
20     cout << x + 10 << " ";
21 }
22
23 int main(void)
24 {
25     // 使用绑定器绑定可调用对象和参数
```

```

26     auto f1 = bind(output, placeholders::_1);
27     for (int i = 0; i < 10; ++i)
28     {
29         callFunc(i, f1);
30     }
31     cout << endl;
32
33     auto f2 = bind(output_add, placeholders::_1);
34     for (int i = 0; i < 10; ++i)
35     {
36         callFunc(i, f2);
37     }
38     cout << endl;
39
40     return 0;
41 }
42
43 /*
44 结果:
45 0 2 4 6 8
46 10 12 14 16 18
47 */

```

Fence 91

- 在上面的程序中，使用了std::bind绑定器，在函数外部通过绑定不同的函数，控制了最后执行的结果。**std::bind绑定器返回的是一个仿函数类型，得到的返回值可以直接赋值给一个std::function，在使用的时候我们并不需要关心绑定器的返回值类型，使用auto进行自动类型推导就可以了。**
- placeholders::_1是一个占位符，代表这个位置将在函数调用时被传入的第一个参数所替代。同样还有其他的占位符placeholders::_2、placeholders::_3、placeholders::_4、placeholders::_5等.....
- 有了占位符的概念之后，使得std::bind的使用变得非常灵活:

```

1  #include <iostream>
2  #include <functional>
3  using namespace std;
4
5  void output(int x, int y)
6  {
7      cout << x << " " << y << endl;
8  }
9
10 int main(void)
11 {
12     // 使用绑定器绑定可调用对象和参数，并调用得到的仿函数
13     bind(output, 1, 2)(); // 两个固定参数
14     bind(output, placeholders::_1, 2)(10); // 一个变量，一个固定参数
15     bind(output, 2, placeholders::_1)(10);
16
17     // error, 调用时没有第二个参数
18     // bind(output, 2, placeholders::_2)(10);
19     // 调用时第一个参数 10 被吞掉了，没有被使用
20     bind(output, 2, placeholders::_2)(10, 20);
21

```



```

22     bind(output, placeholders::_1, placeholders::_2)(10, 20);
23     bind(output, placeholders::_2, placeholders::_1)(10, 20);
24
25
26     return 0;
27 }
28
29 /*
30 结果
31 1 2      // bind(output, 1, 2)();
32 10 2     // bind(output, placeholders::_1, 2)(10);
33 2 10     // bind(output, 2, placeholders::_1)(10);
34 2 20     // bind(output, 2, placeholders::_2)(10, 20);
35 10 20    // bind(output, placeholders::_1, placeholders::_2)(10, 20);
36 20 10    // bind(output, placeholders::_2, placeholders::_1)(10, 20);
37 */

```

Fence 92

- 通过测试可以看到，`std::bind`可以直接绑定函数的所有参数，也可以仅绑定部分参数。在绑定部分参数的时候，通过使用`std::placeholders`来决定空位参数将会属于调用发生时的第几个参数。
- 可调用对象包装器`std::function`是不能实现对类成员函数指针或者类成员指针的包装的，但是通过绑定器`std::bind`的配合之后，就可以完美的解决这个问题了，再来看一个例子，然后再解释里边的细节：

```

1  #include <iostream>
2  #include <functional>
3  using namespace std;
4
5  class Test
6  {
7  public:
8      void output(int x, int y)
9      {
10         cout << "x: " << x << ", y: " << y << endl;
11     }
12     int m_number = 100;
13 };
14
15 void testFunc(int i, int j, const functional <void(int,int)> &f)
16 {
17     if(i % 2 == 0)
18     {
19         f(i, j);
20     }
21 }
22
23 int main(void)
24 {
25     Test t;
26     // 绑定类成员函数
27     function <void(int, int)> f1 =
28         bind(&Test:: output, &t, placeholders::_1, placeholders::_2);
29     // 绑定类成员变量(公共) 不加&, 不能修改变量值
30     function <int&(void)> f2 = bind(&Test:: m_number, &t);

```

```

31     auto f22 = bind(&Test:: m_number, &t);    // 与 f2 类型不一样，一个是仿函数，一个是
    包装器
32
33     //其他写法
34     auto f3 = bind(&Test:: output, &t, placeholders::_1, placeholders::_2);
35     testFunc(10,20, f3);
36
37
38     // 调用
39     f1(520, 1314);
40     f2() = 2333;
41     cout << "t.m_number: " << t.m_number << endl;
42
43     return 0;
44 }
45
46
47 /*
48 结果:
49 x: 520, y: 1314
50 t.m_number: 2333
51 */

```

Fence 93

- 在用绑定器绑定类成员函数或者成员变量的时候需要将它们所属的实例对象一并传递到绑定器函数内部。**f1**的类型是`function<void(int, int)>`，通过使用`std::bind`将Test的成员函数`output`的地址和对象`t`绑定，并转化为一个仿函数并存储到对象`f1`中。
- 使用绑定器绑定的类成员变量`m_number`得到的仿函数被存储到了类型为`function<int&(void)>`的包装器对象`f2`中，并且可以在需要的时候修改这个成员。其中`int`是绑定的类成员的类型，并且允许修改绑定的变量，因此需要指定为变量的引用，由于没有参数因此参数列表指定为`void`。
- 示例程序中是使用`function`包装器保存了`bind`返回的仿函数，如果不知道包装器的模板类型如何指定，可以直接使用`auto`进行类型的自动推导，这样使用起来会更容易一些。

12. Lambda表达式(匿名函数)

12.1 基本用法

lambda表达式是C++11最重要也是最常用的特性之一，这是现代编程语言的一个特点，lambda表达式有如下的一些优点：

- 声明式的编程风格：就地匿名定义目标函数或函数对象，不需要额外写一个命名函数或函数对象。
- 简洁：避免了代码膨胀和功能分散，让开发更加高效。
- 在需要的时间和地点实现功能闭包，使程序更加灵活。

lambda表达式定义了一个匿名函数，并且可以捕获一定范围内的变量。lambda表达式的语法形式简单归纳如下：

```

1 | [capture](params) opt -> ret {body;};

```

其中capture是捕获列表，params是参数列表，opt是函数选项，ret是返回值类型，body是函数体。

1. 捕获列表[]: 捕获一定范围内的变量
2. 参数列表(): 和普通函数的参数列表一样，如果没有参数参数列表可以省略不写。

```
1 auto f = []() { return 1; } // 没有参数，参数列表为空
2 auto f = [] { return 1; }   // 没有参数，参数列表省略不写
```

Fence 95

3. opt 选项，不需要可以省略
 - mutable: 可以修改按值传递进来的拷贝（注意是能修改拷贝，而不是值本身）
 - exception: 指定函数抛出的异常，如抛出整数类型的异常，可以使用throw();
4. 返回值类型: 在C++11中，lambda表达式的返回值是通过返回值后置语法来定义的。
5. 函数体: 函数的实现，这部分不能省略，但函数体可以为空。

调用:

```
1 []() {
2
3 } (传入的实参);
```

Fence 96

12.2 捕获列表

lambda表达式的捕获列表可以捕获一定范围内的变量，具体使用方式如下:

- [] - 不捕捉任何变量
- [&] - 捕获外部作用域中所有变量, 并作为引用在函数体内使用 (**按引用捕获**)
- [=] - 捕获外部作用域中所有变量, 并作为副本在函数体内使用 (**按值捕获**)
 - 拷贝的副本在匿名函数体内部是只读的
- [=, &foo] - 按值捕获外部作用域中所有变量, 并按照引用捕获外部变量 foo
- [bar] - 按值捕获 bar 变量, 同时不捕获其他变量
- [&bar] - 按引用捕获 bar 变量, 同时不捕获其他变量
- [this] - 捕获当前类中的this指针
 - 让lambda表达式拥有和当前类成员函数同样的访问权限
 - 如果已经使用了 & 或者 =, 默认添加此选项

下面通过一个例子，看一下初始化列表的具体用法:

```
1 #include <iostream>
2 #include <functional>
3 using namespace std;
4
5 class Test
```

```

6 {
7 public:
8     void output(int x, int y)
9     {
10         auto x1 = [] {return m_number; }; // error
11         auto x2 = [=] {return m_number + x + y; }; // ok
12         auto x3 = [&] {return m_number + x + y; }; // ok
13         auto x4 = [this] {return m_number; }; // ok
14         auto x5 = [this] {return m_number + x + y; }; // error
15         auto x6 = [this, x, y] {return m_number + x + y; }; // ok
16         auto x7 = [this] {return m_number++; }; // ok
17     }
18     int m_number = 100;
19 };
20
21 /*
22 x1: 错误，没有捕获外部变量，不能使用类成员 m_number
23 x2: 正确，以值拷贝的方式捕获所有外部变量
24 x3: 正确，以引用的方式捕获所有外部变量
25 x4: 正确，捕获 this 指针，可访问对象内部成员
26 x5: 错误，捕获 this 指针，可访问类内部成员，没有捕获到变量 x, y，因此不能访问。
27 x6: 正确，捕获 this 指针，x, y
28 x7: 正确，捕获 this 指针，并且可以修改对象内部变量的值
29 */
30
31 int main(void)
32 {
33     int a = 10, b = 20;
34     auto f1 = [] {return a; }; // error
35     auto f2 = [&] {return a++; }; // ok
36     auto f3 = [=] {return a; }; // ok
37     auto f4 = [=] {return a++; }; // error
38     auto f5 = [a] {return a + b; }; // error
39     auto f6 = [a, &b] {return a + (b++); }; // ok
40     auto f7 = [=, &b] {return a + (b++); }; // ok
41
42     return 0;
43 }
44
45 /*
46 f1: 错误，没有捕获外部变量，因此无法访问变量 a
47 f2: 正确，使用引用的方式捕获外部变量，可读写
48 f3: 正确，使用值拷贝的方式捕获外部变量，可读
49 f4: 错误，使用值拷贝的方式捕获外部变量，可读不能写
50 f5: 错误，使用拷贝的方式捕获了外部变量 a，没有捕获外部变量 b，因此无法访问变量 b
51 f6: 正确，使用拷贝的方式捕获了外部变量 a，只读，使用引用的方式捕获外部变量 b，可读写
52 f7: 正确，使用值拷贝的方式捕获所有外部变量以及 b 的引用，b 可读写，其他只读
53 */

```

在匿名函数内部，需要通过lambda表达式的捕获列表控制如何捕获外部变量，以及访问哪些变量。默认状态下lambda表达式无法修改通过复制方式捕获外部变量，如果希望修改这些外部变量，需要通过引用的方式进行捕获。

12.3 返回值

很多时候，lambda表达式的返回值是非常明显的，因此在C++11中允许省略lambda表达式的返回值。

```
1 // 完整的 lambda 表达式定义
2 auto f = [](int a) -> int
3 {
4     return a+10;
5 };
6
7 // 忽略返回值的 lambda 表达式定义
8 auto f = [](int a)
9 {
10     return a+10;
11 };
```

Fence 98

一般情况下，不指定lambda表达式的返回值，编译器会根据return语句自动推导返回值的类型，**但需要注意的是labmda表达式不能通过列表初始化自动推导出返回值类型。**

```
1 // ok, 可以自动推导出返回值类型
2 auto f = [](int i)
3 {
4     return i;
5 };
6
7 // error, 不能推导出返回值类型 需要将返回值类型写出, 否则编译器不知道初始化成什么类型
8 //auto f1 = []() -> vector<int>{return {1, 2};};
9 auto f1 = []()
10 {
11     return {1, 2}; // 基于列表初始化推导返回值, 错误
12 };
```

Fence 99

12.4 函数本质

使用lambda表达式捕获列表捕获外部变量，如果希望去修改按值捕获的外部变量，那么应该如何处理呢？这就需要使用mutable选项，**被mutable修改是lambda表达式就算没有参数也要写明参数列表(即写上小括号)，并且可以去掉按值捕获的外部变量的只读（const）属性。**

```
1 int a = 0;
2 auto f1 = [=] {return a++; }; // error, 按值捕获外部变量, a 是只读的
3 auto f2 = [=]() mutable {return a++; }; // ok
```

Fence 100

最后再剖析一下为什么通过值拷贝的方式捕获的外部变量是只读的:

1. lambda表达式的类型在C++11中会被看做是一个带operator()的类，即**仿函数**。
2. 按照C++标准，lambda表达式的operator()默认是const的，一个const成员函数是无法修改成员变量值的。

mutable选项的作用就在于取消operator()的const属性。

因为lambda表达式在C++中会被看做是一个仿函数，因此可以使用std::function和std::bind来存储和操作lambda表达式：

```
1  #include <iostream>
2  #include <functional>
3  using namespace std;
4
5  int main(void)
6  {
7      // 包装可调用函数
8      std::function<int(int)> f1 = [](int a) {return a; };
9      // 绑定可调用函数
10     std::function<int(int)> f2 = bind([](int a) {return a; }, placeholders::_1);
11
12     // 函数调用
13     cout << f1(100) << endl;
14     cout << f2(200) << endl;
15     return 0;
16 }
```

Fence 101

对于没有捕获任何变量的lambda表达式，还可以转换成一个普通的函数指针：

```
1  using func_ptr = int (*)(int);
2  // 没有捕获任何外部变量的匿名函数
3  func_ptr f = [](int a)
4  {
5      return a;
6  };
7  // 函数调用
8  f(1314);
9
```

Fence 102

13. 右值引用

13.1 右值

C++11 增加了一个新的类型，称为右值引用（R-value reference），标记为 **&&**。在介绍右值引用类型之前先要了解什么是左值和右值：

- lvalue 是locator value的缩写，rvalue 是 read value的缩写
- 左值是指存储在内存中、有明确存储地址（可取地址）的数据；
- 右值是指可以提供数据值的数据（不可取地址），如常量，常量字符串；

通过描述可以看出，区分左值与右值的便捷方法是：**可以对表达式取地址（&）就是左值，否则为右值。所有有名字的变量或对象都是左值，而右值是匿名的。**

```
1  int a = 520;
2  int b = 1314;
3  a = b;
```

一般情况下，位于=前的表达式为左值，位于=后边的表达式为右值。也就是说例子中的a, b为左值，520,1314为右值。a=b是一种特殊情况，在这个表达式中a, b都是左值，因为变量b是可以被取地址的，不能视为右值。

C++11 中右值可以分为两种：一个是将亡值（xvalue, expiring value），另一个则是纯右值（prvalue, PureRvalue）：

- 纯右值：非引用返回的临时变量、运算表达式产生的临时变量、原始字面量和 lambda 表达式等
- 将亡值：与右值引用相关的表达式，比如，T&&类型函数的返回值、std::move 的返回值等。

```
1  int value = 520;
2  //右值引用
3  int &&b = 10;
4  //常量右值引用
5  const int &&b1 = 11;
6  const int &&b2 = b;           //error, 右值引用只能通过右值进行初始化, b 变量 = 左边时, 编译器还是会将其看成左值
7
8  int &&b3 = b;                 //error
9
10 //常量左值引用
11 const int &a = value;         //正确, 但是不是最优
12 //constexpr int &a = value;   //最优, 但是有些时候编译器会报错 (vs2022, gcc: 13.1.0)
13 const int &a1 = value;
14 const int &a2 = b;
15 const int &a3 = b1;
```

Fence 104

在上面的语句中，value是左值，520是字面量也就是右值。其中value可以被引用，但是520就不行了，因为字面量都是右值。

左值能接受右值，右值只能接受右值

13.2 右值引用（引用实际效果还是取别名，本质是指针常量。作用：延长右值的销毁时间）

指针常量即指向常量的指针，指向的内容不能通过该指针改变，但是能改变指针指向（改变指向的地址）。

常量指针即常量形式的指针，指向方向不能改变（指向的地址固定），指向的内容可以通过该指针改变。（即const永远形容左边）

右值引用就是对一个右值进行引用的类型。因为右值是匿名的，所以我们只能通过引用的方式找到它。无论声明左值引用还是右值引用都必须立即进行初始化，因为引用类型本身并不拥有所绑定对象的内存，只是该对象的一个别名。通过右值引用的声明，该右值又“重获新生”，其生命周期与右值引用类型变量的生命周期一样，只要该变量还活着，该右值临时量将会一直存活下去。

```
1  #include <iostream>
2  using namespace std;
3
4  int&& value = 520;
5  class Test
```

```

6   {
7   public:
8       Test()
9       {
10          cout << "construct: my name is jerry" << endl;
11      }
12      Test(const Test& a)
13      {
14          cout << "copy construct: my name is tom" << endl;
15      }
16  };
17
18  Test getObj()
19  {
20      return Test();
21  }
22
23  int main()
24  {
25      int a1;
26      int &&a2 = a1;          // error
27      Test& t = getObj();    // error
28      Test && t = getObj();
29      const Test& t = getObj();
30      return 0;
31  }

```

Fence 105

- 在上面的例子中 `int&& value = 520`; 里面520是纯右值，value是对字面量520这个右值的引用。
- 在 `int &&a2 = a1`; 中a1虽然写在了=右边，但是它仍然是一个左值，使用左值初始化一个右值引用类型是不合法的。
- 在 `Test& t = getObj()` 这句代码中语法是错误的，右值不能给普通的左值引用赋值。
- 在 `Test && t = getObj()`; 中 `getObj()` 返回的临时对象被称之为将亡值，t是这个将亡值的右值引用。
- `const Test& t = getObj()` 这句代码的语法是正确的，常量左值引用是一个万能引用类型，它可以接受左值、右值、常量左值和常量右值。

13.3 性能优化

在C++中在进行对象赋值操作的时候，很多情况下会发生对象之间的深拷贝，如果堆内存很大，这个拷贝的代价也就非常大，在某些情况下，如果想要避免对象的深拷贝，就可以使用右值引用进行性能的优化。

```

1   #include <iostream>
2   using namespace std;
3
4   class Test
5   {
6   public:
7       Test() : m_num(new int(100))
8       {
9           cout << "construct: my name is jerry" << endl;
10      }

```



```

11
12     Test(const Test& a) : m_num(new int(*a.m_num))
13     {
14         cout << "copy construct: my name is tom" << endl;
15     }
16
17     ~Test()
18     {
19         delete m_num;
20     }
21
22     int* m_num;
23 };
24
25 Test getObj()
26 {
27     Test t;
28     return t;
29 }
30
31 int main()
32 {
33     Test t = getObj();
34     cout << "t.m_num: " << *t.m_num << endl;
35     return 0;
36 };
37

```

Fence 106

测试代码执行的结果为（当时使用的vs版本为2019，vs2022已无法看到相同的输出，代码被优化了）：

```

1   construct: my name is jerry
2   copy construct: my name is tom
3   t.m_num: 100

```

Fence 107

通过输出的结果可以看到调用Test t = getObj();的时候调用拷贝构造函数对返回的临时对象进行了深拷贝得到了对象t，在getObj()函数中创建的对象虽然进行了内存的申请操作，但是没有使用就释放掉了。如果能够使用临时对象已经申请的资源，既能节省资源，还能节省资源申请和释放的时间，如果要执行这样的操作就需要使用右值引用了，**右值引用具有移动语义，移动语义可以将资源（堆、系统对象等）通过浅拷贝从一个对象转移到另一个对象这样就能减少不必要的临时对象的创建、拷贝以及销毁，可以大幅提高C++应用程序的性能。**

```

1   #include <iostream>
2   using namespace std;
3
4   class Test
5   {
6   public:
7       Test() : m_num(new int(100))
8       {
9           cout << "construct: my name is jerry" << endl;
10      }

```

```

11
12     Test(const Test& a) : m_num(new int(*a.m_num))
13     {
14         cout << "copy construct: my name is tom" << endl;
15     }
16
17     // 添加移动构造函数（右值引用构造函数）
18     Test(Test&& a) : m_num(a.m_num)           //m_num(a.m_num)获取所有权（指向同一个内存地址-
-m_num 为指针）
19     {
20         a.m_num = nullptr;                   // 清除原本者的所有权，其改变指针指向
21         cout << "move construct: my name is sunny" << endl;
22     }
23
24     ~Test()
25     {
26         delete m_num;
27         cout << "destruct Test class ..." << endl;
28     }
29
30     int* m_num;
31 };
32
33 Test getObj()
34 {
35     Test t;
36     return t;
37 }
38
39 int main()
40 {
41     Test t = getObj();
42     cout << "t.m_num: " << *t.m_num << endl;
43     return 0;
44 };
45

```

Fence 108

测试代码执行的结果如下（当时使用的vs版本为2019，vs2022已无法看到相同的输出，代码被优化了）：

```

1   construct: my name is jerry
2   move construct: my name is sunny
3   destruct Test class ...
4   t.m_num: 100
5   destruct Test class ...

```

Fence 109

通过修改，在上面的代码给Test类添加了移动构造函数（参数为右值引用类型），这样在进行Test t = getObj();操作的时候并没有调用拷贝构造函数进行深拷贝，而是调用了移动构造函数，在这个函数中只是进行了浅拷贝，没有对临时对象进行深拷贝，提高了性能。

在测试程序中getObj()的返回值就是一个将亡值，也就是说是一个右值，在进行赋值操作的时候如果=右边是一个右值，那么移动构造函数就会被调用。移动构造中使用了右值引用，会将临时对象中的堆内存地址的所有权转移给对象t，这块内存被成功续命，因此在t对象中还可以继续使用这块内存。

对于需要动态申请大量资源的类，应该设计移动构造函数，以提高程序效率。需要注意的是，我们一般在提供移动构造函数的同时，也会提供常量左值引用的拷贝构造函数，以保证移动不成还可以使用拷贝构造函数。

13.4 && 的特性

在C++中，并不是所有情况下 && 都代表是一个右值引用，具体的场景体现在模板和自动类型推导中，如果是模板参数需要指定为T&&，如果是自动类型推导需要指定为auto &&，在这两种场景下 &&被称作未定的引用类型。另外还有一点需要额外注意const T&&表示一个右值引用，不是未定引用类型。

```
1  template <typename T>
2  void f(T&& param);
3  void f1(const T&& param);
4  f(10);
5  int x = 10;
6  f(x);
7  f1(x); // error, x 是左值
8  f1(10); // ok, 10 是右值
9
```

Fence 110

在上面的例子中函数模板进行了自动类型推导，需要通过传入的实参来确定参数param的实际类型。

- 第4行中，对于f(10)来说传入的实参10是右值，因此T&&表示右值引用
- 第6行中，对于f(x)来说传入的实参x是左值，因此T&&表示左值引用
- 第7行中，f1(x)的参数是const T&&不是未定引用类型，不需要推导，本身就表示一个右值引用

```
1  int main()
2  {
3      int x = 520, y = 1314;
4      auto&& v1 = x;
5      auto&& v2 = 250;
6      decltype(x)&& v3 = y; // error
7      cout << "v1: " << v1 << ", v2: " << v2 << endl;
8      return 0;
9  };
```

Fence 111

- 第4行中 auto&&表示一个整形的左值引用
- 第5行中 auto&&表示一个整形的右值引用
- 第6行中decltype(x)&&等价于int&&是一个右值引用不是未定引用类型，y是一个左值，不能使用左值初始化一个右值引用类型。

由于上述代码中存在T&&或者auto&&这种未定引用类型，当它作为参数时，有可能被一个右值引用初始化，也有可能被一个左值引用初始化，在进行类型推导时右值引用类型（&&）会发生变化，这种变化被称为引用折叠。在C++11中引用折叠的规则如下：

1. 通过右值推导 T&& 或者 auto&& 得到的是一个右值引用类型

2. 通过非右值（右值引用、左值、左值引用、常量右值引用、常量左值引用）推导 T&& 或者 auto&& 得到的是一个左值引用类型

```
1  int&& a1 = 5;
2  auto&& bb = a1;
3  auto&& bb1 = 5;
4
5  int a2 = 5;
6  int &a3 = a2;
7  auto&& cc = a3;
8  auto&& cc1 = a2;
9
10 const int& s1 = 100;
11 const int&& s2 = 100;
12 auto&& dd = s1;
13 auto&& ee = s2;
14
15 const auto&& x = 5;
16
```

Fence 112

- 第2行：a1为右值引用，推导出的bb为左值引用类型
- 第3行：5为右值，推导出的bb1为右值引用类型
- 第7行：a3为左值引用，推导出的cc为左值引用类型
- 第8行：a2为左值，推导出的cc1为左值引用类型
- 第12行：s1为常量左值引用，推导出的dd为常量左值引用类型
- 第13行：s2为常量右值引用，推导出的ee为常量左值引用类型
- 第15行：x为右值引用，不需要推导，只能通过右值初始化

```
1  #include <iostream>
2  using namespace std;
3
4  void printValue(int &i)
5  {
6      cout << "l-value: " << i << endl;
7  }
8
9  void printValue(int &&i)
10 {
11     cout << "r-value: " << i << endl;
12 }
13
14 void forward(int &&k)
15 {
16     printValue(k);
17 }
18
19 int main()
20 {
21     int i = 520;
```

```

22     printValue(i);
23     printValue(1314);
24     forward(250);
25
26     return 0;
27 };
28
29 测试代码输出的结果如下：
30 l-value: 520
31 r-value: 1314
32 l-value: 250

```

Fence 113

根据测试代码可以得知，编译器会根据传入的参数的类型（左值还是右值）调用对应的重置函数（printValue），函数forward()接收的是一个右值，但是在这个函数中调用函数printValue()时，参数k变成了一个命名对象，编译器会将其当做左值来处理。

最后总结一下关于&&的使用：

1. 左值和右值是独立于他们的类型的，右值引用类型可能是左值也可能是右值。
2. 编译器会将已命名的右值引用视为左值，将未命名的右值引用视为右值。
3. auto&&或者函数参数类型自动推导的T&&是一个未定的引用类型，它可能是左值引用也可能是右值引用类型，这取决于初始化的值类型（上面有例子）。
4. 通过右值推导 T&& 或者 auto&& 得到的是一个右值引用类型，其余都是左值引用类型。

14. 转移和完美转移

14.1 move（返回值为右值——将亡值）

在C++11添加了右值引用，并且不能使用左值初始化右值引用，如果想要使用左值初始化一个右值引用需要借助std::move()函数，使用std::move方法可以将左值转换为右值。使用这个函数并不能移动任何东西，而是和移动构造函数一样都具有移动语义，将对象的状态或者所有权从一个对象转移到另一个对象，只是转移，没有内存拷贝，但是会修改旧对象的指向，即并将源对象的资源指针设置为 nullptr。

从实现上讲，std::move基本等同于一个类型转换：static_cast<T&&>(lvalue);，函数原型如下：

```

1  template <class _Ty>
2  _NODISCARD constexpr remove_reference_t<_Ty>&& move(_Ty&& _Arg) _NOEXCEPT
3  {    // forward _Arg as movable
4      return (static_cast<remove_reference_t<_Ty>&&>(_Arg));
5  }
6

```

Fence 114

使用方法如下：

```

1  class Test
2  {
3  public:
4      Test(){}
5      .....
6  }
7  int main()
8  {
9      Test t;
10     Test && v1 = t;           // error 使用左值初始化右值引用，因此语法是错误的
11     Test && v2 = move(t);     // ok 使用 move()函数将左值转换为了右值，这样就可以初始化右值引用
    了。
12     return 0;
13 }

```

Fence 115

假设一个临时容器很大，并且需要将这个容器赋值给另一个容器，就可以执行如下操作：

```

1  list<string> ls;
2  ls.push_back("hello");
3  ls.push_back("world");
4  .....
5  list<string> ls1 = ls;       // 需要拷贝，效率低
6  list<string> ls2 = move(ls);

```

Fence 116

如果不使用std::move，拷贝的代价很大，性能较低。使用move几乎没有任何代价，只是转换了资源的所有权。如果一个对象内部有较大的堆内存或者动态数组时，使用move()就可以非常方便的进行数据所有权的转移。另外，我们也可以给类编写相应的移动构造函数（T::T(T&& another)）和具有移动语义的赋值函数（T&& T::operator=(T&& rhs)），在构造对象和赋值的时候尽可能的进行资源的重复利用，因为它们都是接收一个右值引用参数。

14.2 forward

右值引用类型是独立于值的，一个右值引用作为函数参数的形参时，在函数内部转发该参数给内部其他函数时，它就成了一个左值，并不是原来的类型了。如果需要按照参数原来的类型转发到另一个函数，可以使用C++11提供的std::forward()函数，该函数实现的功能称之为**完美转发**。

```

1  // 函数原型
2  template<class T> T&& forward (typename remove_reference<T>::type& t) noexcept;
3  template<class T> T&& forward (typename remove_reference<T>::type&& t) noexcept;
4
5  // 精简之后的样子
6  std::forward<T>(t);
7

```

Fence 117

- 当T为左值引用类型时，t将被转换为T类型的左值
- 当T不是左值引用类型时，t将被转换为T类型的右值

```

1  #include <iostream>
2  using namespace std;

```

```

3
4     template <typename T>
5     void printValue(T& t)
6     {
7         cout << "l-value: " << t << endl;
8     }
9
10    template <typename T>
11    void printValue(T&& t)
12    {
13        cout << "r-value: " << t << endl;
14    }
15
16    template <typename T>
17    void testForward(T && v)
18    {
19        printValue(v);                //v 为左值
20        printValue(move(v));          //move 的返回值为右值
21        printValue(forward <T>(v));
22        cout << endl;
23    }
24
25    int main()
26    {
27        testForward(520);
28        int num = 1314;
29        testForward(num);
30        testForward(forward <int>(num));    // int 左值
31        testForward(forward <int&>(num));    //int & 左值引用
32        testForward(forward <int&&>(num));    //int && 右值引用
33
34        return 0;
35    }
36

```

Fence 118

测试代码打印的结果如下:

```

1     l-value: 520
2     r-value: 520
3     r-value: 520
4
5     l-value: 1314
6     r-value: 1314
7     l-value: 1314
8
9     l-value: 1314
10    r-value: 1314
11    r-value: 1314
12
13    l-value: 1314
14    r-value: 1314
15    l-value: 1314

```

```
16
17 l-value: 1314
18 r-value: 1314
19 r-value: 1314
```

Fence 119

- testForward(520);函数的形参为未定引用类型T&&, 实参为右值, 初始化后被推导为一个右值引用
 - printValue(v);已命名的右值v, 编译器会视为左值处理, 实参为左值
 - printValue(move(v));已命名的右值编译器会视为左值处理, 通过move又将其转换为右值, 实参为右值
 - printValue(forward(v));forward的模板参数为右值引用, 最终得到一个右值, 实参为右值
- testForward(num);函数的形参为未定引用类型T&&, 实参为左值, 初始化后被推导为一个左值引用
 - printValue(v);实参为左值
 - printValue(move(v));通过move将左值转换为右值, 实参为右值
 - printValue(forward(v));forward的模板参数为左值引用, 最终得到一个左值引用, 实参为左值
- testForward(forward(num));forward的模板类型为int, 最终会得到一个右值, 函数的形参为未定引用类型T&&被右值初始化后得到一个右值引用类型
 - printValue(v);已命名的右值v, 编译器会视为左值处理, 实参为左值
 - printValue(move(v));已命名的右值编译器会视为左值处理, 通过move又将其转换为右值, 实参为右值
 - printValue(forward(v));forward的模板参数为右值引用, 最终得到一个右值, 实参为右值
- testForward(forward<int&>(num));forward的模板类型为int&, 最终会得到一个左值, 函数的形参为未定引用类型T&&被左值初始化后得到一个左值引用类型
 - printValue(v);实参为左值
 - printValue(move(v));通过move将左值转换为右值, 实参为右值
 - printValue(forward(v));forward的模板参数为左值引用, 最终得到一个左值, 实参为左值
- testForward(forward<int&&>(num));forward的模板类型为int&&, 最终会得到一个右值, 函数的形参为未定引用类型T&&被右值初始化后得到一个右值引用类型
 - printValue(v);已命名的右值v, 编译器会视为左值处理, 实参为左值
 - printValue(move(v));已命名的右值编译器会视为左值处理, 通过move又将其转换为右值, 实参为右值
 - printValue(forward(v));forward的模板参数为右值引用, 最终得到一个右值, 实参为右值

15. 智能指针

在C++中没有垃圾回收机制, 必须自己释放分配的内存, 否则就会造成内存泄露。解决这个问题最有效的方法是使用智能指针 (smart pointer)。**智能指针是存储指向动态分配 (堆) 对象指针的类, 用于生存期的控制, 能够确保在离开指针所在作用域时, 自动地销毁动态分配的对象, 防止内存泄露。智能指针的核心实现技术是引用计数 (共享内存计数器), 每使用它一次, 内部引用计数加1, 每析构一次内部的引用计数减1, 减为0时, 删除所指向的堆内存。**

C++11中提供了三种智能指针, 使用这些智能指针时需要引用头文件:

1. **std::shared_ptr: 共享的智能指针。**可以多个智能指针同时管理一片堆区, 其内部有共享内存计数器对不同指针进行管理, 当计数从1减为0之前, 会先释放堆区, 在减少指针。
2. **std::unique_ptr: 独占的智能指针。**只能有一个智能指针管理, 可以通过move转移堆区所有权。
3. **std::weak_ptr: 弱引用的智能指针, 它不共享指针, 不能操作资源, 是用来监视shared_ptr的。**

15.1 shared_ptr (共享智能指针)

15.1.1 初始化

共享智能指针是指多个智能指针可以同时管理同一块有效的内存，共享智能指针shared_ptr 是一个模板类，如果要进行初始化有四种方式：通过构造函数、std::make_shared辅助函数以及reset方法。共享智能指针对象初始化完毕之后就指向了要管理的那块堆内存，如果想要查看当前有多少个智能指针同时管理着这块内存可以使用共享智能指针提供的一个成员函数use_count，函数原型如下：

```
1 // 管理当前对象的 shared_ptr 实例数量，或若无被管理对象则为 0。
2 long use_count() const noexcept;
```

Fence 120

15.1.1.1 通过构造函数初始化

```
1 // shared_ptr <T> 类模板中，提供了多种实用的构造函数，语法格式如下：
2 std::shared_ptr <T> 智能指针名字(创建堆内存);
```

Fence 121

例子：

```
1 #include <iostream>
2 #include <memory>
3 using namespace std;
4
5 int main()
6 {
7     // 使用智能指针管理一块 int 型的堆内存
8     shared_ptr <int> ptr1(new int(520));
9     cout << "ptr1 管理的内存引用计数: " << ptr1.use_count() << endl;
10    // 使用智能指针管理一块字符数组对应的堆内存
11    shared_ptr <char> ptr2(new char [12]);
12    cout << "ptr2 管理的内存引用计数: " << ptr2.use_count() << endl;
13    // 创建智能指针对象，不管理任何内存
14    shared_ptr <int> ptr3;
15    cout << "ptr3 管理的内存引用计数: " << ptr3.use_count() << endl;
16    // 创建智能指针对象，初始化为空
17    shared_ptr <int> ptr4(nullptr);
18    cout << "ptr4 管理的内存引用计数: " << ptr4.use_count() << endl;
19
20    return 0;
21 }
```

Fence 122

结果：

```
1 ptr1 管理的内存引用计数: 1
2 ptr2 管理的内存引用计数: 1
3 ptr3 管理的内存引用计数: 0
4 ptr4 管理的内存引用计数: 0
```

Fence 123

如果智能指针被初始化了一块有效内存，那么这块内存的引用计数+1，如果智能指针没有被初始化或者被初始化为nullptr空指针，引用计数不会+1。另外，不要使用一个原始指针初始化多个shared_ptr。

```
1  int *p = new int;
2  shared_ptr <int> p1(p);
3  shared_ptr <int> p2(p);    // error, 编译不会报错, 运行会出错 使用注意事项中的第一点, 会导致重
    析构, 只能通过 p2 = p1 进行赋值
```

Fence 124

15.1.1.2 通过拷贝或者移动构造函数初始化

当一个智能指针被初始化之后，就可以通过这个智能指针初始化其他新对象。在创建新对象的时候，对应的拷贝构造函数或者移动构造函数就被自动调用了。

```
1  #include <iostream>
2  #include <memory>
3  using namespace std;
4
5  int main()
6  {
7      // 使用智能指针管理一块 int 型的堆内存, 内部引用计数为 1
8      shared_ptr <int> ptr1(new int(520));
9      cout << "ptr1 管理的内存引用计数: " << ptr1.use_count() << endl;
10     //调用拷贝构造函数
11     shared_ptr <int> ptr2(ptr1);
12     cout << "ptr2 管理的内存引用计数: " << ptr2.use_count() << endl;
13     shared_ptr <int> ptr3 = ptr1;
14     cout << "ptr3 管理的内存引用计数: " << ptr3.use_count() << endl;
15     //调用移动构造函数
16     shared_ptr <int> ptr4(std::move(ptr1));
17     cout << "ptr4 管理的内存引用计数: " << ptr4.use_count() << endl;
18     std::shared_ptr <int> ptr5 = std::move(ptr2);
19     cout << "ptr5 管理的内存引用计数: " << ptr5.use_count() << endl;
20
21     return 0;
22 }
```

Fence 125

结果:

```
1  ptr1 管理的内存引用计数: 1
2  ptr2 管理的内存引用计数: 2
3  ptr3 管理的内存引用计数: 3
4  ptr4 管理的内存引用计数: 3
5  ptr5 管理的内存引用计数: 3
```

Fence 126

如果使用拷贝的方式初始化共享智能指针对象，这两个对象会同时管理同一块堆内存，堆内存对应的引用计数也会增加；如果使用移动的方式初始智能指针对象，只是转让了内存的所有权，管理内存的对象并不会增加，因此内存的引用计数不会变化。

15.1.1.3 通过std::make_shared初始化

通过C++提供的std::make_shared() 就可以完成内存对象的创建并将其初始化给智能指针，函数原型如下：

```
1  template < class T, class... Args >
2  shared_ptr <T> make_shared( Args&&... args );
```

Fence 127

- **T**：模板参数的数据类型
- **Args&&... args**：要初始化的数据，如果是通过make_shared创建对象，需按照构造函数的参数列表指定

```
1  #include <iostream>
2  #include <string>
3  #include <memory>
4  using namespace std;
5
6  class Test
7  {
8  public:
9      Test()
10     {
11         cout << "construct Test..." << endl;
12     }
13     Test(int x)
14     {
15         cout << "construct Test, x = " << x << endl;
16     }
17     Test(string str)
18     {
19         cout << "construct Test, str = " << str << endl;
20     }
21     ~Test()
22     {
23         cout << "destruct Test ..." << endl;
24     }
25 };
26
27 int main()
28 {
29     // 使用智能指针管理一块 int 型的堆内存，内部引用计数为 1
30     shared_ptr <int> ptr1 = make_shared <int>(520);
31     cout << "ptr1 管理的内存引用计数: " << ptr1.use_count() << endl;
32
33     shared_ptr <Test> ptr2 = make_shared <Test>();
34     cout << "ptr2 管理的内存引用计数: " << ptr2.use_count() << endl;
35
36     shared_ptr <Test> ptr3 = make_shared <Test>(520);
37     cout << "ptr3 管理的内存引用计数: " << ptr3.use_count() << endl;
38
39     shared_ptr <Test> ptr4 = make_shared <Test>("我是要成为海贼王的男人!!!");
40     cout << "ptr4 管理的内存引用计数: " << ptr4.use_count() << endl;
41     return 0;
```

使用std::make_shared()模板函数可以完成内存地址的创建，并将最终得到的内存地址传递给共享智能指针对象管理。如果申请的内存是普通类型，通过函数的 () 可完成地址的初始化，如果要创建一个类对象，函数的 () 内部需要指定构造对象需要的参数，也就是类构造函数的参数。

15.1.1.4 通过 reset方法初始化(重置智能指针)

共享智能指针类提供的std::shared_ptr::reset方法函数原型如下：

```

1  void reset() noexcept;
2
3  template < class Y >
4  void reset( Y* ptr );
5
6  template < class Y, class Deleter >
7  void reset( Y* ptr, Deleter d );
8
9  template < class Y, class Deleter, class Alloc >
10 void reset( Y* ptr, Deleter d, Alloc alloc );
11
```

- ptr：指向要取得所有权的对象的指针
- d：指向要取得所有权的对象的指针
- alloc：内部存储所用的分配器

```

1  #include <iostream>
2  #include <string>
3  #include <memory>
4  using namespace std;
5
6  int main()
7  {
8      // 使用智能指针管理一块 int 型的堆内存，内部引用计数为 1
9      shared_ptr <int> ptr1 = make_shared <int>(520);
10     shared_ptr <int> ptr2 = ptr1;
11     shared_ptr <int> ptr3 = ptr1;
12     shared_ptr <int> ptr4 = ptr1;
13     cout << "ptr1 管理的内存引用计数: " << ptr1.use_count() << endl;
14     cout << "ptr2 管理的内存引用计数: " << ptr2.use_count() << endl;
15     cout << "ptr3 管理的内存引用计数: " << ptr3.use_count() << endl;
16     cout << "ptr4 管理的内存引用计数: " << ptr4.use_count() << endl;
17
18     ptr4.reset();
19     cout << "ptr1 管理的内存引用计数: " << ptr1.use_count() << endl;
20     cout << "ptr2 管理的内存引用计数: " << ptr2.use_count() << endl;
21     cout << "ptr3 管理的内存引用计数: " << ptr3.use_count() << endl;
22     cout << "ptr4 管理的内存引用计数: " << ptr4.use_count() << endl;
23
24     shared_ptr <int> ptr5;
```

```

25     ptr5.reset(new int(250));
26     cout << "ptr5 管理的内存引用计数: " << ptr5.use_count() << endl;
27
28     return 0;
29 }

```

Fence 130

结果:

```

1  ptr1 管理的内存引用计数: 4
2  ptr2 管理的内存引用计数: 4
3  ptr3 管理的内存引用计数: 4
4  ptr4 管理的内存引用计数: 4
5
6  ptr1 管理的内存引用计数: 3
7  ptr2 管理的内存引用计数: 3
8  ptr3 管理的内存引用计数: 3
9  ptr4 管理的内存引用计数: 0
10
11 ptr5 管理的内存引用计数: 1
12

```

Fence 131

对于一个未初始化的共享智能指针，可以通过reset方法来初始化，当智能指针中有值的时候，调用reset会使引用计数减1。

15.1.2 获取原始指针

通过智能指针可以管理一个普通变量或者对象的地址，此时原始地址就不可见了。当我们想要修改变量或者对象中的值的时候，就需要从智能指针对象中先取出数据的原始内存的地址再操作，解决方案是调用共享智能指针类提供的get()方法，其函数原型如下：

```

1  T* get() const noexcept;

```

Fence 132

```

1  #include <iostream>
2  #include <string>
3  #include <memory>
4  using namespace std;
5
6  int main()
7  {
8      int len = 128;
9      shared_ptr<char> ptr(new char [len]);
10     // 得到指针的原始地址，通过原始地址进行操作
11     char* add = ptr.get();
12     memset(add, 0, len);
13     strcpy_s(add, sizeof(char)*len "我是要成为海贼王的男人!!!"); //strcpy()在
14     cout << "string: " << add << endl;
15
16     // 通过智能指针进行操作

```

```

17     shared_ptr<int> p(new int);
18     *p = 100;
19     cout << *p.get() << " " << *p << endl;
20
21     return 0;
22 }
23

```

Fence 133

15.1.3 指定删除器

当智能指针管理的内存对应的引用计数变为0的时候，这块内存就会被智能指针析构掉了。另外，我们在初始化智能指针的时候也可以自己指定删除动作，**这个删除操作对应的函数被称之为删除器**，这个删除器函数本质是一个**回调函数（函数指针）**，我们只需要进行实现，其调用是由智能指针完成的。

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4
5  // 自定义删除器函数，释放 int 型内存
6  void deleteIntPtr(int* p)
7  {
8      delete p;
9      cout << "int 型内存被释放了...";
10 }
11
12 int main()
13 {
14     shared_ptr<int> ptr(new int(250), deleteIntPtr);
15     return 0;
16 }
17
18

```

Fence 134

删除器函数也可以是lambda表达式，因此代码也可以写成下面这样：

```

1  int main()
2  {
3      shared_ptr<int> ptr(new int(250), [](int* p) {delete p; });
4      return 0;
5  }
6

```

Fence 135

在上面的代码中，**lambda表达式的参数就是智能指针管理的内存的地址**，有了这个地址之后函数体内部就可以完成删除操作了。

在C++11中使用**shared_ptr**管理动态数组时，需要指定删除器，因为**std::shared_ptr**的默认删除器不支持数组对象，具体的处理代码如下：

```

1  int main()
2  {
3      shared_ptr<int> ptr(new int [10], [](int* p) {delete [] p; });
4      return 0;
5  }

```

Fence 136

在删除数组内存时，除了自己编写删除器，也可以使用C++提供的**std::default_delete()**函数作为删除器，这个函数内部的删除功能也是通过调用delete来实现的，要释放什么类型的内存就将模板类型T指定为什么类型即可。具体处理代码如下：

```

1  int main()
2  {
3      shared_ptr<int> ptr(new int [10], default_delete<int[]>());
4      return 0;
5  }

```

Fence 137

另外，我们还可以自己封装一个make_shared_array方法来让shared_ptr支持数组，代码如下：

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4
5  template <typename T>
6  shared_ptr<T> make_share_array(size_t size)
7  {
8      // 返回匿名对象
9      return shared_ptr<T>(new T [size], default_delete<T[]>());
10 }
11
12 int main()
13 {
14     shared_ptr<int> ptr1 = make_share_array<int>(10);
15     cout << ptr1.use_count() << endl;
16     shared_ptr<char> ptr2 = make_share_array<char>(128);
17     cout << ptr2.use_count() << endl;
18     return 0;
19 }

```

Fence 138

shared_ptr使用时的注意事项：

1. 不能使用一个原始地址初始化多个共享智能指针
2. 函数不能返回管理了this的共享智能指针对象（本质上就是第一点，返回的数据是原始地址，导致报错，如：
return shared_ptr(this); 或者使用shared_ptr ptr2(ptr1.get());
3. 共享智能指针不能循环引用（老王娶老李的闺女，老李娶老王的闺女，老王与老李同辈）

15.2 unique_ptr (独占智能指针)

15.2.1 初始化

std::unique_ptr是一个独占型的智能指针，它不允许其他的智能指针共享其内部的指针，可以通过它的构造函数初始化一个独占智能指针对象，但是**不允许通过赋值将一个unique_ptr赋值给另一个unique_ptr**。

```
1 // 通过构造函数初始化对象
2 unique_ptr<int> ptr1(new int(10));
3 // error, 不允许将一个 unique_ptr 赋值给另一个 unique_ptr
4 unique_ptr<int> ptr2 = ptr1;
```

Fence 139

std::unique_ptr不允许复制，但是可以通过函数返回给其他的std::unique_ptr，还可以通过std::move来转译给其他的std::unique_ptr，这样原始指针的所有权就被转移了，这个原始指针还是被独占的。

```
1 #include <iostream>
2 #include <memory>
3 using namespace std;
4
5 unique_ptr<int> func()
6 {
7     return unique_ptr<int>(new int(520));
8 }
9
10 int main()
11 {
12     // 通过构造函数初始化
13     unique_ptr<int> ptr1(new int(10));
14     // 通过转移所有权的方式初始化
15     unique_ptr<int> ptr2 = move(ptr1);
16     unique_ptr<int> ptr3 = func();
17
18     return 0;
19 }
```

Fence 140

unique_ptr独占智能指针类也有一个reset方法，函数原型如下：

```
1 void reset( pointer ptr = pointer() ) noexcept;
```

Fence 141

使用reset方法可以让unique_ptr解除对原始内存的管理，也可以用来初始化一个独占的智能指针。


```

1  int main()
2  {
3      unique_ptr<int> ptr1(new int(10));
4      unique_ptr<int> ptr2 = move(ptr1);
5
6      ptr1.reset();
7      ptr2.reset(new int(250));
8
9      return 0;
10 }

```

Fence 142

- ptr1.reset();解除对原始内存的管理
- ptr2.reset(new int(250));重新指定智能指针管理的原始内存

如果想要获取独占智能指针管理的原始地址，可以调用get()方法，函数原型如下：

```

1  pointer get() const noexcept;

```

Fence 143

```

1  int main()
2  {
3      unique_ptr<int> ptr1(new int(10));
4      unique_ptr<int> ptr2 = move(ptr1);
5
6      ptr2.reset(new int(250));
7      cout << *ptr2.get() << endl;    // 得到内存地址中存储的实际数值 250
8
9      return 0;
10 }

```

Fence 144

15.2.2 删除器

unique_ptr指定删除器和shared_ptr指定删除器是有区别的，**unique_ptr指定删除器的时候需要确定删除器的类型**，所以不能像shared_ptr那样直接指定删除器，举例说明：

```

1  shared_ptr<int> ptr1(new int(10), [](int*p) {delete p; }); // ok
2  unique_ptr<int> ptr1(new int(10), [](int*p) {delete p; }); // error
3
4  int main()
5  {
6      using func_ptr = void(*)(int*);
7      unique_ptr<int, func_ptr> ptr1(new int(10), [](int*p) {delete p; });
8
9      return 0;
10 }

```

Fence 145

在上面的代码中第7行，func_ptr的类型和lambda表达式的类型是一致的。在lambda表达式没有捕获任何变量的情况下是正确的，如果捕获了变量，编译时则会报错：

```

1  int main()
2  {
3      using func_ptr = void (*)(int*);
4      unique_ptr<int, func_ptr> ptr1(new int(10), [&](int*p) {delete p; }); // error
5      return 0;
6  }

```

Fence 146

上面的代码中错误原因是这样的，在lambda表达式没有捕获任何外部变量时，可以直接转换为函数指针，一旦捕获了就无法转换了（Lambda成为仿函数，需要包装器进转变成函数指针），如果想要让编译器成功通过编译，那么需要使用可调用对象包装器来处理声明的函数指针：

```

1  int main()
2  {
3      using func_ptr = void (*)(int*);
4      unique_ptr<int, function<void(int*)>> > ptr1(new int(10), [&](int*p) {delete p; });
5      return 0;
6  }
7

```

Fence 147

独占智能指针能够管路数据类型的地址，且能自动释放内存。

```

1  unique_ptr<int[]> ptr1(new int [5]);
2  // C++11 以后，共享智能指针也能自动释放数据类型内存
3  shared_ptr<int[]> ptr2(new int [5]);

```

Fence 148

15.3 weak_ptr（弱引用智能指针）

弱引用智能指针std::weak_ptr可以看做是shared_ptr的助手，它不管理shared_ptr内部的指针。std::weak_ptr没有重载操作符*和->，因为它不共享指针，不能操作资源，所以它的构造不会增加引用计数，析构也不会减少引用计数，它的主要作用就是作为一个旁观者监视shared_ptr中管理的资源是否存在。

15.3.1 初始化

```

1  // 默认构造函数
2  constexpr weak_ptr() noexcept;
3  // 拷贝构造
4  weak_ptr (const weak_ptr& x) noexcept;
5  template <class U> weak_ptr (const weak_ptr<U>& x) noexcept;
6  // 通过 shared_ptr 对象构造
7  template <class U> weak_ptr (const shared_ptr<U>& x) noexcept;

```

Fence 149

在C++11中，weak_ptr的初始化可以通过以上提供的构造函数来完成初始化，具体使用方法如下：

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4

```

```

5  int main()
6  {
7      shared_ptr <int> sp(new int);
8
9      weak_ptr <int> wp1;
10     weak_ptr <int> wp2(wp1);
11     weak_ptr <int> wp3(sp);
12     weak_ptr <int> wp4;
13     wp4 = sp;
14     weak_ptr <int> wp5;
15     wp5 = wp3;
16
17     return 0;
18 }

```

Fence 150

- weak_ptr wp1;构造了一个空weak_ptr对象
- weak_ptr wp2(wp1);通过一个空weak_ptr对象构造了另一个空weak_ptr对象
- weak_ptr wp3(sp);通过一个shared_ptr对象构造了一个可用的weak_ptr实例对象
- wp4 = sp;通过一个shared_ptr对象构造了一个可用的weak_ptr实例对象（这是一个隐式类型转换）
- wp5 = wp3;通过一个weak_ptr对象构造了一个可用的weak_ptr实例对象

15.3.2 其他常用方法

15.3.2.1 use_count()

通过调用std::weak_ptr类提供的use_count()方法可以获得当前所观测资源的引用计数，函数原型如下：

```

1  // 函数返回所监测的资源的引用计数
2  long int use_count() const noexcept;

```

Fence 151

修改一下上面的测试程序，添加打印资源引用计数的代码：

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4
5  int main()
6  {
7      shared_ptr <int> sp(new int);
8
9      weak_ptr <int> wp1;
10     weak_ptr <int> wp2(wp1);
11     weak_ptr <int> wp3(sp);
12     weak_ptr <int> wp4;
13     wp4 = sp;
14     weak_ptr <int> wp5;
15     wp5 = wp3;
16
17     cout << "use_count: " << endl;

```

```

18     cout << "wp1: " << wp1.use_count() << endl;
19     cout << "wp2: " << wp2.use_count() << endl;
20     cout << "wp3: " << wp3.use_count() << endl;
21     cout << "wp4: " << wp4.use_count() << endl;
22     cout << "wp5: " << wp5.use_count() << endl;
23     return 0;
24 }

```

Fence 152

结果:

```

1  use_count:
2  wp1: 0
3  wp2: 0
4  wp3: 1
5  wp4: 1
6  wp5: 1

```

Fence 153

通过打印的结果可以知道，虽然弱引用智能指针wp3、wp4、wp5监测的资源是同一个，但是它的引用计数并没有发生任何的变化，也进一步证明了**weak_ptr只是监测资源，并不管理资源**。

15.3.2.2 expired()

通过调用std::weak_ptr类提供的expired()方法来判断观测的**资源是否已经被释放**，函数原型如下：

```

1  // 返回 true 表示资源已经被释放，返回 false 表示资源没有被释放
2  bool expired() const noexcept;

```

Fence 154

函数的使用方法如下:

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4
5  int main()
6  {
7      shared_ptr<int> shared(new int(10));
8      weak_ptr<int> weak(shared);
9      cout << "1. weak " << (weak.expired() ? "is" : "is not") << " expired" << endl;
10
11     shared.reset();
12     cout << "2. weak " << (weak.expired() ? "is" : "is not") << " expired" << endl;
13
14     return 0;
15 }
16

```

Fence 155

结果:

```
1 | 1. weak is not expired
2 | 2. weak is expired
```

Fence 156

weak_ptr监测的就是shared_ptr管理的资源，当共享智能指针调用shared.reset();之后管理的资源被释放，因此weak.expired()函数的结果返回true，表示监测的资源已经不存在了。

15.3.2.3 lock()

通过调用std::weak_ptr类提供的lock()方法来获取管理所监测资源的shared_ptr对象，函数原型如下：

```
1 | shared_ptr <element_type> lock() const noexcept;
```

Fence 157

函数的使用方法如下：

```
1 | #include <iostream>
2 | #include <memory>
3 | using namespace std;
4 |
5 | int main()
6 | {
7 |     shared_ptr <int> sp1, sp2;
8 |     weak_ptr <int> wp;
9 |
10 |    sp1 = std::make_shared <int>(520);
11 |    wp = sp1;
12 |    sp2 = wp.lock();
13 |    cout << "use_count: " << wp.use_count() << endl;
14 |
15 |    sp1.reset();
16 |    cout << "use_count: " << wp.use_count() << endl;
17 |
18 |    sp1 = wp.lock();
19 |    cout << "use_count: " << wp.use_count() << endl;
20 |
21 |    cout << "*sp1: " << *sp1 << endl;
22 |    cout << "*sp2: " << *sp2 << endl;
23 |
24 |    return 0;
25 | }
```

Fence 158

测试代码输出的结果为：

```
1 | use_count: 2
2 | use_count: 1
3 | use_count: 2
4 | *sp1: 520
5 | *sp2: 520
```

Fence 159

- `sp2 = wp.lock();`通过调用`lock()`方法得到一个用于管理`weak_ptr`对象所监测的资源的共享智能指针对象，使用这个对象初始化`sp2`，此时所监测资源的引用计数为2
- `sp1.reset();`共享智能指针`sp1`被重置，`weak_ptr`对象所监测的资源的引用计数减1
- `sp1 = wp.lock();``sp1`重新被初始化，并且管理的还是`weak_ptr`对象所监测的资源，因此引用计数加1
- 共享智能指针对象`sp1`和`sp2`管理的是同一块内存，因此最终打印的内存中的结果是相同的，都是520

15.3.2.4 reset()

通过调用`std::weak_ptr`类提供的`reset()`方法来清空对象，使其不监测任何资源，函数原型如下：

```
1 | void reset() noexcept;
```

Fence 160

函数的使用是非常简单的，示例代码如下：

```
1 | #include <iostream>
2 | #include <memory>
3 | using namespace std;
4 |
5 | int main()
6 | {
7 |     shared_ptr <int> sp(new int(10));
8 |     weak_ptr <int> wp(sp);
9 |     cout << "1. wp " << (wp.expired() ? "is" : "is not") << " expired" << endl;
10 |
11 |     wp.reset();
12 |     cout << "2. wp " << (wp.expired() ? "is" : "is not") << " expired" << endl;
13 |
14 |     return 0;
15 | }
```

Fence 161

测试代码输出的结果为：

```
1 | 1. wp is not expired
2 | 2. wp is expired
```

Fence 162

`weak_ptr`对象`sp`被重置之后`wp.reset();`变成了空对象，不再监测任何资源，因此`wp.expired()`返回`true`

15.3.3 返回管理this的shared_ptr

如果在一个类中编写了一个函数，通过这个得到管理当前对象的共享智能指针，我们可能会写出如下代码：

```
1 | #include <iostream>
2 | #include <memory>
3 | using namespace std;
4 |
5 | struct Test
6 | {
7 |     shared_ptr <Test> getSharedPtr()
```

```

8      {
9          return shared_ptr <Test>(this);          // 相当于在该原始地址上再创建了一个共享
智能指针
10     }
11
12     ~Test()
13     {
14         cout << "class Test is disstruct ..." << endl;
15     }
16
17 };
18
19 int main()
20 {
21     shared_ptr <Test> sp1(new Test);
22     cout << "use_count: " << sp1.use_count() << endl;
23     shared_ptr <Test> sp2 = sp1-> getSharedPtr();    // 使用-> 调用的是 Test 里面
的方法或者属性, 使用. 调用的是 shared_ptr 中的方法或者属性
24     cout << "use_count: " << sp1.use_count() << endl;
25     return 0;
26 }

```

Fence 163

执行上面的测试代码, 运行中会出现异常, 在终端还是能看到对应的日志输出:

```

1  use_count: 1
2  use_count: 1
3  class Test is disstruct ...
4  class Test is disstruct ...

```

Fence 164

通过输出的结果可以看到一个对象被析构了两次, 其原因是这样的: 在这个例子中使用同一个指针this构造了两个智能指针对象sp1和sp2, 这二者之间是没有任何关系的, 因为sp2并不是通过sp1初始化得到的实例对象。在离开作用域之后this将被构造的两个智能指针各自析构, 导致重复析构的错误。

这个问题可以通过weak_ptr来解决, 通过weak_ptr返回管理this资源的共享智能指针对象shared_ptr。C++11中为我们提供了一个模板类叫做std::enable_shared_from_this, 这个类中有一个方法叫做shared_from_this(), 通过这个方法可以返回一个共享智能指针, 在函数的内部就是使用weak_ptr来监测this对象, 并通过调用weak_ptr的lock()方法返回一个shared_ptr对象。

修改之后的代码为:

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4
5  struct Test : public enable_shared_from_this <Test>
6  {
7      shared_ptr <Test> getSharedPtr()
8      {
9          return shared_from_this();
10     }
11     ~Test()

```

```

12     {
13         cout << "class Test is disstruct ..." << endl;
14     }
15 };
16
17 int main()
18 {
19     shared_ptr <Test> sp1(new Test);           // enable_shared_from_this
20     <Test> 里面的 weak_ptr 关联 Test (初始化)
21     cout << "use_count: " << sp1.use_count() << endl;
22     shared_ptr <Test> sp2 = sp1-> getSharedPtr();
23     cout << "use_count: " << sp1.use_count() << endl;
24     return 0;
25 }

```

Fence 165

测试代码输出的结果为:

```

1 use_count: 1
2 use_count: 2
3 class Test is disstruct ...

```

Fence 166

最后需要强调一个细节：在调用enable_shared_from_this类的shared_from_this()方法之前，必须要先初始化函数内部weak_ptr对象，否则该函数无法返回一个有效的shared_ptr对象（具体处理方法可以参考上面的示例代码）。

15.3.4 解决循环引用问题

智能指针如果循环引用会导致内存泄露，比如下面的例子：

```

1 #include <iostream>
2 #include <memory>
3 using namespace std;
4
5 struct TA;
6 struct TB;
7
8 struct TA
9 {
10     shared_ptr <TB> bptr;
11     ~TA()
12     {
13         cout << "class TA is disstruct ..." << endl;
14     }
15 };
16
17 struct TB
18 {
19     shared_ptr <TA> aptr;
20     ~TB()
21     {
22         cout << "class TB is disstruct ..." << endl;
23     }
24 };

```



```

23     }
24 };
25
26 void testPtr()
27 {
28     shared_ptr <TA> ap(new TA);
29     shared_ptr <TB> bp(new TB);
30     cout << "TA object use_count: " << ap.use_count() << endl;
31     cout << "TB object use_count: " << bp.use_count() << endl;
32
33     ap-> bptr = bp;
34     bp-> aptr = ap;
35     cout << "TA object use_count: " << ap.use_count() << endl;
36     cout << "TB object use_count: " << bp.use_count() << endl;
37 }
38
39 int main()
40 {
41     testPtr();
42     return 0;
43 }

```

Fence 167

测试程序输出的结果如下:

```

1  TA object use_count: 1
2  TB object use_count: 1
3  TA object use_count: 2
4  TB object use_count: 2

```

Fence 168

在测试程序中，共享智能指针ap、bp对TA、TB实例对象的引用计数变为2，在共享智能指针离开作用域之后引用计数只能减为1，这种情况下不会去删除智能指针管理的内存，导致类TA、TB的实例对象**不能被析构**，最终造成内存泄露。通过使用weak_ptr可以解决这个问题，只要将类TA或者TB的任意一个成员改为weak_ptr，修改之后的代码如下:

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4
5  struct TA;
6  struct TB;
7
8  struct TA
9  {
10     weak_ptr <TB> bptr;
11     ~TA()
12     {
13         cout << "class TA is disstruct ..." << endl;
14     }
15 };
16

```

```

17 struct TB
18 {
19     shared_ptr <TA> aptr;
20     ~TB()
21     {
22         cout << "class TB is disstruct ..." << endl;
23     }
24 };
25
26 void testPtr()
27 {
28     shared_ptr <TA> ap(new TA);
29     shared_ptr <TB> bp(new TB);
30     cout << "TA object use_count: " << ap.use_count() << endl;
31     cout << "TB object use_count: " << bp.use_count() << endl;
32
33     ap-> bptr = bp;
34     bp-> aptr = ap;
35     cout << "TA object use_count: " << ap.use_count() << endl;
36     cout << "TB object use_count: " << bp.use_count() << endl;
37 }
38
39 int main()
40 {
41     testPtr();
42     return 0;
43 }

```

Fence 169

程序输出的结果:

```

1 TA object use_count: 1
2 TB object use_count: 1
3 TA object use_count: 2
4 TB object use_count: 1
5 class TB is disstruct ...
6 class TA is disstruct ...

```

Fence 170

通过输出的结果可以看到类TA或者TB的对象被成功析构了。

上面程序中，在对类TA成员赋值时ap->bptr = bp;由于bptr是weak_ptr类型，这个赋值操作并不会增加引用计数，所以bp的引用计数仍然为1，在离开作用域之后bp的引用计数减为0，类TB的实例对象被析构。

在类TB的实例对象被析构的时候，内部的aptr也被析构，其对TA对象的管理解除，内存的引用计数减为1，当共享智能指针ap离开作用域之后，对TA对象的管理也解除了，内存的引用计数减为0，类TA的实例对象被析构。

16. chrono库

C++11中提供了日期和时间相关的库chrono，通过chrono库可以很方便地处理日期和时间，为程序的开发提供了便利。chrono库主要包含三种类型的类：**时间间隔duration**、**时钟clocks**、**时间点time point**。

16.1 时间间隔duration

16.1.1 常用类成员

duration表示一段时间间隔，用来记录时间长度，可以表示几秒、几分钟、几个小时的时间间隔。duration的原型如下：

```
1 // 定义于头文件 <chrono>
2 template <
3     class Rep,
4     class Period = std:: ratio <1>
5 > class duration;
```

Fence 171

- Rep：这是一个数值类型，表示时钟数（周期）的类型（默认为整形）。若 Rep 是浮点数，则 duration 能使用小数描述时钟周期的数目。
- Period：表示时钟的周期，它的原型如下：

```
1 // 定义于头文件 <ratio>
2 template <
3     std:: intmax_t Num,
4     std:: intmax_t Denom = 1
5 > class ratio;
```

Fence 172

ratio类表示每个时钟周期的秒数，其中第一个模板参数Num代表分子，Denom代表分母，该分母值默认为1，因此，ratio代表的是一个分子除以分母的数值，比如：ratio<2>代表一个时钟周期是2秒，ratio<60>代表一分钟，ratio<60*60>代表一个小时，ratio<60*60*24>代表一天。而ratio<1,1000>代表的是1/1000秒，也就是1毫秒，ratio<1,1000000>代表一微秒，ratio<1,1000000000>代表一纳秒。

为了方便使用，在标准库中定义了一些常用的时间间隔，比如：时、分、秒、毫秒、微秒、纳秒，它们都位于chrono命名空间下，定义如下（两根/包裹的是Rep的最大数据类型长度，即long long或者int）：

类型	其官方内部定义
纳秒：std::chrono::nanoseconds	duration<Rep/至少 64 位的有符号整数类型/, std::nano>
微秒：std::chrono::microseconds	duration<Rep/至少 55 位的有符号整数类型/, std::micro>
毫秒：std::chrono::milliseconds	duration<Rep/至少 45 位的有符号整数类型/, std::milli>
秒：std::chrono::seconds	duration<Rep/至少 35 位的有符号整数类型/>
分钟：std::chrono::minutes	duration<Rep/至少 29 位的有符号整数类型/, std::ratio<60>>
小时：std::chrono::hours	duration<Rep/至少 23 位的有符号整数类型/, std::ratio<3600>>

Table 1

注意：到 hours 为止的每个预定义时长类型至少涵盖 ±292 年的范围。

duration类的构造函数原型如下：

```
1 // 1. 拷贝构造函数
2 duration( const duration& ) = default;
3 // 2. 通过指定时钟周期的类型来构造对象
4 template < class Rep2 >
5 constexpr explicit duration( const Rep2& r );
6 // 3. 通过指定时钟周期类型, 和时钟周期长度来构造对象
7 template < class Rep2, class Period2 >
8 constexpr duration( const duration <Rep2,Period2>& d );
```

Fence 173

为了更加方便的进行duration对象之间的操作，类内部进行了操作符重载：

操作符重载	描述
operator=	赋值内容 (公开成员函数)
operator+ operator-	实现一元 + 和一元 - (公开成员函数)
operator++ operator++(int) operator-- operator--(int)	递增或递减周期计数 (公开成员函数)
operator+= operator-= operator*= operator/= operator% =	实现二个时长间的复合赋值 (公开成员函数)

Table 2

duration类还提供了获取时间间隔的时钟周期数（周期次数）的方法count()，函数原型如下：

```
1 | constexpr rep count() const;
```

Fence 174

16.1.2 类的使用

通过构造函数构造事件间隔对象示例代码如下：

```
1 #include <chrono>
2 #include <iostream>
3 using namespace std;
4 int main()
5 {
6     chrono:: hours h(1); // 一小时 里面直接给的总时间
7     chrono:: milliseconds ms{ 3 }; // 3 毫秒 使用了列表初始化
8     chrono:: duration <int, ratio<1000> > ks(3); // 3000 秒
9
10    // chrono:: duration <int, ratio<1000> > d3(3.5); // error
11    chrono:: duration <double> dd(6.6); // 6.6 秒
```

```

12
13 // 使用小数表示时钟周期的次数
14 chrono::duration <double, std::ratio<1, 30> > hz(3.5);
15 }

```

Fence 175

- h(1)时钟周期为1小时，共有1个时钟周期，所以h表示的时间间隔为1小时
- ms(3)时钟周期为1毫秒，共有3个时钟周期，所以ms表示的时间间隔为3毫秒
- ks(3)时钟周期为1000秒，一共有三个时钟周期，所以ks表示的时间间隔为3000秒
- d(3.5)时钟周期为1000秒，时钟周期数量只能用整形来表示，但是此处指定的是浮点数，因此语法错误
- dd(6.6)时钟周期为默认的1秒，共有6.6个时钟周期，所以dd表示的时间间隔为6.6秒
- hz(3.5)时钟周期为1/30秒，共有3.5个时钟周期，所以hz表示的时间间隔为1/30*3.5秒

chrono库中根据duration类封装了不同长度的时钟周期（也可以自定义），基于这个时钟周期再进行周期次数的设置就可以得到总的时间间隔了（时钟周期 * 周期次数 = 总的时间间隔）。

示例代码如下：

```

1  #include <chrono>
2  #include <iostream>
3  int main()
4  {
5      std::chrono::milliseconds ms{3}; // 3 毫秒
6      std::chrono::microseconds us = 2*ms; // 6000 微秒
7      // 时间间隔周期为 1/30 秒
8      std::chrono::duration <double, std::ratio<1, 30> > hz(3.5);
9
10     std::cout << "3 ms duration has " << ms.count() << " ticks\n"
11               << "6000 us duration has " << us.count() << " ticks\n"
12               << "3.5 hz duration has " << hz.count() << " ticks\n";
13 }

```

Fence 176

输出的结果为：

```

1  3 ms duration has 3 ticks
2  6000 us duration has 6000 ticks
3  3.5 hz duration has 3.5 ticks

```

Fence 177

- ms时间单位为毫秒，初始化操作ms{3}表示时间间隔为3毫秒，一共有3个时间周期，每个周期为1毫秒
- us时间单位为微秒，初始化操作2*ms表示时间间隔为6000微秒，一共有6000个时间周期，每个周期为1微秒
- hz时间单位为秒，初始化操作hz(3.5)表示时间间隔为1/30*3.5秒，一共有3.5个时间周期，每个周期为1/30秒

由于在duration类内部做了操作符重载，因此时间间隔之间可以直接进行算术运算，比如我们要计算两个时间间隔的差值，就可以在代码中做如下处理：

```

1  #include <iostream>
2  #include <chrono>
3  using namespace std;

```

```

4
5  int main()
6  {
7      chrono:: minutes t1(10);
8      chrono:: seconds t2(60);
9      chrono:: seconds t3 = t1 - t2;
10     cout << t3.count() << " second" << endl;
11 }
12
13 程序输出的结果:
14 540 second

```

Fence 178

在上面的测试程序中，t1代表10分钟，t2代表60秒，t3是t1减去t2，也就是 $60 \times 10 - 60 = 540$ ，这个540表示的时钟周期，每个时钟周期是1秒，因此两个时间间隔之间的差值为540秒。

注意事项：duration的加减运算有一定的规则，当两个duration时钟周期不相同的时候，会先统一成一种时钟，然后再进行算术运算，统一的规则如下：假设有ratio<x1,y1> 和 ratio<x2,y2>两个时钟周期，首先需要求出x1, x2的最大公约数X，然后求出y1, y2的最小公倍数Y，统一之后的时钟周期ratio为ratio<X,Y>。

```

1  #include <iostream>
2  #include <chrono>
3  using namespace std;
4
5  int main()
6  {
7      chrono:: duration <double, ratio<9, 7> > d1(3);
8      chrono:: duration <double, ratio<6, 5> > d2(1);
9      // d1 和 d2 统一之后的时钟周期
10     chrono:: duration <double, ratio<3, 35> > d3 = d1 - d2;
11 }

```

Fence 179

对于分子6、9最大公约数为3，对于分母7、5最小公倍数为35，因此推导出的时钟周期为ratio<3,35>，d3约为31

16.2 时间点 time point

chrono库中提供了一个表示时间点的类time_point，该类的定义如下：

```

1  // 定义于头文件 <chrono>
2  template <
3      class Clock,
4      class Duration = typename Clock:: duration
5  > class time_point;

```

Fence 180

它被实现成如同存储一个 Duration 类型的自 Clock 的纪元起始开始的时间间隔的值，通过这个类最终可以得到时间中的某一个时间点。

- Clock：此时间点在此时钟上计量
- Duration：用于计量从纪元起时间的 std::chrono::duration 类型

time_point类的构造函数原型如下：

```
1 // 1. 构造一个以新纪元(epoch, 即: 1970.1.1)作为值的对象, 需要和时钟类一起使用, 不能单独使用该无参构造函数
2 time_point();
3 // 2. 构造一个对象, 表示一个时间点, 其中 d 的持续时间从 epoch 开始, 需要和时钟类一起使用, 不能单独使用该构造函数
4 explicit time_point( const duration& d );
5 // 3. 拷贝构造函数, 构造与 t 相同时间点的对象, 使用的时候需要指定模板参数
6 template < class Duration2 >
7 time_point( const time_point <Clock,Duration2>& t );
```

Fence 181

在这个类中除了构造函数还提供了另外一个time_since_epoch()函数, 用来获得1970年1月1日到time_point对象中记录的时间经过的时间间隔 (duration) , 函数原型如下:

```
1 duration time_since_epoch() const;
```

Fence 182

除此之外, 时间点time_point对象和时间段对象duration之间还支持直接进行算术运算(即加减运算), 时间点对象之间可以进行逻辑运算, 具体细节可以参考下面的表格:

其中 tp 和 tp2 是time_point 类型的对象, dtn 是duration类型的对象。

描述	操作	返回值
复合赋值(成员函数) operator+=	tp += dtn	*this
复合赋值(成员函数) operator-=	tp -= dtn	*this
算术运算符(非成员函数) operator+	tp + dtn	a time_point value
算术运算符(非成员函数) operator+	dtn + tp	a time_point value
算术运算符(非成员函数) operator-	tp - dtn	a time_point value
算术运算符(非成员函数) operator-	tp - tp2	a duration value
关系操作符(非成员函数) operator==	tp == tp2	a bool value
关系操作符(非成员函数) operator!=	tp != tp2	a bool value
关系操作符(非成员函数) operator<	tp < tp2	a bool value
关系操作符(非成员函数) operator>	tp > tp2	a bool value
关系操作符(非成员函数) operator>=	tp >= tp2	a bool value
关系操作符(非成员函数) operator<=	tp <= tp2	a bool value

Table 3

该时间点类经常和下面的时钟类一起使用。

16.3 时钟类clocks

chrono库中提供了获取当前的系统时间的时钟类，包含的时钟一共有三种：

- system_clock：系统的时钟，系统的时钟可以修改，甚至可以网络对时，因此使用系统时间计算时间差可能不准。
- steady_clock：是固定的时钟，相当于秒表。开始计时后，时间只会增长并且不能修改，**适用于记录程序耗时**
- high_resolution_clock：和时钟类 steady_clock 是等价的（是它的别名）。

在这些时钟类的内部有time_point、duration、Rep、Period等信息，基于这些信息来获取当前时间，以及实现time_t和time_point之间的相互转换。

时钟类成员类型	描述
rep	表示时钟周期次数的有符号算术类型
period	表示时钟计次周期的 std::ratio 类型
duration	时间间隔，可以表示负时长
time_point	表示在当前时钟里边记录的时间点

Table 4

在使用chrono提供的时钟类的时候，不需要创建类对象，直接调用类的静态方法就可以得到想要的时间了。

16.3.1 system_clock

具体来说，时钟类system_clock是一个系统范围的实时时钟。system_clock提供了对当前时间点time_point的访问，将得到时间点转换为time_t类型的时间对象，就可以基于这个时间对象获取到当前的时间信息了。

system_clock时钟类在底层源码中的定义如下：

```
1 struct system_clock { // wraps GetSystemTimePreciseAsFileTime/GetSystemTimeAsFileTime
2     using rep = long long;
3     using period = ratio <1, 10'000'000>; // 100 nanoseconds
4     using duration = chrono:: duration <rep, period>;
5     using time_point = chrono:: time_point <system_clock>;
6     static constexpr bool is_steady = false;
7
8     _NODISCARD static time_point now() noexcept
9     { // get current time
10         return time_point(duration(_Xtime_get_ticks()));
11     }
12
13     _NODISCARD static __time64_t to_time_t(const time_point& _Time) noexcept
14     { // convert to __time64_t
15         return duration_cast <seconds>(_Time.time_since_epoch()).count();
16     }
17
18     _NODISCARD static time_point from_time_t(__time64_t _Tm) noexcept
19     { // convert from __time64_t
20         return time_point{seconds{_Tm}};
```



```

21     }
22 };

```

Fence 183

通过以上源码可以了解到在system_clock类中的一些细节信息：

- rep：时钟周期次数是通过整形来记录的long long
- period：一个时钟周期是100纳秒ratio<1, 10'000'000>
- duration：时间间隔为rep*period纳秒chrono::duration<rep, period>
- time_point：时间点通过系统时钟做了初始化chrono::time_point<system_clock>，里面记录了新纪元时间点

另外还可以看到system_clock类一共提供了三个静态成员函数：

```

1  // 返回表示当前时间的时间点。
2  static std::chrono::time_point<std::chrono::system_clock> now() noexcept;
3  // 将 time_point 时间点类型转换为 std::time_t 类型
4  static std::time_t to_time_t( const time_point& t ) noexcept;
5  // 将 std::time_t 类型转换为 time_point 时间点类型
6  static std::chrono::system_clock::time_point from_time_t( std::time_t t ) noexcept;

```

Fence 184

比如，我们要获取当前的系统时间，并且需要将其以能够识别的方式打印出来，示例代码如下：

```

1  #include <chrono>
2  #include <iostream>
3  using namespace std;
4  using namespace std::chrono;
5  int main()
6  {
7      // 新纪元 1970.1.1 时间
8      system_clock::time_point epoch;
9
10     duration<int, ratio<60*60*24>> day(1);
11     // 新纪元 1970.1.1 时间 + 1 天
12     system_clock::time_point ppt(day);
13
14     using dday = duration<int, ratio<60 * 60 * 24>>;
15     // 新纪元 1970.1.1 时间 + 10 天
16     time_point<system_clock, dday> t(dday(10));
17
18     // 系统当前时间
19     system_clock::time_point today = system_clock::now();
20
21     // 转换为 time_t 时间类型
22     time_t tm = system_clock::to_time_t(today);
23     cout << "今天的日期是： " << ctime(&tm);
24
25     time_t tm1 = system_clock::to_time_t(today+day);
26     cout << "明天的日期是： " << ctime(&tm1);
27
28     time_t tm2 = system_clock::to_time_t(epoch);
29     cout << "新纪元时间： " << ctime(&tm2);

```

```

30
31     time_t tm3 = system_clock:: to_time_t(ppt);
32     cout << "新纪元时间+1 天: " << ctime(&tm3);
33
34     time_t tm4 = system_clock:: to_time_t(t);
35     cout << "新纪元时间+10 天: " << ctime(&tm4);
36
37     // vs2022 中使用 ctime 会被认为不安全, 可以使用 ctime_s();
38     // char buf [26];
39     // ctime_s(buf, sizeof(buf), &tm4);
40     // cout << "新纪元时间+10 天: " << buf;
41 }

```

Fence 185

示例代码打印的结果为:

```

1  今天的日期是:    Thu Jul  3 13:35:02 2025
2  明天的日期是:    Fri Jul  4 13:35:02 2025
3  新纪元时间:      Thu Jan  1 08:00:00 1970
4  新纪元时间+1 天:  Fri Jan  2 08:00:00 1970
5  新纪元时间+10 天: Sun Jan 11 08:00:00 1970

```

Fence 186

16.3.2 steady_clock

如果我们通过时钟不是为了获取当前的系统时间, 而是进行程序耗时的时长, 此时使用system_clock就不合适了, 因为这个时间可以跟随系统的设置发生变化。在C++11中提供的**时钟类steady_clock**相当于秒表, 只要启动就会进行时间的累加, 并且不能被修改, 非常适合于进行耗时的统计。

steady_clock时钟类在底层源码中的定义如下:

```

1  struct steady_clock { // wraps QueryPerformanceCounter
2      using rep          = long long;
3      using period       = nano;
4      using duration     = nanoseconds;
5      using time_point   = chrono:: time_point <steady_clock>;
6      static constexpr bool is_steady = true;
7
8      // get current time
9      _NODISCARD static time_point now() noexcept
10     {
11         // doesn't change after system boot
12         const long long _Freq = _Query_perf_frequency();
13         const long long _Ctr  = _Query_perf_counter();
14         static_assert(period:: num == 1, " This assumes period:: num == 1.");
15         const long long _Whole = (_Ctr / _Freq) * period:: den;
16         const long long _Part  = (_Ctr % _Freq) * period:: den / _Freq;
17         return time_point(duration(_Whole + _Part));
18     }
19 };

```

Fence 187

通过以上源码可以了解到在steady_clock类中的一些细节信息:

- rep: 时钟周期次数是通过整形来记录的long long
- period: 一个时钟周期是1纳秒nano
- duration: 时间间隔为1纳秒nanoseconds
- time_point: 时间点通过系统时钟做了初始化chrono::time_point<steady_clock>

另外，在这个类中也提供了一个静态的**now()**方法，用于得到当前的时间点，函数原型如下：

```
1 | static std::chrono::time_point <std::chrono::steady_clock> now() noexcept;
```

Fence 188

假设要测试某一段程序的执行效率，可以计算它执行期间消耗的总时长，示例代码如下：

```
1 | #include <chrono>
2 | #include <iostream>
3 | using namespace std;
4 | using namespace std:: chrono;
5 | int main()
6 | {
7 |     // 获取开始时间点
8 |     steady_clock:: time_point start = steady_clock:: now();
9 |     // 执行业务流程
10 |    cout << "print 1000 stars ...." << endl;
11 |    for (int i = 0; i < 1000; ++i)
12 |    {
13 |        cout << "*";
14 |    }
15 |    cout << endl;
16 |    // 获取结束时间点
17 |    steady_clock:: time_point last = steady_clock:: now();
18 |    // 计算差值
19 |    auto dt = last - start;          // dt 类型为 duration (时间段)
20 |    cout << "总共耗时: " << dt.count() << "纳秒" << endl;
21 | }
```

Fence 189

16.3.3 high_resolution_clock

high_resolution_clock提供的时钟精度比system_clock要高，它也是不可以修改的。在底层源码中，这个类其实是steady_clock类的别名。

```
1 | using high_resolution_clock = steady_clock;
```

Fence 190

此high_resolution_clock的使用方式和steady_clock是一样的。

16.4 转换函数

16.4.1 duration_cast

`duration_cast`是chrono库提供的一个模板函数，这个函数不属于duration类。通过这个函数可以对duration类对象内部的时钟周期Period，和周期次数的类型Rep进行修改，该函数原型如下：

```
1  template <class ToDuration, class Rep, class Period>
2  constexpr ToDuration duration_cast (const duration <Rep,Period>& dtn);
```

Fence 191

使用duration与duration_cast的场景：

1. 如果是对时钟周期进行转换：源时钟周期必须能够整除目的时钟周期（比如：小时到分钟（时间大单位转小单位））。
2. 如果是对时钟周期次数的类型进行转换：低等类型默认可以向高等类型进行转换（比如：int 转 double）。
3. 如果时钟周期和时钟周期次数类型都变了，根据第二点进行推导（也就是看时间周期次数类型）。
4. 以上条件都不满足，那么就需要使用 `duration_cast` 进行显示转换。

我们可以修改一下上面测试程序执行时间的代码，在代码中修改duration对象的属性：

```
1  #include <iostream>
2  #include <chrono>
3  using namespace std;
4  using namespace std:: chrono;
5
6  void f()
7  {
8      cout << "print 1000 stars ...." << endl;
9      for (int i = 0; i < 1000; ++i)
10     {
11         cout << "*";
12     }
13     cout << endl;
14 }
15
16 int main()
17 {
18     auto t1 = steady_clock:: now();
19     f();
20     auto t2 = steady_clock:: now();
21
22     // 整数时长: 时钟周期纳秒转毫秒, 要求 duration_cast
23     auto int_ms = duration_cast <chrono::milliseconds>(t2 - t1);
24
25     // 小数时长: 不要求 duration_cast
26     duration <double, ratio<1, 1000> > fp_ms = t2 - t1;
27
28     cout << "f() took " << fp_ms.count() << " ms, "
29          << "or " << int_ms.count() << " whole milliseconds\n";
30 }
```



```

17 {
18     TimePoint <Sec> time_point_sec(Sec(6));
19     // 无精度损失，可以进行隐式类型转换（秒转毫秒）
20     TimePoint <Ms> time_point_ms(time_point_sec);
21     print_ms(time_point_ms);    // 6000 ms
22
23     time_point_ms = TimePoint <Ms>(Ms(6789));
24     // error，会损失精度，不允许进行隐式的类型转换（毫秒转秒）
25     TimePoint <Sec> sec(time_point_ms);
26
27     // 显示类型转换，会损失精度。6789 truncated to 6000（毫秒转秒）
28     time_point_sec = std::chrono::time_point_cast<Sec>(time_point_ms);
29     print_ms(time_point_sec); // 6000 ms
30 }

```

Fence 195

注意事项：关于时间点的转换如果没有精度的损失可以直接进行隐式类型转换，如果会损失精度只能通过显示类型转换，也就是调用time_point_cast函数来完成该操作。

17. 数字类型与字符串之间的转换

17.1 数值转换为字符串

使用to_string()方法可以非常方便地将各种数值类型转换为字符串类型，这是一个重载函，函数声明位于头文件中，函数原型如下：

```

1 // 头文件 <string>
2 string to_string (int val);
3 string to_string (long val);
4 string to_string (long long val);
5 string to_string (unsigned val);
6 string to_string (unsigned long val);
7 string to_string (unsigned long long val);
8 string to_string (float val);
9 string to_string (double val);
10 string to_string (long double val);

```

Fence 196

关于函数的使用是非常简单的，示例代码如下：

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main()
6 {
7     string pi = "pi is " + to_string(3.1415926);
8     string love = "love is " + to_string(5.20 + 13.14);
9     cout << pi << endl;
10    cout << love << endl;
11    return 0;
12 }

```

17.2 字符串转换为数值

Fence 197

由于C++中的数值类型包括整形和浮点型，因此针对于不同的类型提供了不同的函数，通过调用这些函数可以将字符串类型转换为对应的数值类型。

```
1 // 定义于头文件 <string>
2 int      stoi( const std:: string& str, std:: size_t* pos = 0, int base = 10 );
3 long     stol( const std:: string& str, std:: size_t* pos = 0, int base = 10 );
4 long long stoll( const std:: string& str, std:: size_t* pos = 0, int base = 10 );
5
6 unsigned long      stoul( const std:: string& str, std:: size_t* pos = 0, int base =
7 10 );
8 unsigned long long stoull( const std:: string& str, std:: size_t* pos = 0, int base =
9 10 );
10
11 float      stof( const std:: string& str, std:: size_t* pos = 0 );
12 double     stod( const std:: string& str, std:: size_t* pos = 0 );
13 long double stold( const std:: string& str, std:: size_t* pos = 0 );
```

Fence 198

- str: 要转换的字符串
- pos: 传出参数, 记录从哪个字符开始无法继续进行解析, 比如: 123abc, 传出的位置为3
- base: 若 base 为 0, 则自动检测数值进制: 若前缀为 0, 则为八进制, 若前缀为 0x 或 0X, 则为十六进制, 否则为十进制。

这些函数虽然都有多个参数，但是除去第一个参数外其他都有默认值，一般情况下使用默认值就能满足需求。关于函数的使用也给大家提供了一个例子，示例代码如下：

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 int main()
5 {
6     string str1 = "45";
7     string str2 = "3.14159";
8     string str3 = "9527 with words";
9     string str4 = "words and 2";
10
11     int myint1 = std:: stoi(str1);
12     float myint2 = std:: stof(str2);
13     int myint3 = std:: stoi(str3);
14     // 错误: 抛出异常'std:: invalid_argument'
15     // int myint4 = std:: stoi(str4);
16     // 使用 std:: istringstream (会从字符串中提取第一个数字, 直到非数字结束) 可以解决, 在复杂一些
17     // 则需要使用正则表达式
18     // std:: istringstream iss(str4);
19
20     cout << "std:: stoi(\"" << str1 << "\") is " << myint1 << endl;
21     cout << "std:: stof(\"" << str2 << "\") is " << myint2 << endl;
22     cout << "std:: stoi(\"" << str3 << "\") is " << myint3 << endl;
23     // cout << "std:: stoi(\"" << str4 << "\") is " << myint4 << endl;
```

示例代码输入的结果如下：

```
1 std::stoi("45") is 45
2 std::stof("3.14159") is 3.14159
3 std::stoi("9527 with words") is 9527
```

从上述测试程序可以得出这样的结论，在C++11提供的这些转换函数将字符串转换为数值的过程中：

- 如果字符串中所有字符都是数值类型，整个字符串会被转换为对应的数值，并通过返回值返回
- 如果字符串的前半部分字符是数值类型，后半部不是，那么前半部分会被转换为对应的数值，并通过返回值返回
- 如果字符第一个字符不是数值类型转换失败

18. POD类型

POD是英文中 Plain Old Data 的缩写，翻译过来就是普通的旧数据。POD在C++中是非常重要的一个概念，通常用于说明一个类型的属性，尤其是用户自定义类型的属性。

POD属性在C++11中往往又是构建其他C++概念的基础，事实上，在C++11标准中，POD出现的概率相当高。因此学习C++，尤其是在C++11中，了解POD的概念是非常必要的。

- Plain：表示是个普通的类型
- Old：体现了其与C的兼容性，支持标准C函数

在C++11中将POD划分为两个基本概念的合集，即：**平凡的（trivial）**和**标准布局的（standard layout）**。

18.1 “平凡”类型

一个平凡的类或者结构体应该符合以下几点要求：

1. 有平凡的默认构造函数（trivial constructor）和析构函数（trivial destructor）。

平凡的默认构造函数就是说构造函数什么都不干。

- 通常情况下，不定义类的构造函数，编译器就会为我们生成一个平凡的默认构造函数。

```
1 // 使用默认的构造函数
2 class Test {};
```

- 一旦定义了构造函数，即使构造函数不包含参数，函数体里也没有任何的代码，那么该构造函数也不再是“平凡”的。

```
1 class Test1
2 {
3     Test1(); // 程序员定义的构造函数，非默认构造
4 };
```


关于析构函数也和上面列举的构造函数类似，一旦被定义就不平凡了。但是这也并非无药可救，使用**=default**关键字可以显式地声明默认的构造函数，从而使得类型恢复“平凡化”。

2. 拥有平凡的拷贝构造函数 (trivial copy constructor) 和移动构造函数 (trivial move constructor)

- 平凡的拷贝构造函数基本上等同于使用memcpy 进行类型的构造。

```
1  #include <iostream>
2  #include <cstring> // for memcpy
3
4  struct MyPOD {
5      int a;
6      double b;
7  };
8
9  int main() {
10     MyPOD src = {42, 3.14};
11     MyPOD dest;
12
13     // 用 memcpy 进行构造
14     std::memcpy(&dest, &src, sizeof(MyPOD));
15
16     std::cout << "dest.a = " << dest.a << ", dest.b = " << dest.b <<
std::endl;
17     return 0;
18 }
```

Fence 203

- 同平凡的默认构造函数一样，不声明拷贝构造函数的话，编译器会帮程序员自动地生成。
- 可以显式地使用**=default** 声明默认拷贝构造函数。
- 而平凡移动构造函数跟平凡的拷贝构造函数类似，只不过是用于移动语义。

3. 拥有平凡的拷贝赋值运算符 (trivial assignment operator) 和移动赋值运算符 (trivial move operator)

这基本上与平凡的拷贝构造函数和平凡的移动构造运算符类似

4. 不包含虚函数以及虚基类。

- 类中使用**virtual** 关键字修饰的函数 叫做虚函数

```
1  class Base
2  {
3  public:
4      Base() {}
5      virtual void print() {}
6  };
```

Fence 204

- 虚基类是在创建子类的时候在继承的基类前加**virtual** 关键字 修饰

```
1  语法: class 派生类名: virtual 继承方式 基类名
```

```
1  class Base
2  {
3  public:
4      Base() {}
5  };
6  // 子类Child, 虚基类: Base
7  class Child : virtual public Base
8  {
9      Child() {}
10 };
```

18.2 “标准布局”类型

标准布局类型主要指的是类或者结构体的结构或者组合方式。

标准布局类型的类应该符合以下五点定义，最重要的为前两条：

1. 所有非静态成员有相同的访问权限（public, private, protected）。
 - 类成员拥有不同的访问权限（非标准布局类型）

```
1  class Base
2  {
3  public:
4      Base() {}
5      int a;
6  protected:
7      int b;
8  private:
9      int c;
10 };
```

- 类成员拥有相同的访问权限（标准布局类型）

```
1  class Base
2  {
3  public:
4      Base() {}
5      int a;
6      int b;
7      int c;
8  };
```

2. 在类或者结构体继承时，满足以下两种情况之一：

- 派生类中有非静态成员，基类中包含静态成员（或基类没有变量）。

- 基类有非静态成员，而派生类没有非静态成员。

```
1 struct Base { static int a;};
2 struct Child: public Base{ int b;};           // ok
3 struct Base1 { int a;};
4 struct Child1: public Base1{ static int c;}; // ok
5 struct Child2:public Base, public Base1 { static int d;}; // ok
6 struct Child3:public Base1{ int d;};          // error C++14中依然属于POD类型，
        但不是标准类型
7 struct Child4:public Base1, public Child      // error
8 {
9     static int num;
10 };
```

Fence 209

通过上述例子得到的结论：

- 非静态成员只要同时出现在派生类和基类间，即不属于标准布局。
- 对于多重继承，一旦非静态成员出现在多个基类中，即使派生类中没有非静态成员变量，派生类也不属于标准布局。

3. 子类中第一个非静态成员的类型与其基类不同。

此处基于G++编译器讲解，如果使用VS的编译器和G++编译器得到的结果是不一样的。

```
1 struct Parent{};
2 struct Child : public Parent
3 {
4     Parent p;    // 子类的第一个非静态成员
5     int foo;
6 };
```

Fence 210

上面的例子中Child不是一个标准布局类型，因为它的第一个非静态成员变量p和父类的类型相同，改成下面这样子类就变成了一个标准布局类型：

```
1 struct Parent{};
2 struct Child1 : public Parent
3 {
4     int foo;    // 子类的第一个非静态成员
5     Parent p;
6 };
```

Fence 211

这条规则对于我们来说是比较特别的，这样规定的目的主要是节约内存，提高数据的读取效率。对于上面的两个子类Child和Child1来说它们的内存结构是不一样的，在基类没有成员的情况下：

- C++标准允许**标准布局类型**（Child1）派生类的第一个成员foo与基类共享地址，此时基类并没有占据任何的实际空间（可以节省一点数据）
- 对于子类Child而言，如果子类的第一个成员仍然是基类类型，C++标准**要求类型相同的对象它们的地址必须不同**（基类地址不能和子类中的变量 p 类型相同），此时需要分配额外的地址空间将二者的地址错开。

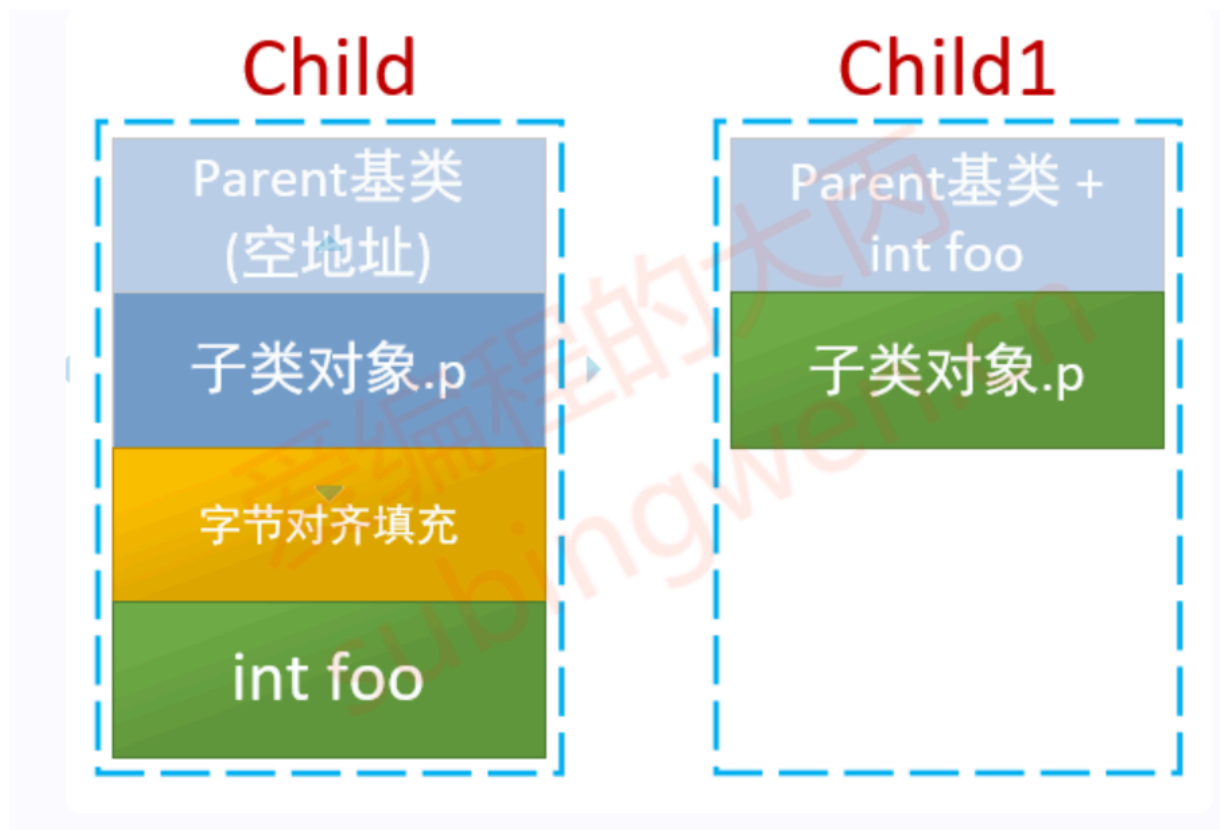


Figure 3

4. 没有虚函数和虚基类。

5. 所有非静态数据成员均符合标准布局类型，其基类也符合标准布局，这是一个递归的定义。

18.3 对POD类型的判断

想要判断某个数据类型是不是属于 POD 类型，可以使用C++11给我们提供的相关函数

18.3.1 对“平凡类型”的判断

C++11提供的类模板叫做 `is_trivial`，其定义如下：

```
1 | template <class T> struct std::is_trivial;
```

Fence 212

`std::is_trivial` 的成员 `value` 可以用于判断T的类型是否是一个平凡的类型（`value` 函数返回值为布尔类型）。除了类和结构体外，`is_trivial`还可以对内置的标准类型数据（比如int、float都属于平凡类型）及数组类型（元素是平凡类型的数组总是平凡的）进行判断。

关于类型的判断，示例程序如下：

```
1 | #include <iostream>
2 | #include <type_traits>
3 | using namespace std;
4 |
5 | class A {};
6 | class B { B() {} };
7 | class C : B {};
```

```

8   class D { virtual void fn() {} };
9   class E : virtual public A { };
10
11  int main()
12  {
13      cout << std::boolalpha;
14      cout << "is_trivial:" << std::endl;
15      cout << "int: " << is_trivial<int>::value << endl;
16      cout << "A: " << is_trivial<A>::value << endl;
17      cout << "B: " << is_trivial<B>::value << endl;
18      cout << "C: " << is_trivial<C>::value << endl;
19      cout << "D: " << is_trivial<D>::value << endl;
20      cout << "E: " << is_trivial<E>::value << endl;
21      return 0;
22  }

```

Fence 213

输出的结果：

```

1   is_trivial:
2   int: true
3   A: true
4   B: false
5   C: false
6   D: false
7   E: false

```

Fence 214

- int：内置标准数据类型，属于 trivial 类型
- A：拥有默认的构造和析构函数，属于 trivial 类型
- B：自定义了构造函数，因此不属于 trivial 类型
- C：基类中自定义了构造函数，因此不属于 trivial 类型
- D：类成员函数中有虚函数，因此不属于 trivial 类型
- E：继承关系中有虚基类，因此不属于 trivial 类型

18.3.2 对“标准布局”类型的判断

同样，在C++11中，我们可以使用模板类来帮助判断类型是否是一个标准布局的类型，其定义如下：

```

1   template <typename T> struct std::is_standard_layout;

```

Fence 215

通过 `is_standard_layout` 模板类的成员 `value` (`is_standard_layout::value`)，我们可以在代码中打印出类型的标准布局属性，函数返回值为布尔类型。

关于类型的判断，示例程序如下：

```

1   // pod.cpp
2   #include <iostream>
3   #include <type_traits>
4   using namespace std;

```

```

5
6 struct A { };
7 struct B : A { int j; };
8 struct C
9 {
10 public:
11     int a;
12 private:
13     int c;
14 };
15 struct D1 { static int i; };
16 struct D2 { int i; };
17 struct E1 { static int i; };
18 struct E2 { int i; };
19 struct D : public D1, public E1 { int a; };
20 struct E : public D1, public E2 { int a; };
21 struct F : public D2, public E2 { static int a; };
22 struct G : public A
23 {
24     int foo;
25     A a;
26 };
27 struct H : public A
28 {
29     A a;
30     int foo;
31 };
32
33 int main()
34 {
35     cout << std::boolalpha;
36     cout << "is_standard_layout:" << std::endl;
37     cout << "A: " << is_standard_layout<A>::value << endl;
38     cout << "B: " << is_standard_layout<B>::value << endl;
39     cout << "C: " << is_standard_layout<C>::value << endl;
40     cout << "D: " << is_standard_layout<D>::value << endl;
41     cout << "D1: " << is_standard_layout<D1>::value << endl;
42     cout << "E: " << is_standard_layout<E>::value << endl;
43     cout << "F: " << is_standard_layout<F>::value << endl;
44     cout << "G: " << is_standard_layout<G>::value << endl;
45     cout << "H: " << is_standard_layout<H>::value << endl;
46     return 0;
47 }

```

Fence 216

G++ 编译输出的结果:

```

1 # 编译命令
2 g++ pod.cpp -std=c++11
3
4 # 输出结果
5 is_standard_layout:
6 A: true

```

```
7 B: true
8 C: false
9 D: true
10 D1: true
11 E: false
12 F: false
13 G: true
14 H: false
```

Fence 217

vs2022s输出的结果:

```
1 is_standard_layout:
2 A: true
3 B: true
4 C: false
5 D: true
6 D1: true
7 E: false
8 F: false
9 G: false
10 H: false
```

Fence 218

- A：没有虚基类和虚函数，属于 standard_layout 类型
- B：没有虚基类和虚函数，属于 standard_layout 类型
- C：所有非静态成员访问权限不一致，不属于 standard_layout 类型
- D：基类和子类没有同时出现非静态成员变量，属于 standard_layout 类型
- D1：没有虚基类和虚函数，属于 standard_layout 类型
- E：基类和子类中同时出现了非静态成员变量，不属于 standard_layout 类型
- F：多重继承中在基类里同时出现了非静态成员变量，不属于 standard_layout 类型
- G：使用的编译器不同，得到的结果也不同。
- H：子类中第一个非静态成员的类型与其基类类型不能相同，不属于 standard_layout 类型

POD优点:

- 字节赋值，代码中我们可以安全地使用memset 和 memcpy 对 POD类型进行初始化和拷贝等操作。
- 提供对C内存布局兼容。C++程序可以与C 函数进行相互操作，因为POD类型的数据在C与C++ 间的操作总是安全的。
- 保证了静态初始化的安全有效。静态初始化在很多时候能够提高程序的性能，而POD类型的对象初始化往往更加简单。

19. 默认函数控制 =default 与 =delete

19.1 类与默认函数

在C++中声明自定义的类，编译器会默认帮助程序员生成一些他们未自定义的成员函数。这样的函数版本被称为“默认函数”。这样的函数一共有六个，我们一起来看一下：

- 无参构造函数：创建类对象
- 拷贝构造函数：拷贝类对象
- 移动构造函数：拷贝类对象
- 拷贝赋值函数：类对象赋值
- 移动赋值函数：类对象赋值
- 析构函数：销毁类对象

在C++语法规则中，一旦程序员实现了这些函数的自定义版本，则编译器不会为该自动类生成默认版本。

有时程序员会忘记上面提到的规则，最常见的是声明了带参数的构造，如果还需要无参构造函数，这时候必须定义出不带参数的版本。不过通过编译器的提示，这样的问题通常会得到更正。但更为严重的问题是，一旦声明了自定义版本的构造函数，则**有可能导致我们定义的类型不再是POD类型**，我们便不再能够享受POD类型为我们带来的便利。

对于上面提到的这些，我们无需过度担心，因为C++11非常贴心地为我们提供了解决方案，就是使用=default。

在C++11标准中称=default修饰的函数为**显式默认【缺省】（explicit defaulted）函数**，而称=delete修饰的函数为**删除（deleted）函数或者显示删除函数**。

C++11引入显式默认和显式删除是为了增强对类默认函数的控制，让程序员能够更加精细地控制默认版本的函数。

19.2 =default

我们可以在类内部修饰满足条件的类函数为显示默认函数，也可以在类定义之外修饰成员函数为默认函数。下面举例说明：

19.2.1 在类内部指定函数为默认函数

一般情况下，我们可以在定义类的时候直接在类内部指定默认函数，如下所示：

```
1  class Base
2  {
3  public:
4      Base() = default;
5      Base(const Base& obj) = default;
6      Base(Base&& obj) = default;
7      Base& operator= (const Base& obj) = default;
8      Base& operator= (Base&& obj) = default;
9      ~Base() = default;
10 };
```

Fence 219

- 第4行：指定无参构造为默认函数
- 第5行：指定拷贝构造函数为默认函数
- 第6行：指定移动构造函数为默认函数

- 第7行：指定复制赋值操作符重载函数为默认函数
- 第8行：指定移动赋值操作符重载函数为默认函数
- 第9行：指定析构函数为默认函数

使用 `=default` 指定的默认函数和类提供的默认函数是等价的

19.2.2 在类外部指定函数为默认函数

默认函数除了在类定义的内部指定，也可以在类的外部指定，如下所示：

```

1  // 类定义
2  class Base
3  {
4  public:
5      Base();
6      Base(const Base& obj);
7      Base(Base&& obj);
8      Base& operator= (const Base& obj);
9      Base& operator= (Base&& obj);
10     ~Base();
11 };
12 // 在类定义之外指定成员函数为默认函数
13 Base::Base() = default;
14 Base::Base(const Base& obj) = default;
15 Base::Base(Base&& obj) = default;
16 Base& Base::operator= (const Base& obj) = default;
17 Base& Base::operator= (Base&& obj) = default;
18 Base::~~Base() = default;

```

Fence 220

定义默认函数的注意事项：

如果程序猿对C++类提供的默认函数（上面提到的六个函数）进行了实现，那么可以通过 `=default` 将他们再次指定为默认函数，不能使用 `=default` 修饰这六个函数以外的函数。

```

1  class Base
2  {
3  public:
4      Base() = default;
5      Base(const Base& obj) = default;
6      Base(Base&& obj) = default;
7      Base& operator= (const Base& obj) = default;
8      Base& operator= (Base&& obj) = default;
9      ~Base() = default;
10
11     // 以下写法全部都是错误的
12     Base(int a = 0) = default;
13     Base(int a, int b) = default;
14     void print() = default;
15     bool operator== (const Base& obj) = default;
16     bool operator>=(const Base& obj) = default;
17 };

```

- 第12行：自定义带参构造，不允许使用 =default 修饰（即使有默认参数也不行）
- 第13行：自定义带参构造，不允许使用 =default 修饰
- 第14行：自定义函数，不允许使用 =default 修饰
- 第15、16行：不是移动、复制赋值运算符重载，不允许使用 =default 修饰

19.3 =delete

=delete 表示显示删除，显式删除可以避免用户使用一些不应该使用的类的成员函数，使用这种方式可以有效的防止某些类型之间自动进行隐式类型转换产生的错误。下面举例说明：

19.3.1 禁止使用默认生成的函数

```

1  class Base
2  {
3  public:
4      Base() = default;
5      Base(const Base& obj) = delete;
6      Base& operator= (const Base& obj) = delete;
7  };
8
9  int main()
10 {
11     Base b;
12     Base tmp1(b);    // error
13     Base tmp = b;    // error
14     return 0;
15 }
```

Fence 222

- 第5行：禁用拷贝构造函数
- 第6行：禁用 = 进行对象复制
- 第12行：拷贝构造函数已被显示删除，无法拷贝对象
- 第13行：复制对象的赋值操作符重载函数已被显示删除，无法复制对象

19.3.2 禁止使用自定义函数

```

1  class Base
2  {
3  public:
4      Base(int num) : m_num(num) {}
5      Base(char c) = delete;
6      void print(char c) = delete;
7      void print()
8      {
9          cout << "num: " << m_num << endl;
10     }
11     void print(int num)
12     {
13         cout << "num: " << num << endl;
14     }
15 }
```

```

15     private:
16         int m_num;
17     };
18
19     int main()
20     {
21         Base b(97);          // 'a' 对应的 ascii 值为97
22         Base b1('a');        // error
23         b.print();
24         b.print(97);
25         b.print('a');        // error
26         return 0;
27     }

```

Fence 223

- 第5行：禁用带 char类型参数的构造函数，防止隐式类型转换（char转int）
- 第6行：禁止使用带char类型的自定义函数，防止隐式类型转换（char转int）
- 第22行：对应的构造函数被禁用，因此无法使用该构造函数构造对象
- 第25行：对应的打印函数被禁用，因此无法给函数传递char类型参数

20. 静态断言 static_assert

20.1 断言

断言（assertion）是一种编程中常用的手段。在通常情况下，**断言就是将一个返回值总是需要为真的判断表达式放在语句中，用于排除在设计的逻辑上不应该产生的情况。**

比如：一个函数总需要输入在一定的范围内的参数，那么程序员就可以对该参数使用断言，以迫使在该参数发生异常的时候程序退出，从而避免程序陷入逻辑的混乱。

从一些意义上讲，断言并不是正常程序所必需的，不过对于程序调试来说，通常断言能够帮助程序开发者快速定位那些违反了某些前提条件的程序错误。

如果我们要在C++程序中使用断言，需要在程序中包含头文件**或**<assert.h>，头文件中为我们提供了 **assert 宏**，用于在运行时进行断言。举例说明：

```

1     #include <iostream>
2     #include <cassert>
3     using namespace std;
4
5     // 创建一个指定大小的 char 类型数组
6     char* createArray(int size)
7     {
8         // 通过断言判断数组大小是否大于0
9         assert(size > 0);    // 必须大于0，否则程序中断
10        char* array = new char[size];
11        return array;
12    }
13
14    int main()
15    {
16        char* buf = createArray(0);

```

```

17 // 此处使用的是vs提供的安全函数，也可以使用 strcpy
18 strcpy_s(buf, 16, "hello, world!");
19 cout << "buf = " << buf << endl;
20 delete[] buf;
21 return 0;
22 }
23

```

Fence 224

在程序的第9行，使用了断言`assert(expression)`，这是一个宏，它的参数是一个表达式，这个表达式通常返回一个布尔类型的值，并且要求表达式必须为 `true` 程序才能继续向下执行，否则会直接中断。

- 如果 `createArray`参数大于0，程序在16行正常运行直到结束
- 如果 `createArray`参数小于等于0，程序运行到16行直接退出，会看到如下图的提示信息：

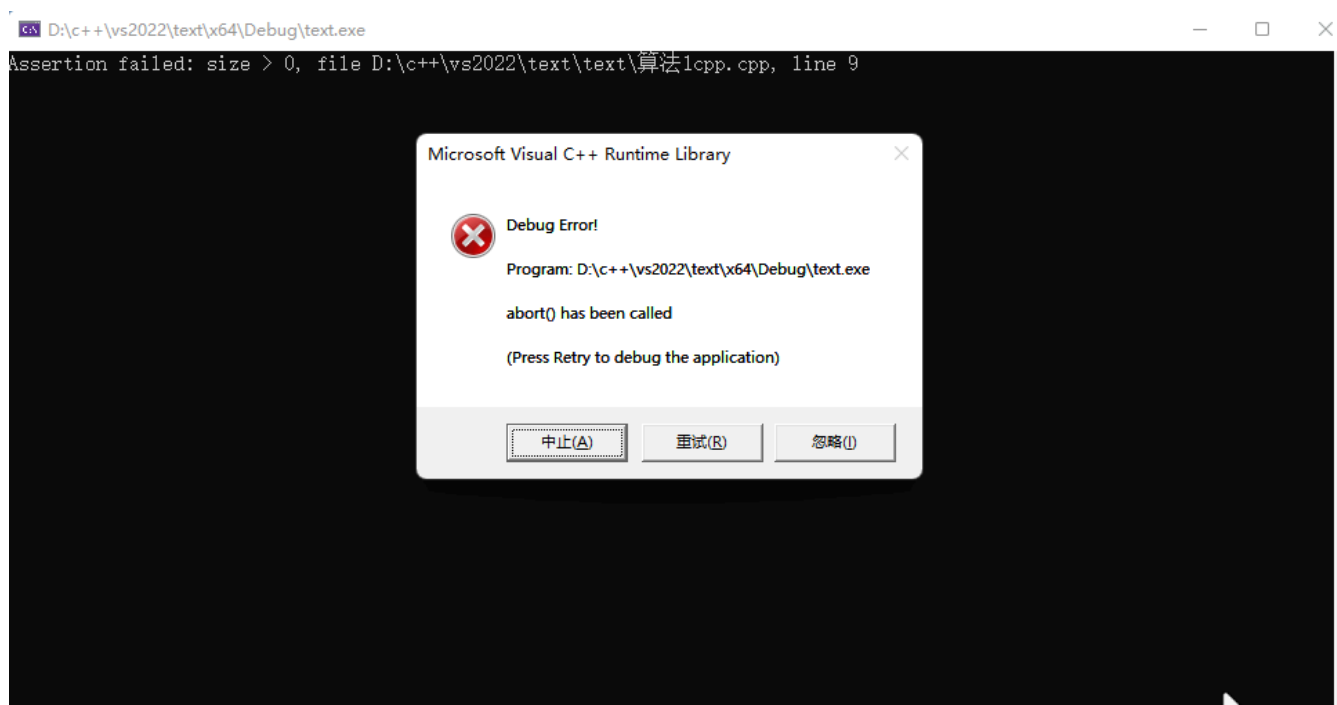


Figure 4

20.2 静态断言

在上面的例子中我们使用了断言 `assert`。但 `assert`是一个运行时断言，也就是说它只有在程序运行时才能起作用。这意味着不运行程序我们将无法得知某些条件是否是成立的。比如：我们想知道当前是32位还是64位平台，对于这个需求我们应该是在程序运行之前就应该得到结果，如果使用断言显然是无法做到的，对于这种情况我们就需要使用C++11提供的静态断言了。

静态断言`static_assert`，所谓静态就是在编译时就能够进行检查的断言，使用时不需要引用头文件。静态断言的另一个好处是，可以自定义违反断言时的错误提示信息。静态断言使用起来非常简单，它接收两个参数：

- **参数1：断言表达式，这个表达式通常需要返回一个 `bool`值**
- **参数2：警告信息，它通常就是一段字符串，在违反断言（表达式为`false`）时提示该信息**

由于基于VS计算的字节大小和理论值有出入，下面程序基于64位Linux进行测试，使用静态断言验证当前操作系统是否是32位：

```

1 // assert.cpp
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     // 字体原因看起来是一个=, 其实这是两个=
8     static_assert(sizeof(long) == 4, "错误, 不是32位平台...");
9     cout << "64bit Linux 指针大小: " << sizeof(char*) << endl;
10    cout << "64bit Linux long 大小: " << sizeof(long) << endl;
11
12    return 0;
13 }
14

```

Fence 225

通过g++编译程序:

```

1 $ g++ assert.cpp -std=c++11
2 assert.cpp: In function 'int main()':
3 assert.cpp:6:5: error: static assertion failed: 错误, 不是32位平台...
4 static_assert(sizeof(long) == 4, "错误, 不是32位平台...");

```

Fence 226

由于使用的Linux是64位的, 因此在编译阶段静态断言检测条件失败, 提示的错误信息就是我们给静态断言指定的第二个参数对应的那个字符串。

如果我们将静态断言的条件判断修改一下:

```

1 static_assert(sizeof(long) == 8, "错误, 不是64位平台...");

```

Fence 227

然后在进行编译:

```

1 $ g++ assert.cpp -std=c++11

```

Fence 228

编译通过, 得到可执行程序a.out, 然后执行这个程序可以看到如下输出:

```

1 $ ./a.out
2 64bit Linux 指针大小: 8
3 64bit Linux long 大小: 8

```

Fence 229

静态断言条件判断为true, 程序就可以继续向下执行了。

注意事项:

由于静态断言的表达式是在编译阶段进行检测, 所以在它的表达式中不能出现变量, 也就是说这个表达式必须是常量表达式。

20.3 附录

下面是32位系统与64位系统各数据类型对比：

数据类型	说明	32位字节数	64位字节数	取值范围/精度说明
bool	布尔型	1	1	true, false
char	字符型	1	1	-128 ~ 127
unsigned char	无符号字符型	1	1	0 ~ 255
short	短整型	2	2	-32768 ~ 32767
unsigned short	无符号短整型	2	2	0 ~ 65535
int	整型	4	4	-2147483648 ~ 2147483647
unsigned int	无符号整型	4	4	0 ~ 4294967295
long	长整型	4	8	—
unsigned long	无符号长整型	4	8	—
unsigned long long	无符号超长整型	≥8	≥8	8字节时：0 ~ 2^64
float	单精度浮点数	4	4	范围约-2 ¹²⁸ ~2 ¹²⁸ ，精度6~7位有效数字
double	双精度浮点数	8	8	范围约-2 ¹⁰²⁴ ~2 ¹⁰²⁴ ，精度15~16位
long double	扩展精度浮点数	8	8	范围约-2 ¹⁰²⁴ ~2 ¹⁰²⁴ ，精度15~16位
*	地址（指针）	4	8	—

Table 5

21. noexcept

21.1 异常

异常通常用于处理逻辑上可能发生的错误，在C++98中为我们提供了一套完善的异常处理机制，我们可以直接在程序中将各种类型的异常抛出，从而强制终止程序的运行。

21.1.1 基本语法

关于异常的基本语法如下：

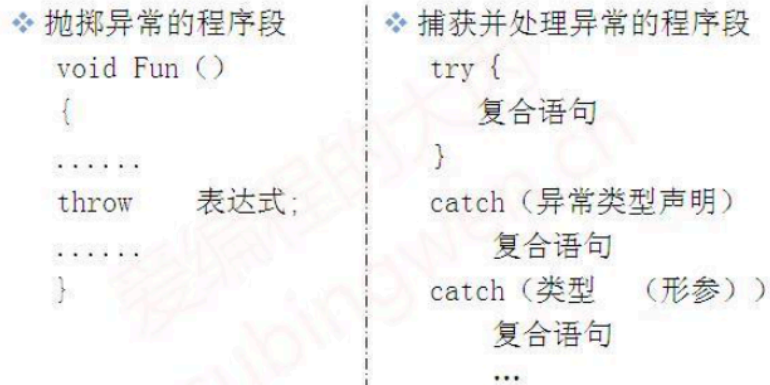


Figure 5

```

1  int main()
2  {
3      try
4      {
5          throw -1;
6      }
7      catch (int e)
8      {
9          cout << "int exception, value: " << e << endl;
10     }
11     cout << "That's ok!" << endl;
12     return 0;
13 }
```

Fence 230

异常被抛出后，从进入try块起，到异常被抛掷前，这期间在栈上构造的所有对象，都会被自动析构。析构的顺序与构造的顺序相反。这一过程称为栈的解旋。

21.1.2 异常接口声明

为了加强程序的可读性，可以在函数声明中列出可能抛出的所有异常类型，常用的有如下三种书写方式：

1. 显示指定可以抛出的异常类型

```

1  struct MyException
2  {
3      MyException(string s) :msg(s) {}
4      string msg;
5  };
6
7  double divisionMethod(int a, int b) throw(MyException, int)    // 异常类型
8  {
9      if (b == 0)
10     {
11         throw MyException("division by zero!!!");
12         // throw 100;
13     }
14     return a / b;
```

```

15     }
16
17     int main()
18     {
19         try
20         {
21             double v = divisionMethod(100, 0);
22             cout << "value: " << v << endl;
23         }
24         catch (int e)                                // b != 0
25         {
26             cout << "catch except: " << e << endl;
27         }
28         catch (MyException e)                        // b == 0
29         {
30             cout << "catch except: " << e.msg << endl;
31         }
32         return 0;
33     }

```

Fence 231

第7行代码在divisionMethod函数后添加了throw异常接口声明，其参数表示可以抛出的异常类型，分别为int 和 MyException 类型。

2. 抛出任意异常类型

```

1     struct MyException
2     {
3         MyException(string s) :msg(s) {}
4         string msg;
5     };
6
7     double divisionMethod(int a, int b)                // 未声明异常类型
8     {
9         if (b == 0)
10        {
11            throw MyException("division by zero!!!");
12            // throw 100;
13        }
14        return a / b;
15    }

```

Fence 232

第7行代码在divisionMethod 没有添加异常接口声明，表示在该函数中可以抛出任意类型的异常。

3. 不抛出任何异常

```

1     struct MyException
2     {
3         MyException(string s) :msg(s) {}
4         string msg;
5     };

```



```

6
7 double divisionMethod(int a, int b) throw() // 不抛出异常
8 {
9     if (b == 0)
10    {
11        cout << "division by zero!!!" << endl;
12    }
13    return a / b;
14 }

```

Fence 233

第7行代码在divisionMethod 函数后添加了throw异常接口声明，其参数列表为空，表示该函数不允许抛出异常。

温馨提示：以上程序在VS上的测试结果和在Linux上基于G++的测试结果是不同的，如果违反了规则VS只会给出警告，而G++则会直接终止程序的运行。（PS：VS使用的不是G++编译器）

21.2 noexcept

上面的例子中，在 divisionMethod 函数声明之后，我们定义了一个动态异常声明 throw(MyException, int)，该声明指出了divisionMethod可能抛出的异常的类型。事实上，该特性很少被使用，因此在C++11中被弃用了，而表示函数不会抛出异常的动态异常声明 throw() 也被新的 noexcept 异常声明所取代。

noexcept 形如其名，表示其修饰的函数不会抛出异常。不过与 throw() 动态异常声明不同的是，在 C++11 中如果 noexcept 修饰的函数抛出了异常，编译器会直接调用 std::terminate() 函数来终止程序的运行，这比基于异常机制的 throw() 在效率上会高一些。这是因为异常机制会带来一些额外开销，比如函数抛出异常，会导致函数栈被依次地展开（栈解旋），并自动调用析构函数释放栈上的所有对象。

noexcept 在C++中有两种用法：修饰符 和 操作符。它们的作用和语法不同。

21.2.1 noexcept修饰符（常用于移动构造、析构等这些不应该抛出异常的情况下）

因此对于**不会抛出异常的函数**我们可以这样写：

```

1 double divisionMethod(int a, int b) noexcept
2 {
3     if (b == 0)
4     {
5         cout << "division by zero!!!" << endl;
6         return -1;
7     }
8     return a / b;
9 }

```

Fence 234

从语法上讲，noexcept 修饰符有两种形式：

1. 简单地在函数声明后加上 noexcept 关键字
2. 可以接受一个常量表达式作为参数，如下所示：

```

1 double divisionMethod(int a, int b) noexcept(常量表达式);

```

Fence 235

常量表达式的结果会被转换成一个bool类型的值：

- 值为 true，表示函数不会抛出异常
- 值为 false，表示有可能抛出异常这里
- 不带常量表达式的noexcept相当于声明了noexcept（true），即不会抛出异常。

21.2.2 noexcept 操作符（模板元编程、条件性地声明函数为 noexcept。）

用于编译期判断一个表达式是否会抛出异常，结果为 true 或 false。

```
1 | bool b = noexcept(expr);
```

Fence 236

如果 expr 在编译期被判定为不会抛异常，结果为 true，否则为 false。

```
1 | #include <iostream>
2 | using namespace std;
3 |
4 | // noexcept修饰符：声明foo不会抛异常
5 | void foo() noexcept
6 | {
7 |     cout << "foo() called" << endl;
8 | }
9 |
10 | // 没有noexcept修饰，可能抛异常
11 | void bar()
12 | {
13 |     cout << "bar() called" << endl;
14 | }
15 |
16 | int main()
17 | {
18 |     // noexcept操作符：编译期判断表达式是否noexcept
19 |     static_assert(noexcept(foo()), "foo不会抛异常");
20 |     static_assert(!noexcept(bar()), "bar可能抛异常");
21 |
22 |     foo();
23 |     bar();
24 |     return 0;
25 | }
26 |
```

Fence 237

22. 类成员的快速初始化

22.1 C++98 标准的类成员初始化

在C++98中，支持了在类声明中使用等号 = 加初始值 的方式，来初始化类中静态成员常量。这种声明方式我们也称之为“就地”声明。而非静态成员变量的初始化则必须在构造函数中进行。

```

1 struct Base
2 {
3     Base() : a(250) {}
4     Base(int num) : a(num) {}
5
6     int a;
7     int b = 1;
8     static int c = 0;
9     static const double d = 3.14;
10    static const char* const e = "i am luffy";
11    const static int f = 0;
12 };

```

Fence 238

如果按照 C++98 标准来解读上面这段代码，其中有这么几行语法是错误的：

- 第7行：类的非静态成员，必须在构造函数中进行初始化
- 第8行：类的静态成员，必须在类的外部进行初始化
- 第9行：类的静态常量成员，但不是整形或者枚举，无法通过编译

如果使用 g++ 可能发现就地初始化 double 类型的静态常量是可以通过编译的，不过这实际是 GNU 对C++的一个扩展，并不遵从C++ 标准。

- 第10行：类的静态常量成员，但不是整形或者枚举，无法通过编译
- 第8、9、10行的变量初始化方式是一样的，都是在类的外部

```

1 int Base::c = 110;
2 const double Base::d = 3.14;
3 const char* const Base::e = "i am luffy";

```

Fence 239

上面程序中的 static const 和 const static 是等价的。

22.2 C++11 标准的类成员初始化

22.2.1 初始化类的非静态成员

在进行类成员变量初始化的时候，C++11标准对于C++98做了补充，允许在定义类的时候在类内部直接对非静态成员变量进行初始化，在初始化的时候可以使用等号 = 也可以使用花括号 {}。

```

1 class Test
2 {
3 private:
4     int a = 9;
5     int b = {5};
6     int c{12};
7     double array[4] = { 3.14, 3.15, 3.16, 3.17};
8     double array1[4] { 3.14, 3.15, 3.16, 3.17 };
9     string s1("hello"); // error
10    string s2{ "hello, world" };
11 };

```

Fence 240

可以看到如果使用花括号 {} 的方式对类的非静态成员进行初始化，等号是可以省略不写的。

- 第9行：错误，不能使用小括号() 初始化对象，应该使用花括号{}

22.2.2 类内部赋值和初始化列表

在C++11之前对于非静态的类成员变量我们除了在构造函数内部进行赋值，也可以在类的初始化列表中进行初始化（这种方式比在构造函数内部赋值效率高）。那么，如果同时在类内部对非静态成员变量就地初始化和在初始化列表中进行初始化会怎么样呢？下面来测试一下：

```
1  class Init
2  {
3  public:
4      Init(int x, int y, int z) :a(x), b(y), c(z) {}
5
6      int a = 1;
7      int b = 2;
8      int c = 3;
9  };
10
11 int main()
12 {
13     Init tmp(10, 20, 30);
14     cout << "a: " << tmp.a << ", b: " << tmp.b << ", c: " << tmp.c << endl;
15     return 0;
16 }
```

Fence 241

- 第4行：使用初始化列表对类的非静态成员进行初始化
- 第6、7、8行：在类内部对非静态成员变量就地初始化（C++11新特性）

执行程序，输出的结果如下：

```
1  a: 10, b: 20, c: 30
```

Fence 242

我们可以从函数的打印输出中看到，在类内部就地初始化和初始化列表并不冲突（程序可以正常运行）。程序员可以为同一成员变量既在类内部就地初始化，又在初始化列表中进行初始化，只不过初始化列表总是看起来后作用于非静态成员。也就是说，**通过初始化列表指定的值会覆盖就地初始化时指定的值。**

23. long long 整形

相比于C++98标准，C++11整型的最大改变就是多了long long。但事实上，long long 整型本来就离C++标准很近，早在1995年，long long 就被提议写入C++98标准，却被C++标准委员会拒绝了。而后来，long long类型却进入了C99标准，而且也事实上也被很多编译器支持。于是辗转地，C++标准委员会又掉头决定将long long纳入C++11标准。

23.1 long long 类型

C++11 标准要求 long long 整型可以在不同平台上有不同的长度，但**至少有64位**。long long 整型有两种：

- long long - 对应类型的数值可以使用 LL (大写) 或者 ll (小写) 后缀

```

1 | long long num1 = 123456789LL;
2 | long long num2 = 123456789ll;

```

Fence 243

- **unsigned long long** - 对应类型的数值可以使用 ULL (大写) 或者 ull (小写) 或者 Ull、uLL (等大小写混合)后缀

```

1 | unsigned long long num1 = 123456789ULL;
2 | unsigned long long num2 = 123456789ull;
3 | unsigned long long num3 = 123456789uLL;
4 | unsigned long long num4 = 123456789Ull;

```

Fence 244

事实上在C++11中还有一些类型与以上两种类型是等价的:

- **对于有符号类型的 long long 和以下三种类型等价**
 - long long int
 - signed long long
 - signed long long int
- **对于无符号类型的 unsigned long long 和 unsigned long long int 是等价的**

同其他的整型一样，要了解平台上 long long 最值的方法就是查看（或 `<limits.h>`）中的宏与 long long 整型相关的一共有3个:

- **LLONG_MIN** - 最小的 long long 值
- **LLONG_MAX** - 最大的 long long 值
- **ULLONG_MAX** - 最大的 unsigned long long 值

测试代码如下:

```

1 | #include <iostream>
2 | using namespace std;
3 |
4 | int main()
5 | {
6 |     long long max = LLONG_MAX;
7 |     long long min = LLONG_MIN;
8 |     unsigned long long ullMax = ULLONG_MAX;
9 |
10 |    cout << "Max Long Long value: " << max << endl
11 |         << "Min Long Long value: " << min << endl
12 |         << "Max unsigned Long Long value: " << ullMax << endl;
13 |    return 0;
14 | }

```

Fence 245

程序输出的结果:

```

1 | Max Long Long value: 9223372036854775807
2 | Min Long Long value: -9223372036854775808
3 | Max unsigned Long Long value: 18446744073709551615

```

可以看到 long long 类型能够存储的最大/最小值还是非常大/小的，但是这个值根据平台不同会有所变化，原因是因为C++11标准规定该类型至少占8字节，它占的字节数越多，对应能够存储的数值也就越大。

23.2 扩展的整形

在C++11中一共只定义了以下5种标准的有符号整型：

- signed char
- short int
- int
- long int
- long long int

标准同时规定，每一种有符号整型都有一种对应的无符号整数版本，且有符号整型与其对应的无符号整型具有相同的存储空间大小。比如与 signed int对应的无符号版本的整型是 unsigned int。

当我们在C++中处理数据的时候，如果参与运算的数据或者传递的参数类型不匹配，整型间会发生隐式的转换，这种过程通常被称为整型的提升。比如如下表达式：

```
1 | (int)num1 + (long long)num2
```

Fence 247

关于这种整形提升的隐式转换遵循如下原则：

- 长度越大的整型等级越高，比如 long long int 的等级会高于int。
- 长度相同的情况下，标准整型的等级高于扩展类型，比如 long long int 和 int64 如果都是64 位长度，则long long int类型的等级更高。
- 相同大小的有符号类型和无符号类型的等级相同，long long int 和unsigned longlong int的等级就相同。
- 转换过程中，低等级整型需要转换为高等级整型，有符号的需要转换为无符号整形。

24. 扩展的 friend 语法

friend关键字在C++中是一个比较特别的存在。因为在大多数编程语言中是没有提供friend关键字的，比如Java。friend关键字用于声明类的友元，友元可以无视类中成员的属性（public、protected 或是 private），友元类或友元函数都可以访问，这就**完全破坏了面向对象编程中封装性的概念**。但有的时候，friend关键字确实会让程序猿少写很多代码，因此 friend 还是在很多程序中被使用到。

24.1 语法改进

在 C++11 标准中对 friend关键字进行了一些改进，以保证其更加好用：

声明一个类为另外一个类的友元时，不再需要使用class关键字，并且还可以使用类的别名（使用 typedef 或者 using 定义）。

我们可以看看下面的例子：

```
1 | #include <iostream>
2 | using namespace std;
3 |
4 | // 类声明
```

```
5  class Tom;
6  // 定义别名
7  using Honey = Tom;
8
9  // 定义两个测试类
10 class Jack
11 {
12     // 声明友元
13     // friend class Tom;    // C++98 标准语法
14     friend Tom;            // C++11 标准语法
15     string name = "jack";  // 默认私有
16     void print()           // 默认私有
17     {
18         cout << "my name is " << name << endl;
19     }
20 };
21
22 class Lucy
23 {
24 protected:
25     // 声明友元
26     // friend class Tom;    // C++98 标准语法
27     friend Honey;          // C++11 标准语法
28     string name = "lucy";
29     void print()
30     {
31         cout << "my name is " << name << endl;
32     }
33 };
34
35 class Tom
36 {
37 public:
38     void print()
39     {
40         // 通过类成员对象访问其私有成员
41         cout << "invoke Jack private member: " << jObj.name << endl;
42         cout << "invoke Jack private function: " << endl;
43         jObj.print();
44
45         cout << "invoke Lucy private member: " << lObj.name << endl;
46         cout << "invoke Lucy private function: " << endl;
47         lObj.print();
48     }
49 private:
50     string name = "tom";
51     Jack jObj;
52     Lucy lObj;
53 };
54
55 int main()
56 {
57     Tom t;
```

```

58     t.print();
59     return 0;
60 }

```

Fence 248

在上面的例子中 Tom 类分别作为 Jack 类和 Lucy 类的友元类，然后在 Tom 类中定义了 Jack 类和 Lucy 类的对象 jObj 和 lObj，这样我们就可以在 Tom 类中通过这两个类对象直接访问它们各自的私有或者受保护的成员变量或者成员函数了。

24.2 为类模板声明友元

虽然在 C++11 标准中对友元的改进不大，却会带来应用的变化——程序员可以为类模板声明友元了，这在 C++98 中是无法做到的。使用方法如下：

```

1   class Tom;
2
3   template<typename T>
4   class Person
5   {
6       friend T;
7   };
8
9   int main()
10  {
11      Person<Tom> p;
12      Person<int> pp;
13      return 0;
14  }

```

Fence 249

- 第11行：Tom类是Person类的友元
- 第12行：对于int类型的模板参数，友元声明被忽略（第6行）

这样一来，我们就可以在**模板实例化时才确定一个模板类是否有友元**，以及谁是这个模板类的友元。

下面基于一个实际场景来讲解一下如何给模板类指定友元：

假设有一个矩形类，一个圆形类，我们在对其进行了一系列的操作之后，需要验证一下矩形的宽度和高度、圆形的半径是否满足要求，并且要求这个校验操作要在另一个类中完成。

```

1   template<typename T>
2   class Rectangle
3   {
4   public:
5       friend T;
6       Rectangle(int w, int h) : width(w), height(h) {}
7   private:
8       int width;
9       int height;
10  };
11
12  template<typename T>
13  class Circle

```



```

14 {
15 public:
16     friend T;
17     Circle(int r) : radius(r) {}
18 private:
19     int radius;
20 };
21
22 // 校验类
23 class Verify
24 {
25 public:
26     void verifyRectangle(int w, int h, Rectangle<Verify> &r)
27     {
28         if (r.width >= w && r.height >= h)
29         {
30             cout << "矩形的宽度和高度满足条件!" << endl;
31         }
32         else
33         {
34             cout << "矩形的宽度和高度不满足条件!" << endl;
35         }
36     }
37
38     void verifyCircle(int r, Circle<Verify> &c)
39     {
40         if (r >= c.radius)
41         {
42             cout << "圆形的半径满足条件!" << endl;
43         }
44         else
45         {
46             cout << "圆形的半径不满足条件!" << endl;
47         }
48     }
49 };
50
51 int main()
52 {
53     Verify v;
54     Circle<Verify> circle(30);
55     Rectangle<Verify> rect(90, 100);
56     v.verifyCircle(60, circle);
57     v.verifyRectangle(100, 100, rect);
58     return 0;
59 }

```

Fence 250

- 第28行：在Verify类中访问了Rectangle类的私有成员变量
- 第40行：在Verify类中访问了Circle类的私有成员变量

程序输出的结果：

```
1 | 圆形的半径满足条件！
2 | 矩形的宽度和高度不满足条件！
```

Fence 251

在上面的例子中我们定义了两个类模板Rectangle和Circle并且将其模板类型定义为了它们的友元（如果是模板类型是基础类型友元的定义就被忽略了）。在main()函数中测试的时候将Verify类指定为了两个模板类的实际友元类型。这样我们在Verify类中就可以通过Rectangle类和Circle类的实例对象访问它们内部的私有成员变量了。

补充说明：

1. 在上面的测试程序中Rectangle类和Circle类我们没有提供对应的set方法来设置私有成员的值，为了简化程序直接通过构造函数的初始化列表完成了它们的初始化。
2. 在上面的程序中也没有给Rectangle类和Circle类提供get方法，这样如果想要在类外部访问私有（或受保护）成员就只能使用友元了（此处这样处理完全是为了测试的需要）。

25. 强类型枚举

25.1 枚举

25.1.1 枚举的使用

枚举类型是C及C++中一个基本的内置类型，不过也是一个有点“奇怪”的类型。从枚举的本意上来讲，就是要定义一个类别，并穷举同一类别下的个体以供代码中使用。由于枚举来源于C，所以出于设计上的简单的目的，枚举值常常是对应到整型数值的一些名字，比如：

```
1 | // 匿名枚举
2 | enum {Red, Green, Blue};
3 | // 有名枚举
4 | enum Colors{Red, Green, Blue};
```

Fence 252

在枚举类型中的枚举值编译器会默认从0开始赋值，而后依次向下递增，也就是说Red=0，Green=1，Blue=2。

25.1.2 枚举的缺陷

C/C++的enum有个很“奇怪”的设定，就是具名（有名字）的enum类型的名字，以及enum的成员的名字都是全局可见的。这与C++中具名的namespace、class/struct及union必须通过名字::成员名的方式访问相比是格格不入的，编码过程中一不小心程序员就容易遇到问题。比如：

```
1 | enum China {Shanghai, Dongjing, Beijing, Nanjing};
2 | enum Japan {Dongjing, Daban, Hengbin, Fudao};
```

Fence 253

上面定义的两个枚举在编译的时候，编译器会报错，具体信息如下：

```
1 | error C2365: “Dongjing”: 重定义；以前的定义是“枚举数”
```

Fence 254

错误的原因上面也提到了，在这两个具名的枚举中Dongjing是全局可见的，所有编译器就会提示其重定义了。

另外，由于C中枚举被设计为常量数值的“别名”的本性，所以枚举的成员总是可以被隐式地转换为整型，但是很多时候我们并不想这样。

25.2 强类型枚举

25.2.1 优势

针对枚举的缺陷，C++11标准引入了一种新的枚举类型，即枚举类，又称强类型枚举（strong-typed enum）。声明强类型枚举非常简单，只需要在 `enum` 后加上关键字 `class`。比如：

```
1 // 定义强类型枚举
2 enum class Colors{Red, Green, Blue};
```

Fence 255

强类型枚举具有以下几点优势：

- 强作用域，强类型枚举成员的名称不会被输出到其父作用域空间。
 - 强类型枚举只能是有名枚举，如果是匿名枚举会导致枚举值无法使用（因为没有作用域名称）。
- 转换限制，强类型枚举成员的值不可以与整型隐式地相互转换。
- 可以指定底层类型。强类型枚举默认的底层类型为 `int`，但也可以显式地指定底层类型，具体方法为在枚举名称后面加上 `: type`，其中 `type` 可以是除 `wchar_t` 以外的任何整型。比如：

```
1 enum class Colors :char { Red, Green, Blue };
```

Fence 256

`wchar_t` 是什么？

双字节类型，或宽字符类型，是C/C++的一种扩展的存储方式，一般为16位或32位，所能表示的字符数远超 `char` 型。

主要用在国际化程序的实现中，但它不等同于 `unicode` 编码。`unicode` 编码的字符一般以 `wchar_t` 类型存储。

了解了强类型枚举的优势之后，我们再看一段程序：

```
1 enum class China { Shanghai, Dongjing, Beijing, Nanjing, };
2 enum class Japan:char { Dongjing, Daban, Hengbin, Fudao };
3 int main()
4 {
5     int m = Shanghai;           // error
6     int n = China::Shanghai;    // error
7     int j = static_cast<int>(China::Shanghai);           // 强行类型转换，static_cast为
C++的显示类型转换，比(int)这种c语言形式更安全
8     if ((int)China::Beijing >= 2)
9     {
10         cout << "ok!" << endl;
11     }
12     cout << "size1: " << sizeof(China::Dongjing) << endl;
13     cout << "size2: " << sizeof(Japan::Dongjing) << endl;
14     return 0;
15 }
```

Fence 257

- 第5行：该行的代码有两处错误
 - 强类型枚举属于强作用于类型，不能直接使用，枚举值前必须加枚举类型

- 强类型枚举不会进行隐式类型转换，因此枚举值不能直接给int行变量赋值（虽然强类型枚举的枚举值默认就是整形，但其不能作为整形使用）。
- 第6行：语法错误，将强类型枚举值作为整形使用，此处不会进行隐式类型转换
- 第7行：语法正确，强类型枚举值在和整数比较之前做了强制类型转换。
- 第11行：打印的结果为4，强类型枚举底层类型值默认为int，因此占用的内存是4个字节
- 第12行：打印的结果为1，显示指定了强类型枚举值的类型为char，因此占用的内存大小为1个字节，这样我们就可以节省更多的内存空间了。

25.2.2 对原有枚举的扩展

相比于原来的枚举，强类型枚举更像是一个属于C++的枚举。但为了配合新的枚举类型，C++11还对原有枚举类型进行了扩展：

1. 原有枚举类型的底层类型在默认情况下，仍然由编译器来具体指定实现。但也可以跟强类型枚举类一样，显式地由程序员来指定。其指定的方式跟强类型枚举一样，都是枚举名称后面加上：`type`，其中`type`可以是除`wchar_t`以外的任何整型。比如：

```
1 | enum Colors : char { Red, Green, Blue };
```

Fence 258

2. 关于作用域，在C++11中，枚举成员的名字除了会自动输出到父作用域，也可以在枚举类型定义的作用域内有效。比如：

```
1 | enum Colors : char { Red, Green, Blue };
2 | int main()
3 | {
4 |     Colors c1 = Green;           // C++11以前的用法
5 |     Colors c2 = Colors::Green;  // C++11的扩展语法
6 |     return 0;
7 | }
```

Fence 259

上面程序中第4、5行的写法都是合法的。

C++11中对原有枚举类型的这两个扩展都保留了向后兼容性，也方便了程序员在代码中同时操作两种枚举类型。此外，我们在声明强类型枚举的时候，也可以使用关键字`enum struct`。实际上 `enum struct` 和 `enum class` 在语法上没有任何区别（强类型枚举的成员没有公有私有之分，也不会使用模板来支持泛化的声明）。

26. 非受限联合体

26.1 什么是非受限联合体

联合体又叫共用体，我将其称之为`union`，它的使用方式和结构体类似，程序猿可以在联合体内部定义多种不同类型的数据成员，但是这些数据会共享同一块内存空间（也就是如果对多个数据成员同时赋值会发生数据的覆盖）。在某些特定的场景下，通过这种特殊的数据结构我们就可以实现内存的复用，从而达到节省内存空间的目的。

在C++11之前我们使用的联合体是有局限性的，主要有以下三点：

1. 不允许联合体拥有非POD类型的成员

2. 不允许联合体拥有静态成员

3. 不允许联合体拥有引用类型的成员

在新的C++11标准中，取消了关于联合体对于数据成员类型的限定，规定**任何非引用类型都可以成为联合体的数据成员**，这样的联合体称之为**非受限联合体 (Unrestricted Union)**

26.2 非受限联合体的使用

26.2.1 静态类型的成员

对于非受限联合体来说，静态成员有两种分别是**静态成员变量**和**静态成员函数**，我们来看一下下面的代码：

```
1  union Test
2  {
3      int age;
4      long id;
5      // int& tmp = age; // error
6      static char c;
7      static int print()
8      {
9          cout << "c value: " << c << endl;
10         return 0;
11     }
12 };
13 char Test::c;
14 // char Test::c = 'a';
15
16 int main()
17 {
18     Test t;
19     Test t1;
20     t.c = 'b';
21     t1.c = 'c';
22     t1.age = 666;
23     cout << "t.c: " << t.c << endl;
24     cout << "t1.c: " << t1.c << endl;
25     cout << "t1.age: " << t1.age << endl;
26     cout << "t1.id: " << t1.id << endl;
27     t.print();
28     Test::print();
29     return 0;
30 }
```

Fence 260

执行程序输出的结果如下：

```
1  t.c: c
2  t1.c: c
3  t1.age: 666
4  t1.id: 666
5  c value: c
6  c value: c
```

- 第5行：语法错误，非受限联合体中不允许出现引用类型⁶¹
- 第6行：非受限联合体中的静态成员变量
 - 需要在非受限联合体外部声明（第13行）或者初始化（第14行）之后才能使用
 - 通过打印的结果可以发现18、19行的t和t1对象共享这个静态成员变量（和类 class/struct 中的静态成员变量的使用是一样的）。
- 第7行：非受限联合体中的静态成员函数
 - 在静态函数print()只能访问非受限联合体Test中的静态变量，对于非静态成员变量（age、id）是无法访问的。
 - 调用这个静态方法可以通过对象（第27行）也可以通过类名（第28行）实现。
- 第24、25、26行：通过打印的结果可以得出结论**在非受限联合体中静态成员变量和非静态成员变量使用的不是同一块内存。**

26.2.2 非POD类型成员

在 C++11 标准中会默认删除一些非受限联合体的默认函数。比如，非受限联合体有一个**非 POD 的成员**，而该**非 POD 成员类型拥有 非平凡的构造函数**，那么**非受限联合体的默认构造函数将被编译器删除**。其他的特殊成员函数，例如**默认拷贝构造函数、拷贝赋值操作符以及析构函数等**，也将遵从此规则。下面来举例说明：

```

1  union Student
2  {
3      int id;
4      string name;
5  };
6
7  int main()
8  {
9      Student s;
10     return 0;
11 }
```

Fence 262

编译程序会看到如下的错误提示：

```

1  warning C4624: "Student": 已将析构函数隐式定义为"已删除"
2  error C2280: "Student::Student(void)": 尝试引用已删除的函数
```

Fence 263

上面代码中的非受限联合体Student中拥有一个非PDO类型的成员string name，string 类中有非平凡构造函数，因此Student的构造函数被删除（通过警告信息可以得知它的析构函数也被删除了）导致对象无法被成功创建出来。解决这个问题的办法就是**由程序猿自己为非受限联合体定义构造函数**，在定义构造函数的时候我们需要用到**定位放置 new**操作。

26.2.2.1 placement new

一般情况下，使用new申请空间时，是从系统的**堆 (heap)** 中分配空间，申请所得的空间的位置是根据当时的内存的实际使用情况决定的。但是，在某些特殊情况下，可能需要在已分配的特定内存创建对象，这种操作就叫做**placement new即定位放置 new**。

定位放置new操作的语法形式不同于普通的new操作：

- 使用new申请内存空间：Base* ptr = new Base;
- 使用定位放置new申请内存空间：

```
1 | ClassName* ptr = new (定位的内存地址)ClassName;
```

Fence 264

```
1 | #include <iostream>
2 | using namespace std;
3 |
4 | class Base
5 | {
6 | public:
7 |     Base() {}
8 |     ~Base() {}
9 |     void print()
10 |    {
11 |        cout << "number value: " << number << endl;
12 |    }
13 | private:
14 |     int number;
15 | };
16 |
17 | int main()
18 | {
19 |     int n = 100;
20 |     Base* b = new (&n)Base;
21 |     b->print();
22 |     return 0;
23 | }
```

Fence 265

程序运行输出的结果为:

```
1 | number value: 100
```

Fence 266

在程序的第20行，使用定位放置的方式为指针b申请了一块内存，也就是说此时**指针 b指向的内存地址和变量 n对应的内存地址是同一块（栈内存）**，而在Base类中成员变量 number的起始地址和Base对象的起始地址是相同的，所以打印出 number 的值为100也就是整形变量 n 的值。

最后，给大家总结一下关于placement new的一些细节：

1. 使用定位放置new操作，既可以在栈(stack)上生成对象，也可以在堆（heap）上生成对象，这取决于定位时指定的内存地址是在堆还是在栈上。
2. 从表面上看，定位放置new操作是申请空间，其本质是利用已经申请好的空间，真正的申请空间的工作是在此之前完成的。
3. 使用定位放置new 创建对象时会自动调用对应类的构造函数，但是由于对象的空间不会自动释放，如果需要释放堆内存必须显示调用类的析构函数。
4. 使用定位放置new操作，我们可以反复动态申请到同一块堆内存，这样可以避免内存的重复创建销毁，从而提高程序的执行效率（比如网络通信中数据的接收和发送）。

26.2.2.2 自定义非受限联合体构造函数

掌握了placement new的使用，我们通过一段程序来演示一下如果在非受限联合体中自定义构造函数：

```
1  class Base
2  {
3  public:
4      void setText(string str)
5      {
6          notes = str;
7      }
8      void print()
9      {
10         cout << "Base notes: " << notes << endl;
11     }
12 private:
13     string notes;
14 };
15
16 union Student
17 {
18     Student()
19     {
20         new (&name)string;
21     }
22     ~Student() {}
23
24     int id;
25     Base tmp;
26     string name;
27 };
28
29 int main()
30 {
31     Student s;
32     s.name = "蒙奇·D·路飞";
33     s.tmp.setText("我是要成为海贼王的男人!");
34     s.tmp.print();
35     cout << "Student name: " << s.name << endl;
36     return 0;
37 }
```

Fence 267

程序打印的结果如下：

```
1  Base notes: 我是要成为海贼王的男人!
2  Student name: 我是要成为海贼王的男人!
```

Fence 268

我们在上面的程序里边给非受限联合体显示的指定了构造函数和析构函数，在程序的第31行需要创建一个非受限联合体对象，这时便调用了联合体内部的构造函数，在构造函数的第20行通过定位放置 new的方式将构造出的对象地址定位到了联合体的成员string name的地址上了，这样联合体内部其他非静态成员也就可以访问这块地址了（通过输出的结果可以看到对联合体内的tmp对象赋值，会覆盖name对象中的数据，在Student中 id、tmp 和 name 成员变量都位于同一块内存空间中）。

26.2.2.3 匿名的非受限联合体

一般情况下我们使用的非受限联合体都是具名的（有名字），但是我们可以定义匿名的非受限联合体，一个比较实用的场景就是配合着类的定义使用。我们来设定一个场景：

```
1  木叶村要进行第99次人口普查，人员的登记方式如下：
2      - 学生只需要登记所在学校的编号
3      - 本村学生以外的人员需要登记其身份证号码
4      - 本村外来人员需要登记户口所在地+联系方式
```

Fence 269

```
1  // 外来人口信息
2  struct Foreigner
3  {
4      Foreigner(string s, string ph) : addr(s), phone(ph) {}
5      string addr;
6      string phone;
7  };
8
9  // 登记人口信息
10 class Person
11 {
12 public:
13     enum class Category : char {Student, Local, Foreign};
14     Person(int num) : number(num), type(Category::Student) {}
15     Person(string id) : idNum(id), type(Category::Local) {}
16     Person(string addr, string phone) : foreign(addr, phone), type(Category::Foreign)
17 {}
18
19 ~Person() {}
20
21 void print()
22 {
23     cout << "Person category: " << (int)type << endl;
24     switch (type)
25     {
26     case Category::Student:
27         cout << "Student school number: " << number << endl;
28         break;
29     case Category::Local:
30         cout << "Local people ID number: " << idNum << endl;
31         break;
32     case Category::Foreign:
33         cout << "Foreigner address: " << foreign.addr
34             << ", phone: " << foreign.phone << endl;
35         break;
36     default:
```

```

35         break;
36     }
37 }
38
39 private:
40     Category type;
41     union
42     {
43         int number;
44         string idNum;
45         Foreigner foreign;
46     };
47 };
48
49 int main()
50 {
51     Person p1(9527);
52     Person p2("1101122022x");
53     Person p3("砂隐村村北", "1301810001");
54     p1.print();
55     p2.print();
56     p3.print();
57     return 0;
58 }

```

Fence 270

根据需求我们将木叶村的人口分为了三类并通过枚举记录了下来，在**Person类中添加了一个匿名的非受限联合体用来存储人口信息**，仔细分析之后就会发现这种处理方式的优势非常明显：尽可能地节省了内存空间。

- Person类可以直接访问匿名非受限联合体内部的数据成员。
- 不使用匿名非受限联合体申请的内存空间等于 number、idNum、foreign 三者内存之和。
- 使用匿名非受限联合体之后number、idNum、foreign 三者共用同一块内存。